第四讲
深度学习中的连续动力学机制

肖桐

xiaotong@mail.neu.edu.cn

东北大学 计算机科学与工程学院 人工智能系 自然语言处理实验室



東北大學
Northeastern University



深度学习中的连续动力学机制

4.1 连续动力学的数学基础

4.2 表示空间中的连续动力学

4.3 确定性概率动力学

4.4 随机概率动力学

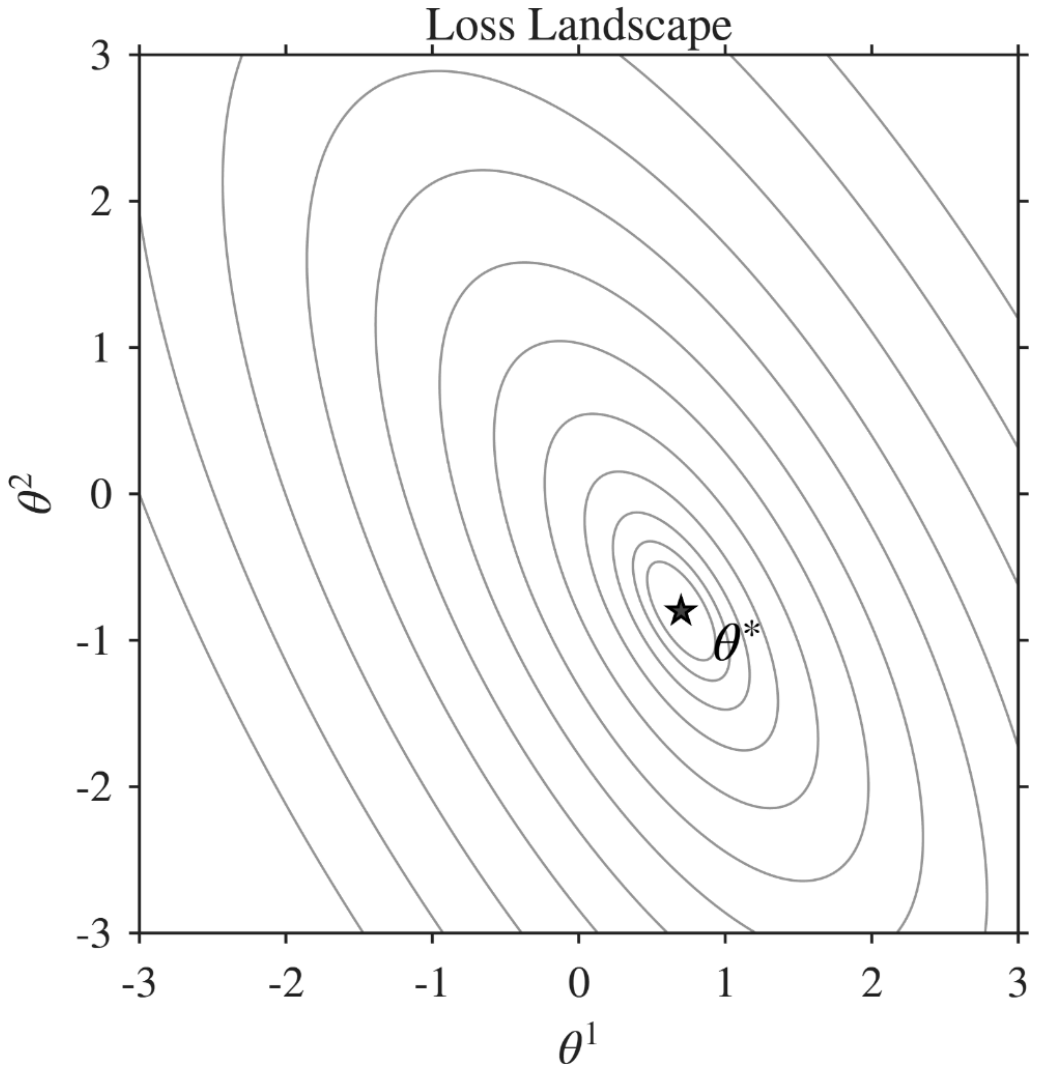
4.5 离散状态上的连续时间动力学

机器学习与无约束优化

- 从优化理论视角出发，机器学习任务通常被视为求解一个 **无约束优化** 问题：

$$\min_{\theta} L(\theta)$$

  - $L(\theta)$：$\mathbb{R}^2 \to \mathbb{R}$，损失函数



自变量 θ 在损失函数 L 的定义域内取值无约束

Vandenberghe L, Boyd S. Convex optimization[M]. Cambridge: Cambridge university press, 2004.

机器学习与无约束优化

- 从优化理论视角出发，机器学习任务通常被视为求解一个 无约束优化 问题：

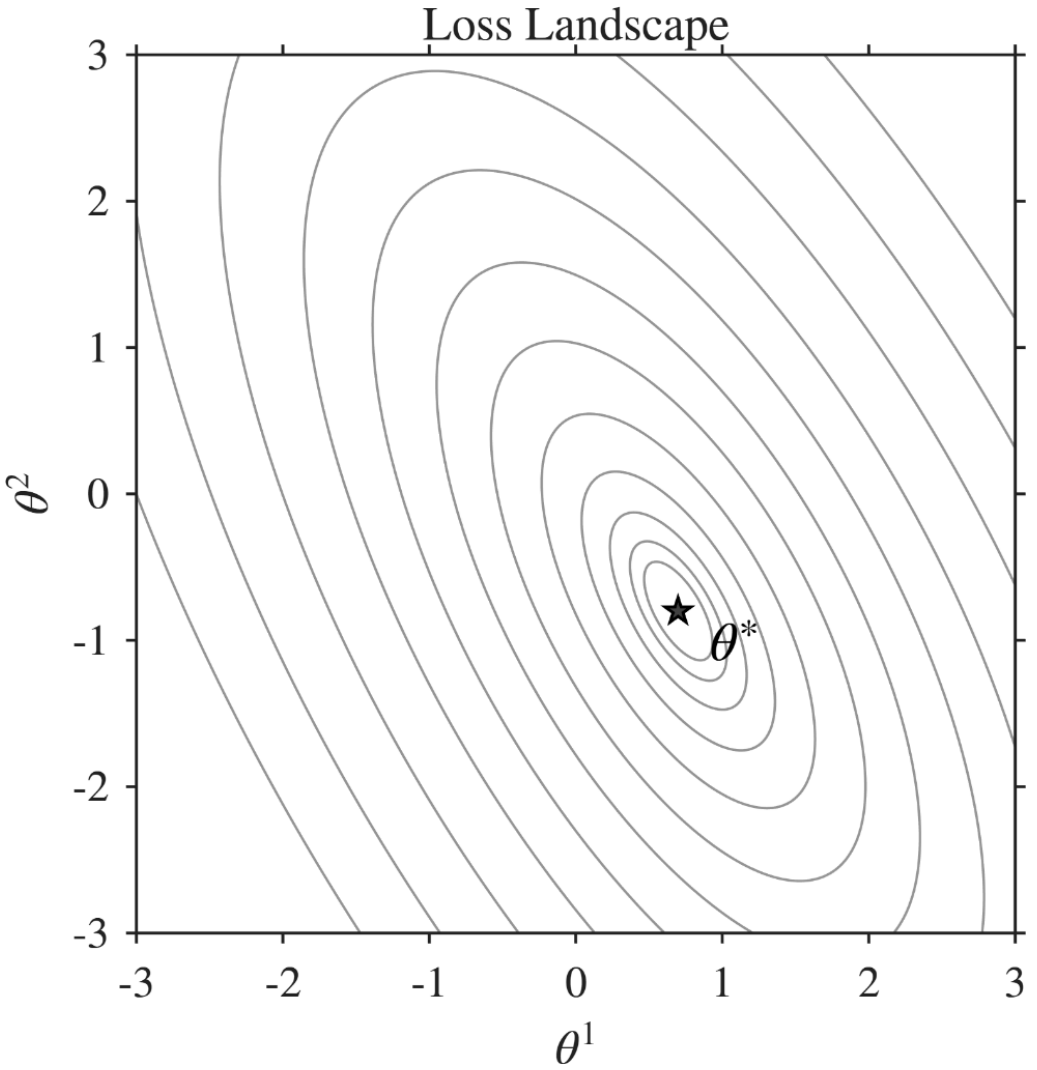
$$\min_{\theta} L(\theta)$$

  - $L(\theta)$：$\mathbb{R}^2 \to \mathbb{R}$，损失函数

- 在局部，$L(\theta)$ 可看成可微凸函数，假设存在局部最优点 θ^*，其满足充要条件：

$$\nabla L(\theta^*) = 0$$

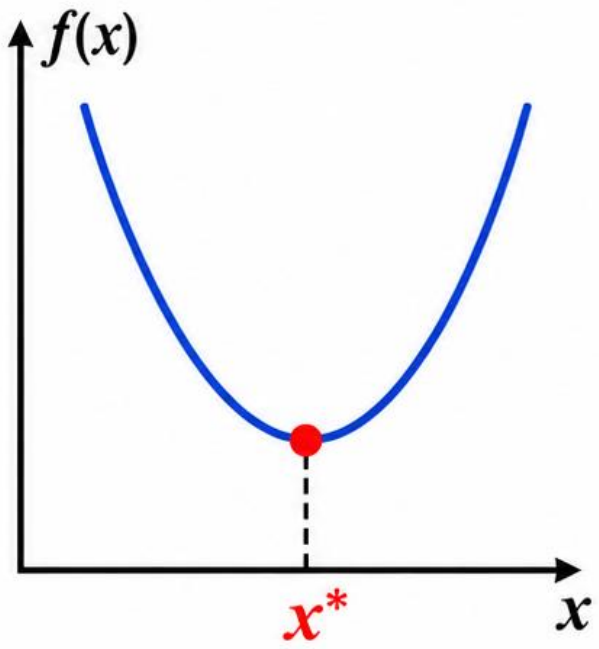
  - ∇：微分算子



自变量 θ 在损失函数 L 的定义域内取值无约束

Vandenberghe L, Boyd S. Convex optimization[M]. Cambridge: Cambridge university press, 2004.

求解无约束优化问题 $\nabla L(\theta) = 0$ 的两种方法



$$f(x) = x^2 + 2x + 5$$

$$f'(x) = 2x + 2$$

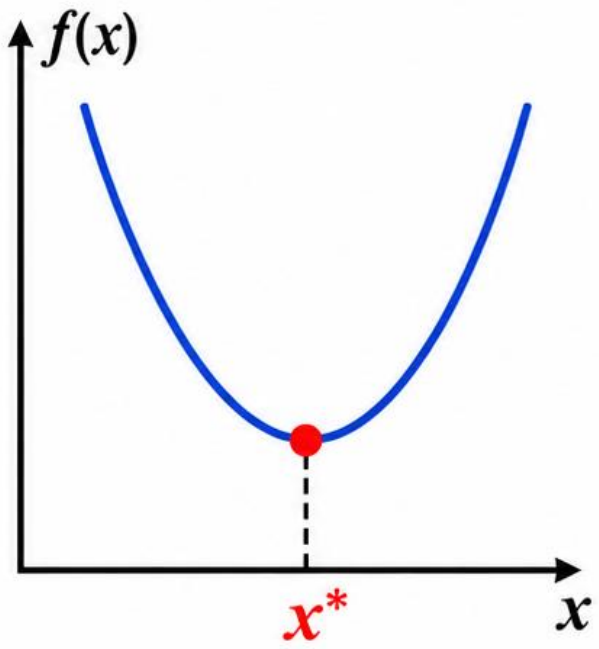
$$x^* = -1$$

$$f(x*) = 4$$

求解析解

Vandenberghe L, Boyd S. Convex optimization[M]. Cambridge: Cambridge university press, 2004.

求解无约束优化问题 $\nabla L(\theta) = 0$ 的两种方法



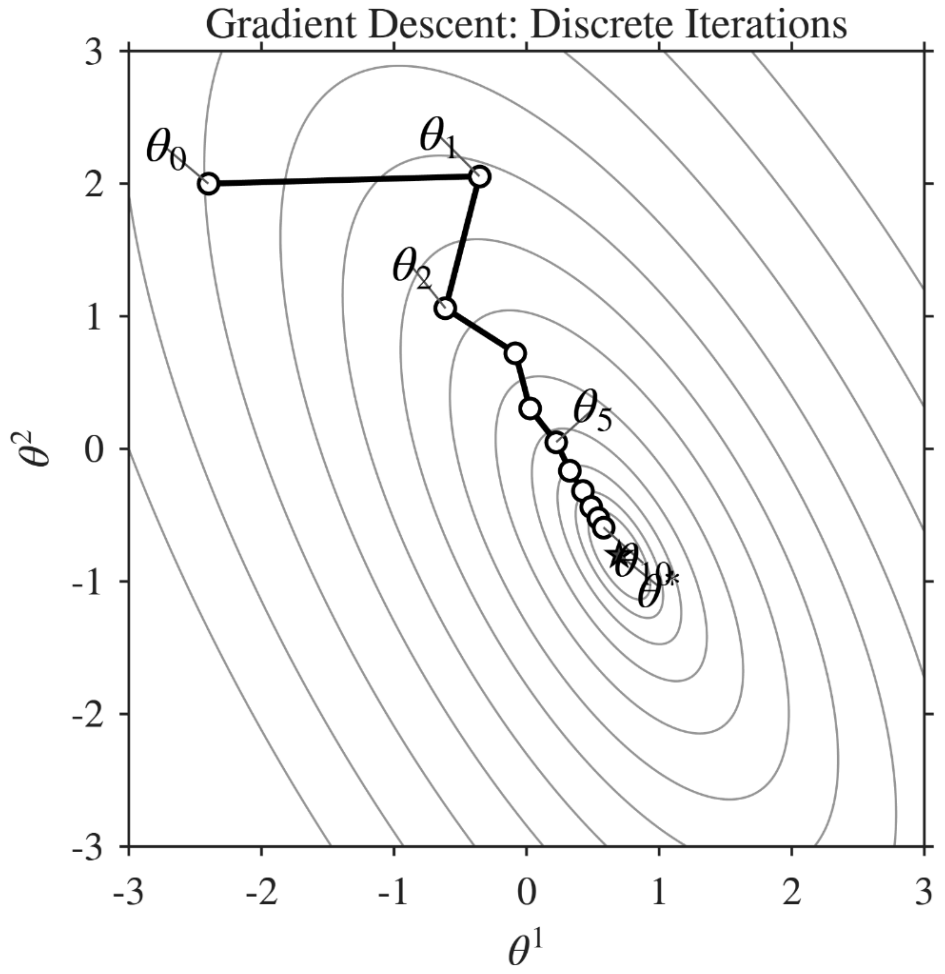
$$f(x) = x^2 + 2x + 5$$

$$f'(x) = 2x + 2$$

$$x^* = -1$$

$$f(x*) = 4$$

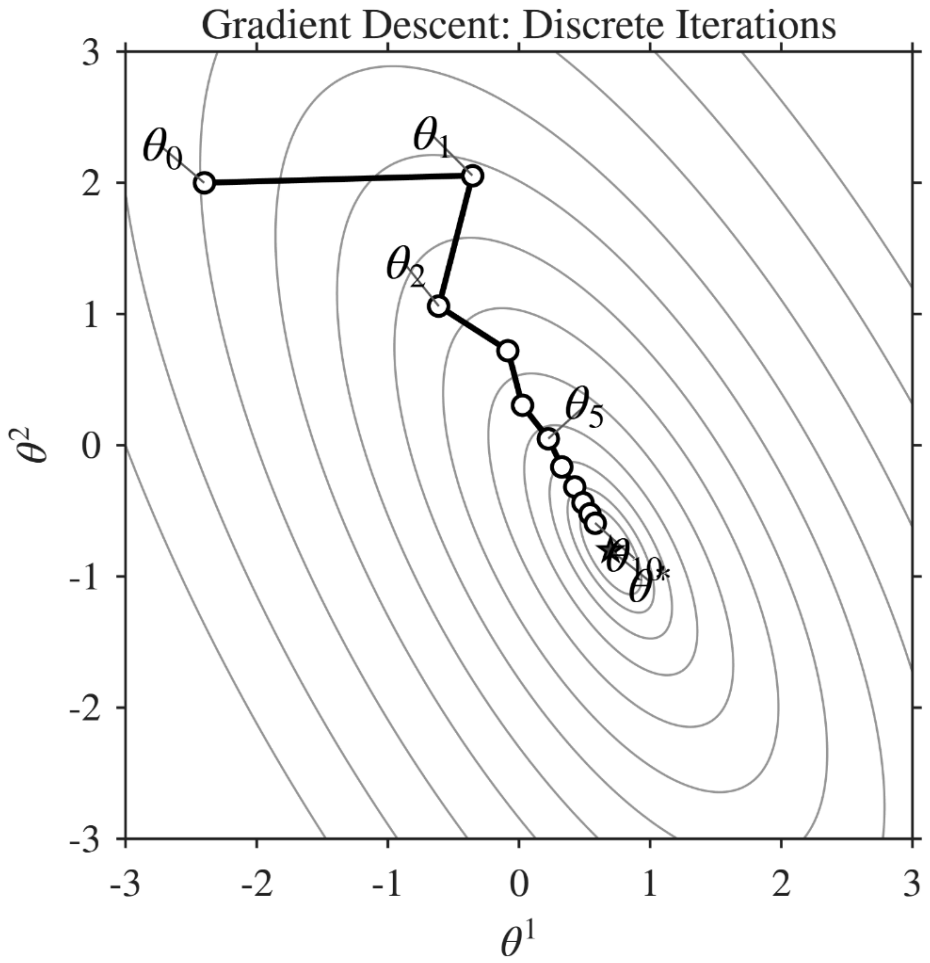
求解析解



迭代算法

Vandenberghe L, Boyd S. Convex optimization[M]. Cambridge: Cambridge university press, 2004.

迭代算法的离散更新



用迭代算法计算点列 $\theta_0, \theta_1, \ldots \in \mathbb{R}^2$

这样的离散点列被称为迭代算法的 **离散更新**

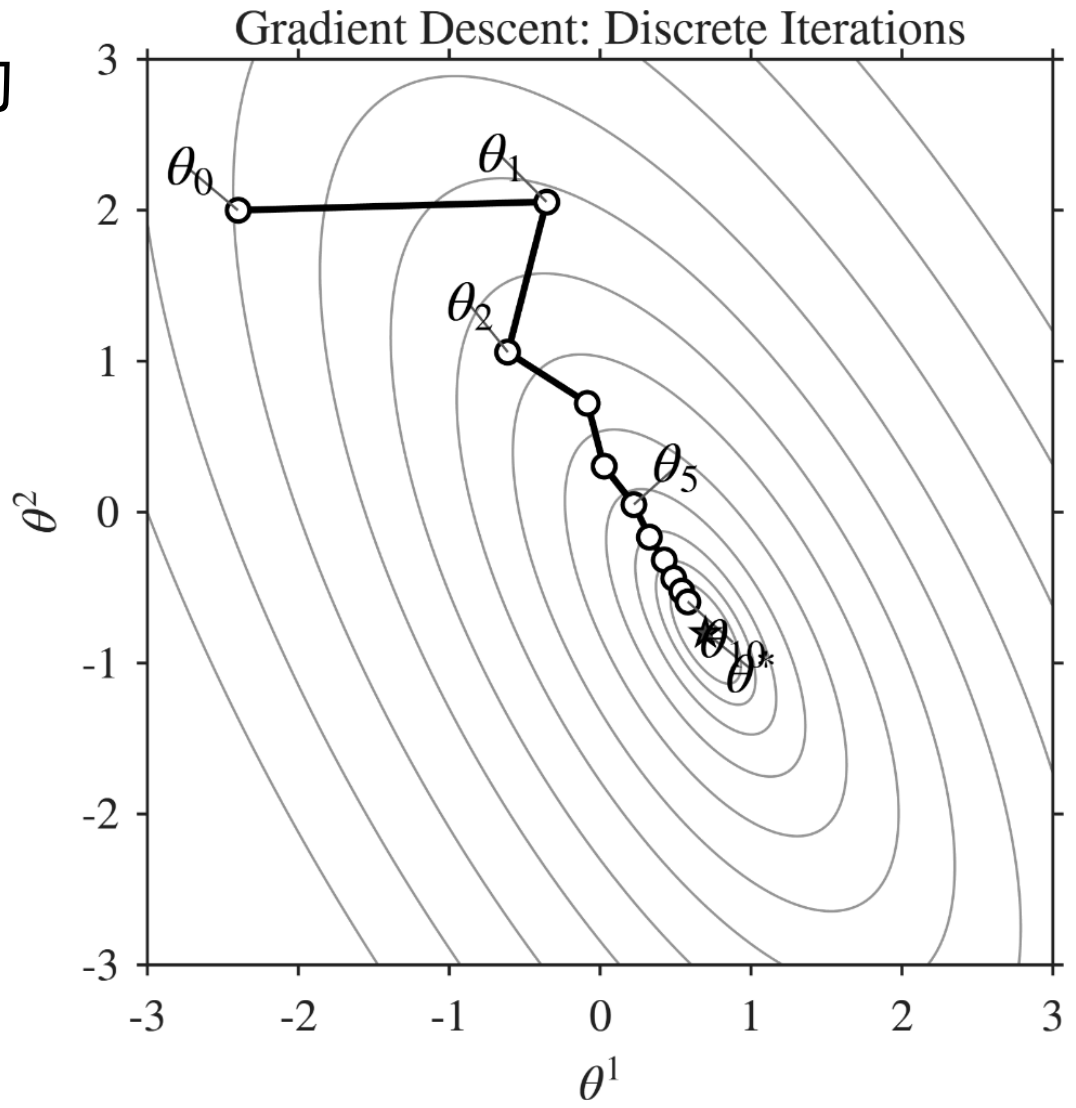
Vandenberghe L, Boyd S. Convex optimization[M]. Cambridge: Cambridge university press, 2004.

迭代算法的例子：梯度下降法

- 在迭代算法中，以梯度负方向为搜索方向的下降方法称为 **梯度下降法**：

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_k)$$

- θ_k：第k次迭代的模型参数
- η：学习率，用于控制每次更新的幅度
- ∇_θ：微分算子，对 θ 求微分
- $L(\theta_k)$：训练损失



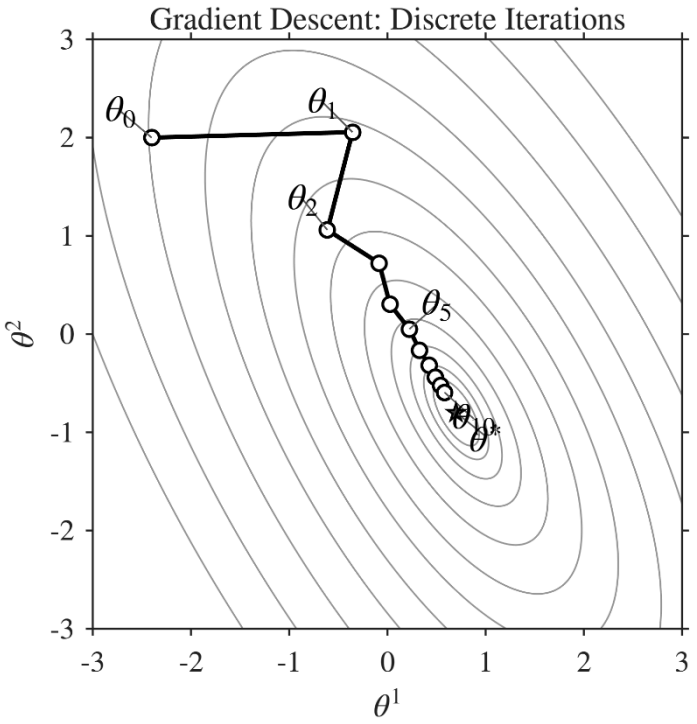
Vandenberghe L, Boyd S. Convex optimization[M]. Cambridge: Cambridge university press, 2004.

从离散更新到连续演化

将梯度下降改写成 **平均变化率** 形式

令 $t_k = k\eta,\ t_{k+1} - t_k = \eta,$ 则有：

$$\frac{\theta_{k+1} - \theta_k}{t_{k+1} - t_k} = -\nabla_\theta L(\theta_k)$$

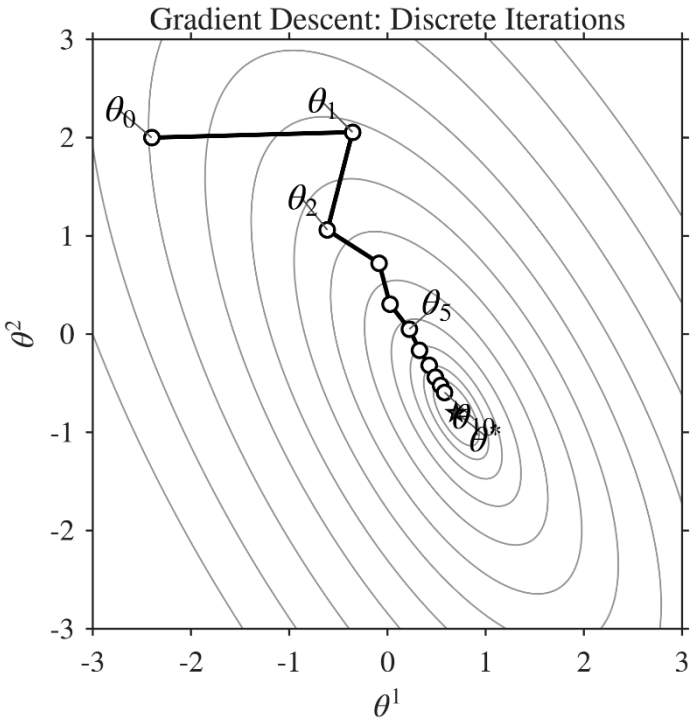


从离散更新到连续演化

将梯度下降改写成 平均变化率 形式

令 $t_k = k\eta,\ t_{k+1} - t_k = \eta,$ 则有:

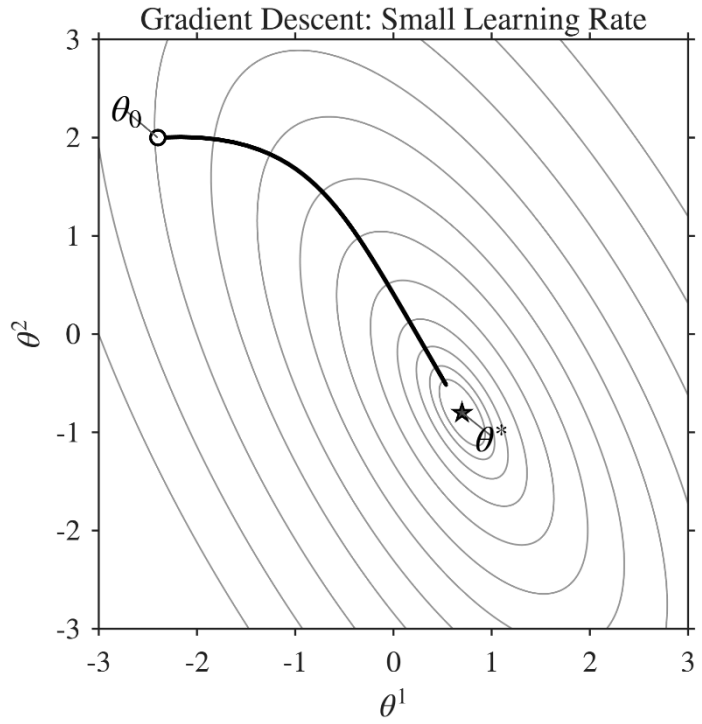
$$\frac{\theta_{k+1} - \theta_k}{t_{k+1} - t_k} = -\nabla_\theta L(\theta_k)$$



平均变化率过渡为 **瞬时变化率**

当学习率 η 趋近于 0，则有:

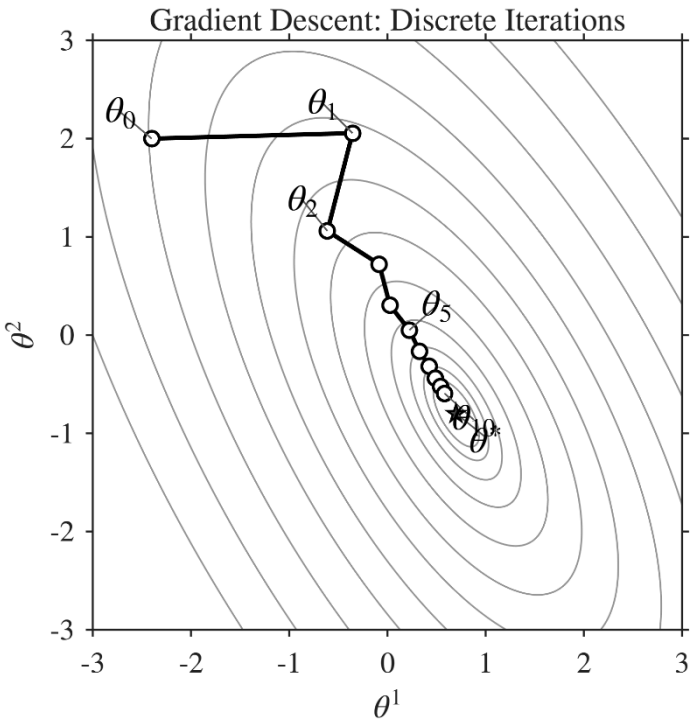
$$\frac{\mathrm{d}\theta(t)}{\mathrm{d}t} = -\nabla_\theta L\big(\theta(t)\big)$$

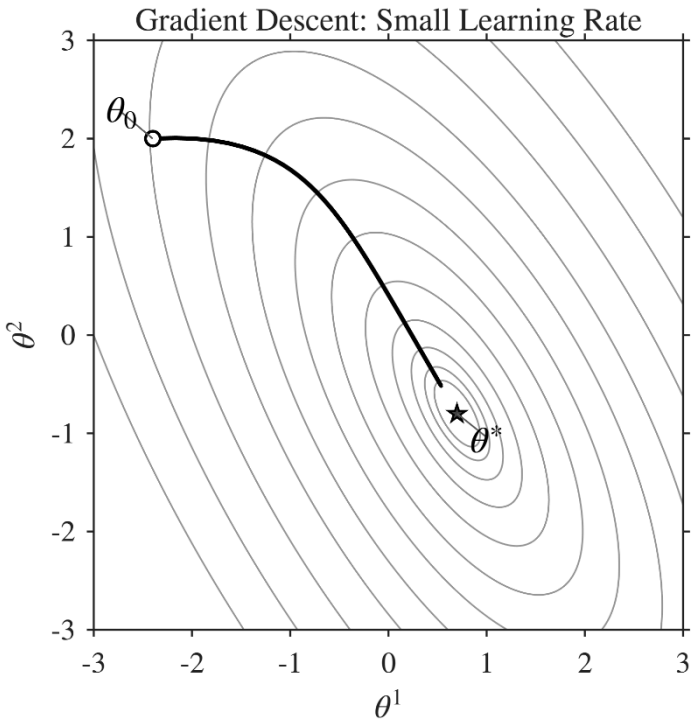


从离散更新到连续演化

$$\frac{\theta_{k+1}-\theta_k}{t_{k+1}-t_k}=-\nabla_\theta L(\theta_k)$$

$$\frac{\mathrm{d}\theta(t)}{\mathrm{d}t}=-\nabla_\theta L\big(\theta(t)\big)$$



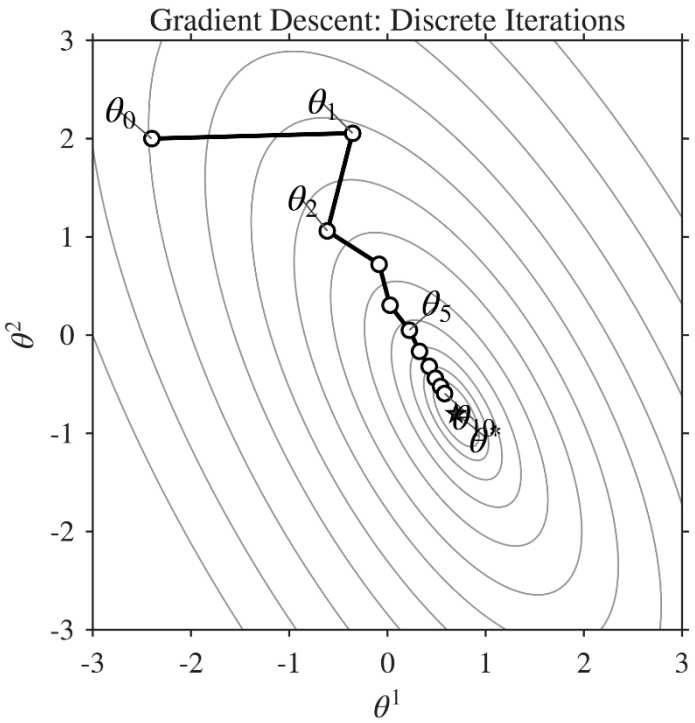


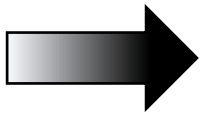
把 **离散序列 θ_k** 看作一条 **连续参数轨迹 $\theta(t)$** 在时刻 t_k 上的采样

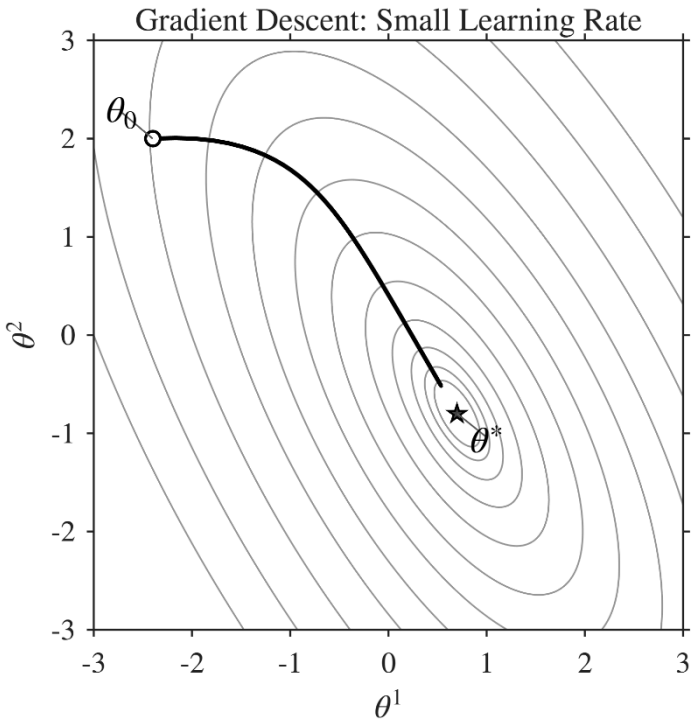
从离散更新到连续演化

$$\frac{\theta_{k+1}-\theta_k}{t_{k+1}-t_k}=-\nabla_\theta L(\theta_k)$$

$$\frac{\mathrm{d}\theta(t)}{\mathrm{d}t}=-\nabla_\theta L\big(\theta(t)\big)$$





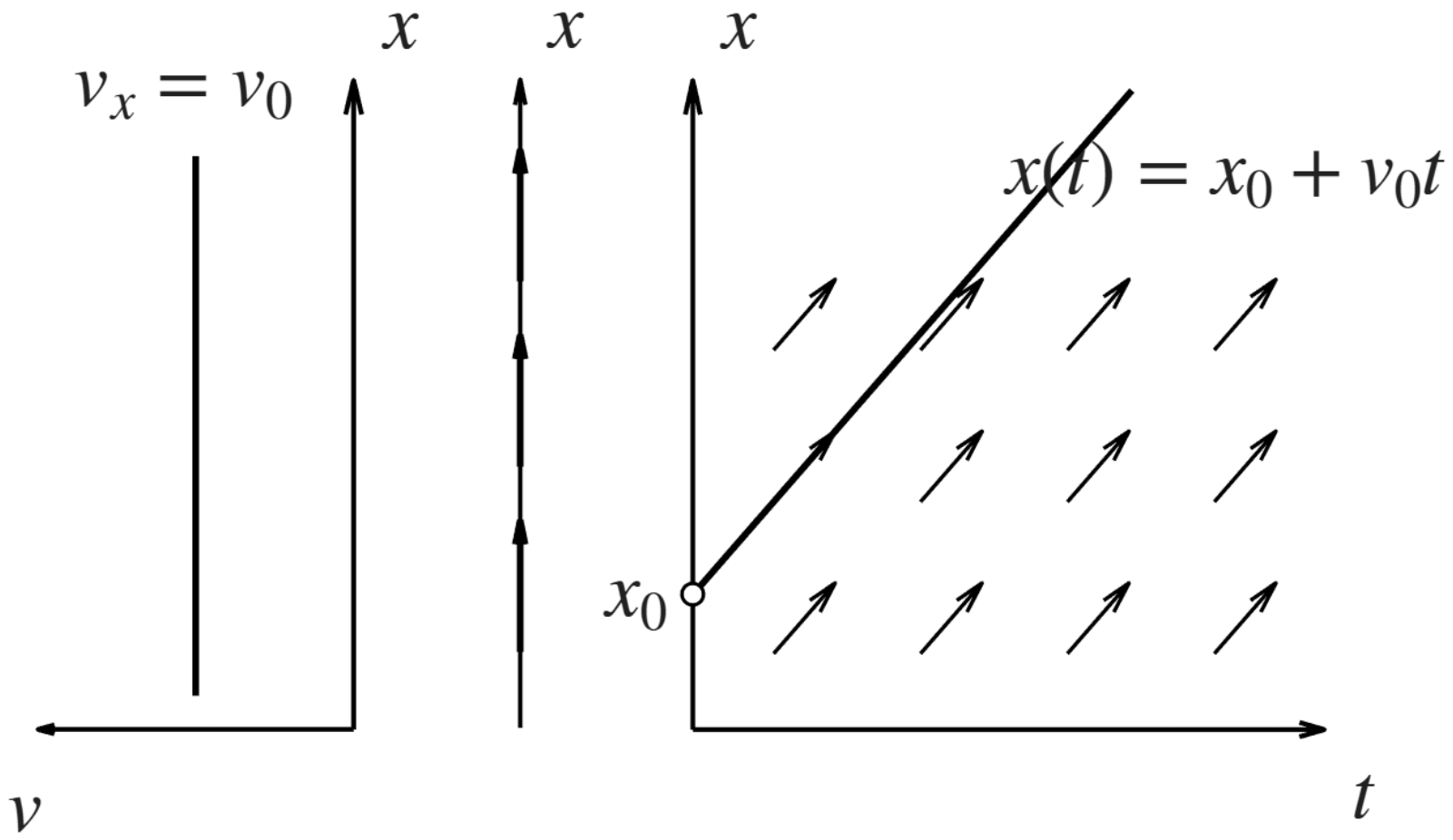


离散更新

连续演化

把 离散序列 θ_k 看作一条 连续参数轨迹 $\theta(t)$ 在时刻 t_k 上的采样

从匀速直线运动到常微分方程

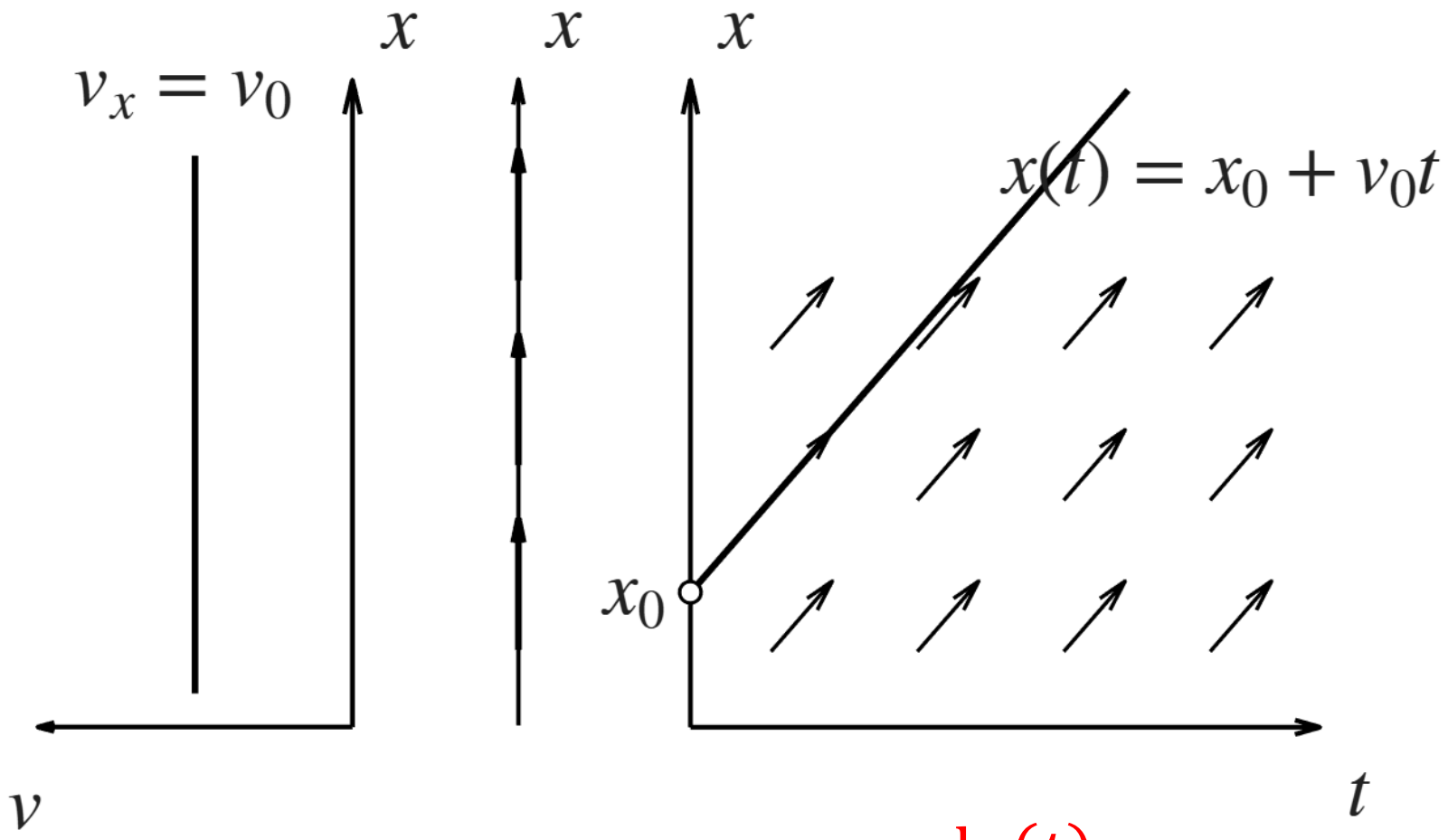


$$x(t) = x_0 + v_0 t$$

从 t_0 时刻，质点从位置 x_0 以速度 v_0 做 **匀速直线运动**

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

从匀速直线运动到常微分方程

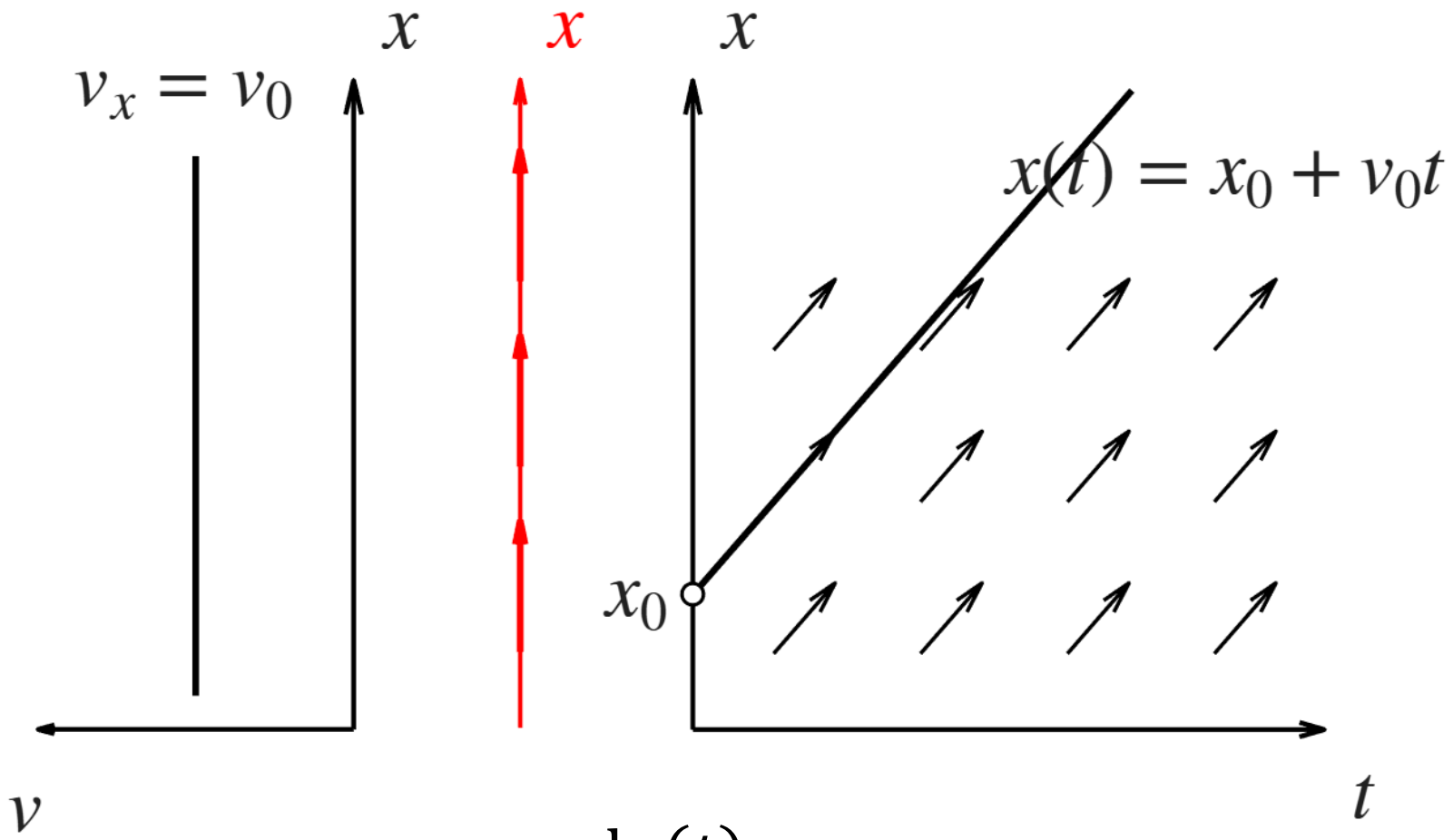


$$x(t) = x_0 + v_0 t \quad \Rightarrow \quad \frac{\mathrm{d}x(t)}{\mathrm{d}t} = v$$

改写成 **微分方程** 的形式：状态函数 $x(t)$ 对时间 t 的瞬时变化率为常数 v

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

微分方程的状态空间

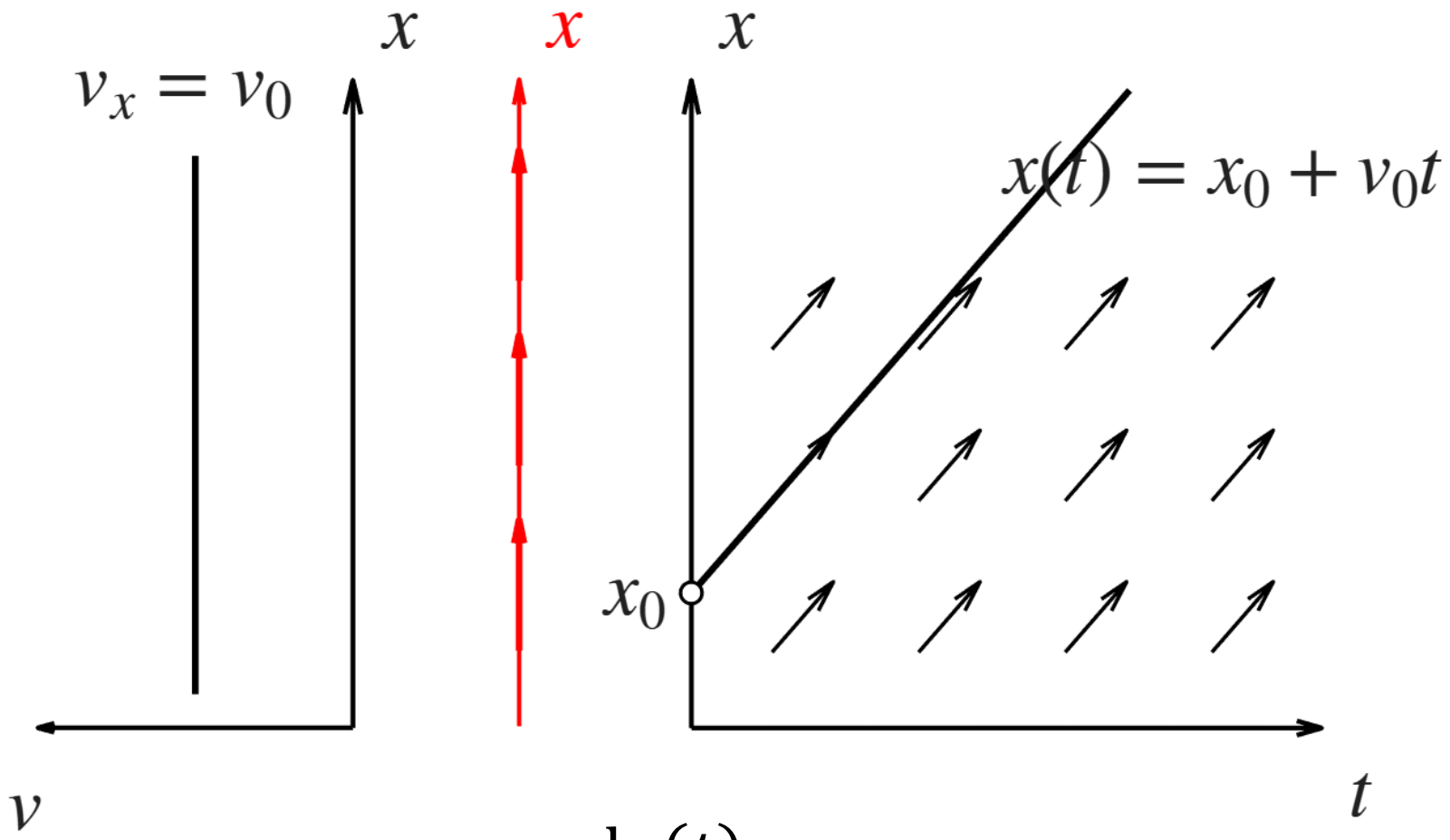


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = v$$

未知函数 x 所有可能状态的集合称为它的 **状态空间**

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

微分方程的状态空间

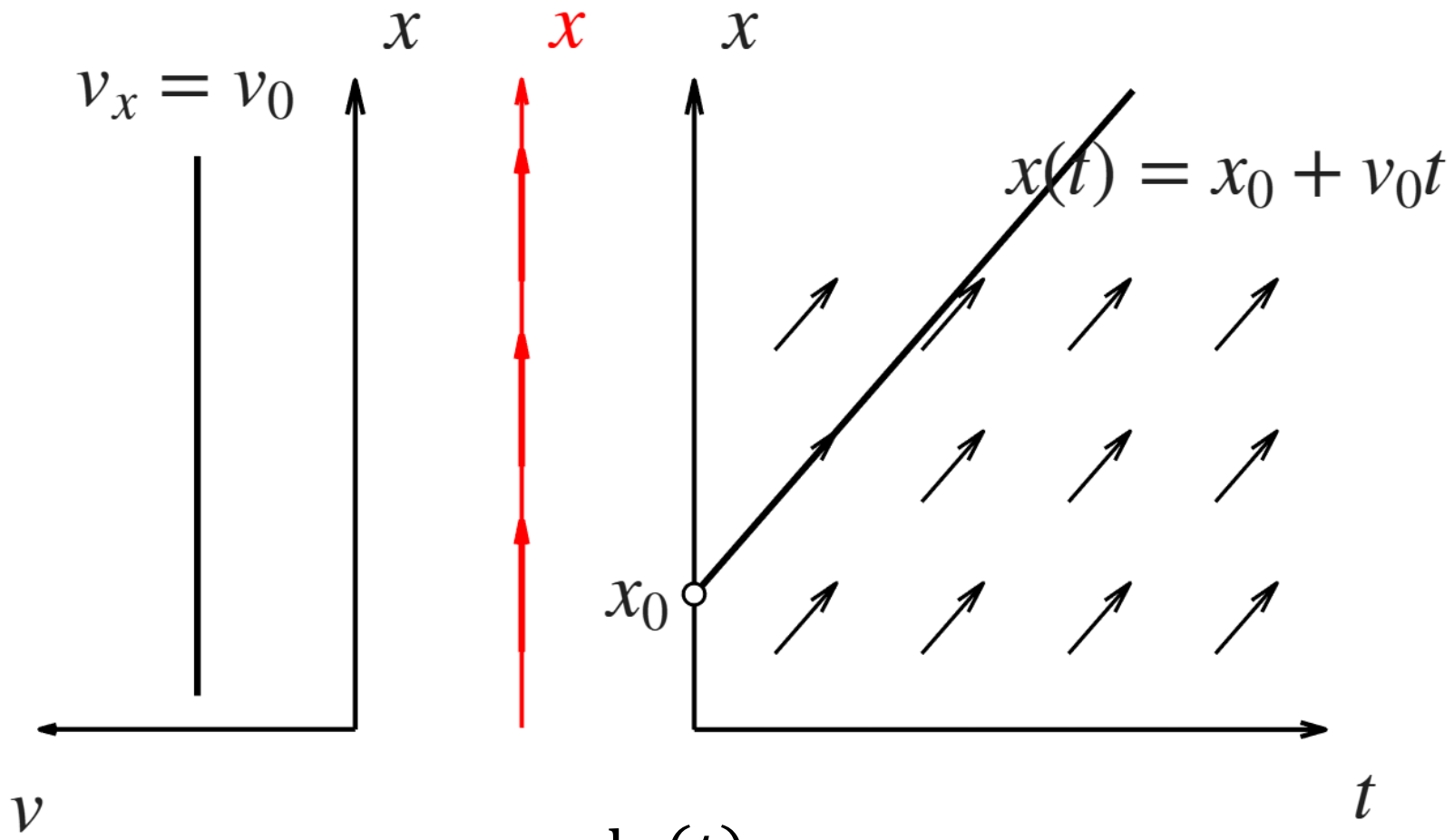


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = v$$

$x \in \mathbb{R}$，其所有可能状态的集合表示为 **实数轴**

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

微分方程的向量场

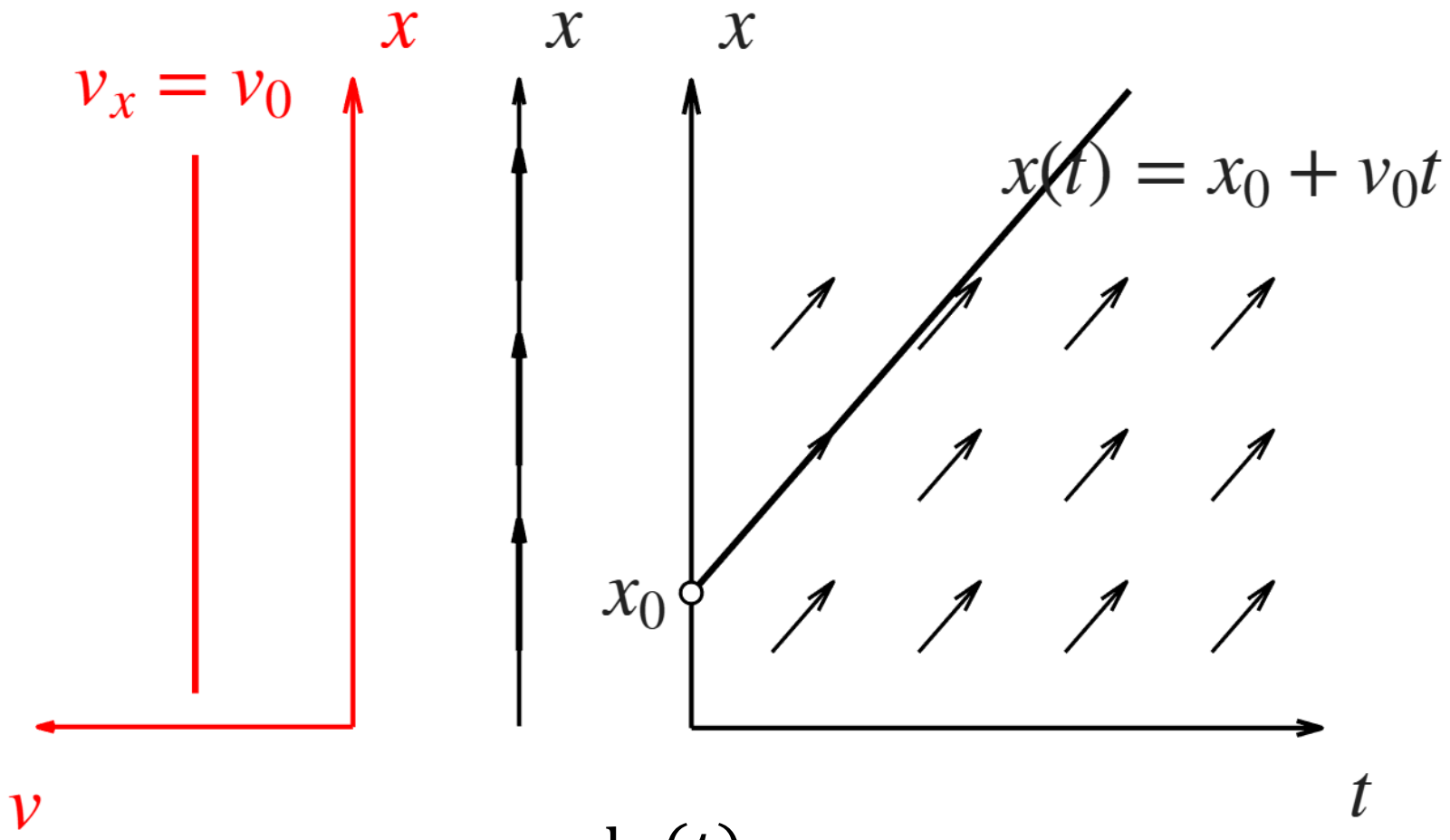


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = v$$

每个状态 $x \in \mathbb{R}$ 都伴随一个从 x 点出发的向量 v，这就在 $\mathbb{R}$ 上定义了一个 **向量场**

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

变化率为常数的微分方程

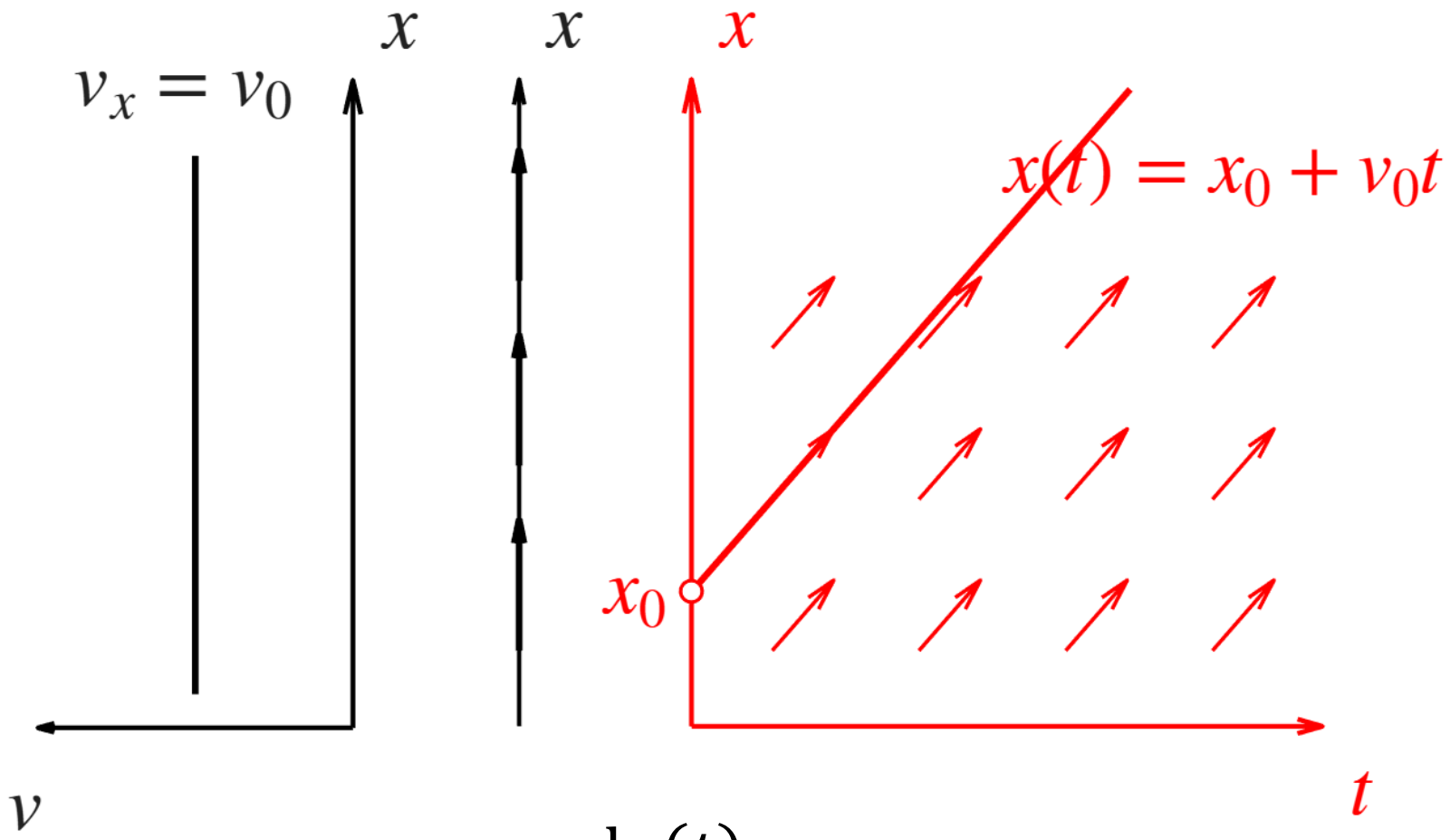


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = v$$

在 (v, x) 状态空间中，**速度始终保持为常数** $v_x = v_0$

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

微分方程的扩展向量场

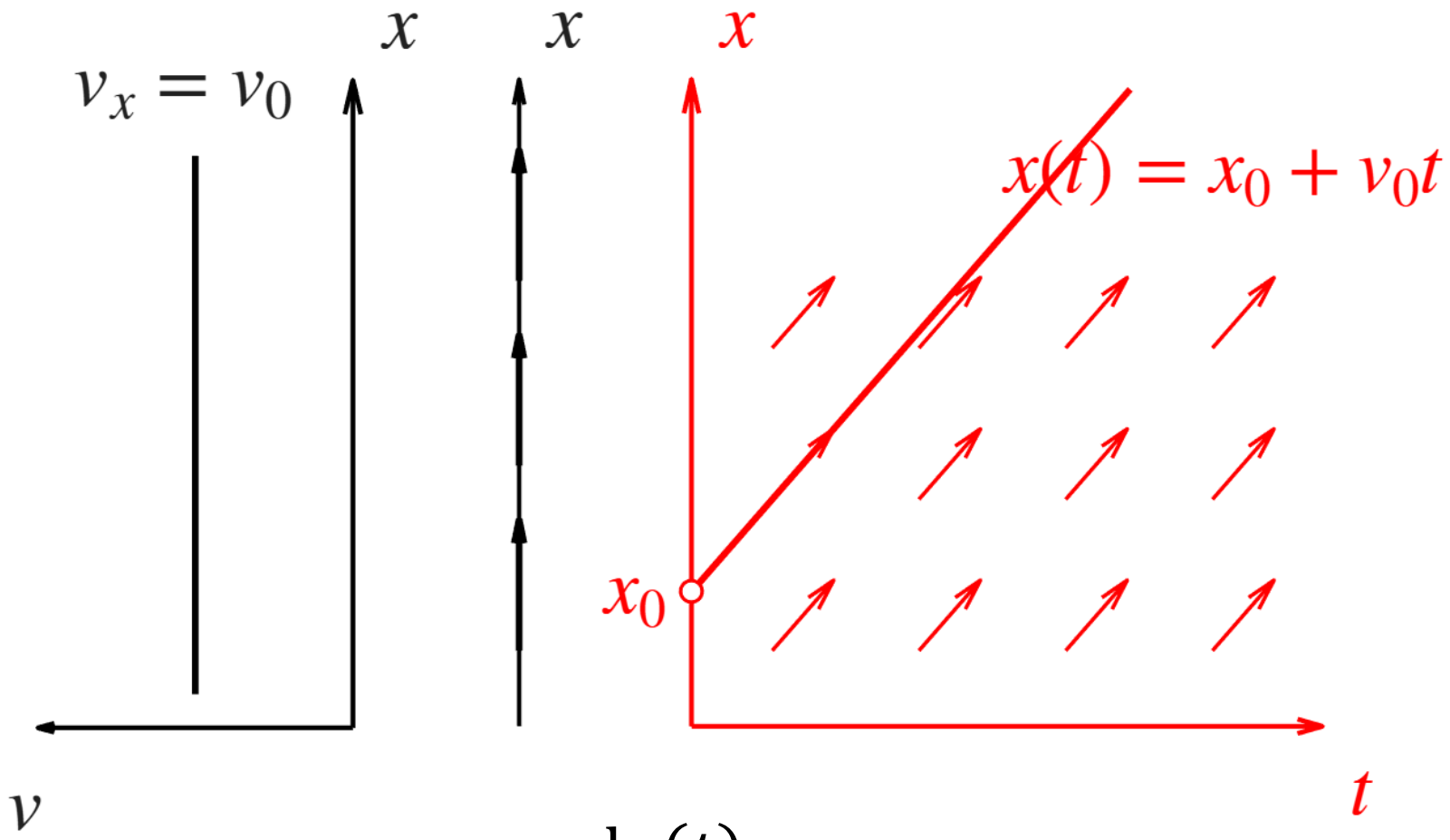


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = v$$

将时间 t 增广为一个状态变量，取时间分量的变化率为 1，得到 **扩展向量场** $(1, v)$

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

微分方程的轨迹

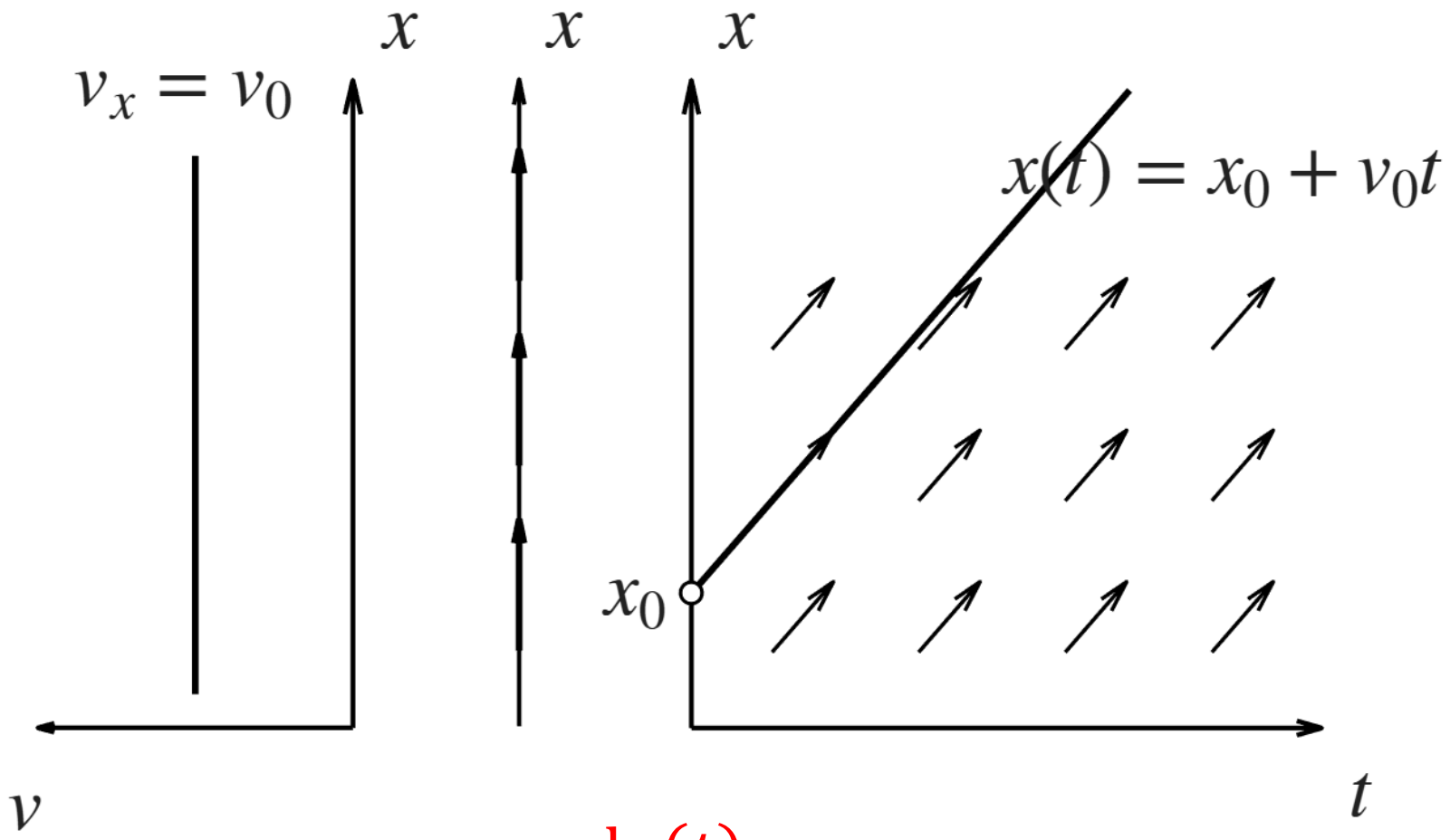


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = v$$

初始状态 x_0 在时间 t 的推进下不断演化，并描出 **轨迹** $x(t) = x_0 + v_0 t$

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

常微分方程的定义

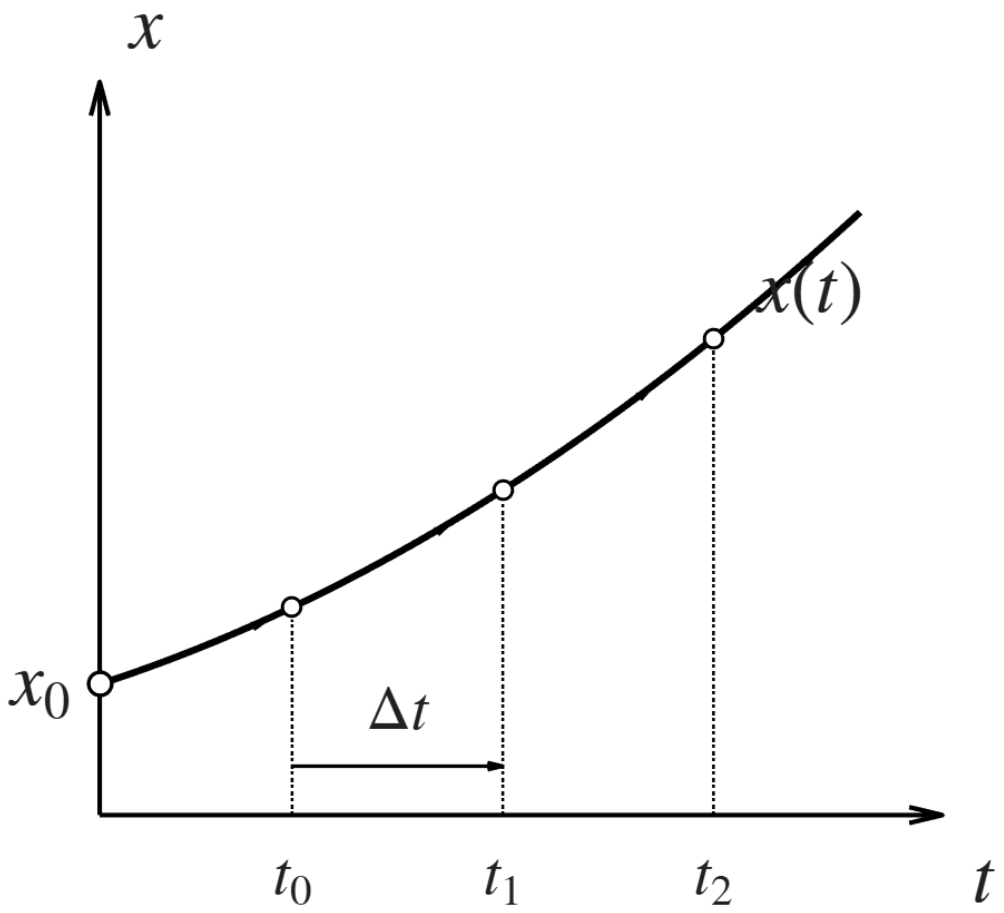


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = v$$

常微分方程：未知函数 x 的变化只与单个自变量有关的微分方程

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

从变化率到动力系统

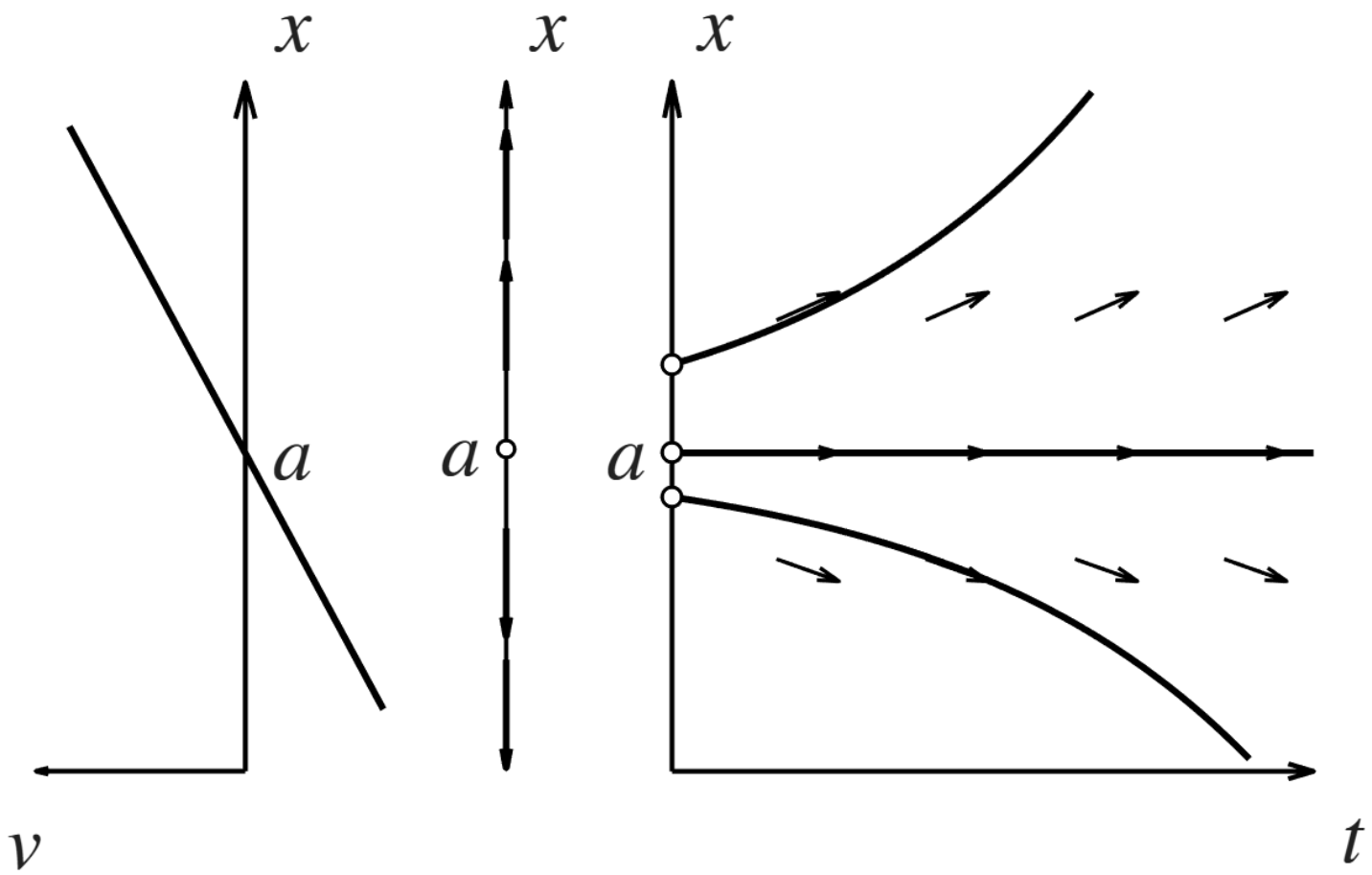


$$x(t+\Delta t) \approx x(t) + \frac{\mathrm{d}x(t)}{\mathrm{d}t}\Delta t$$

常微分方程给出状态的 **变化率**，决定状态随时间推进的演化，描述一个 **动力系统**

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

变化率由什么决定?

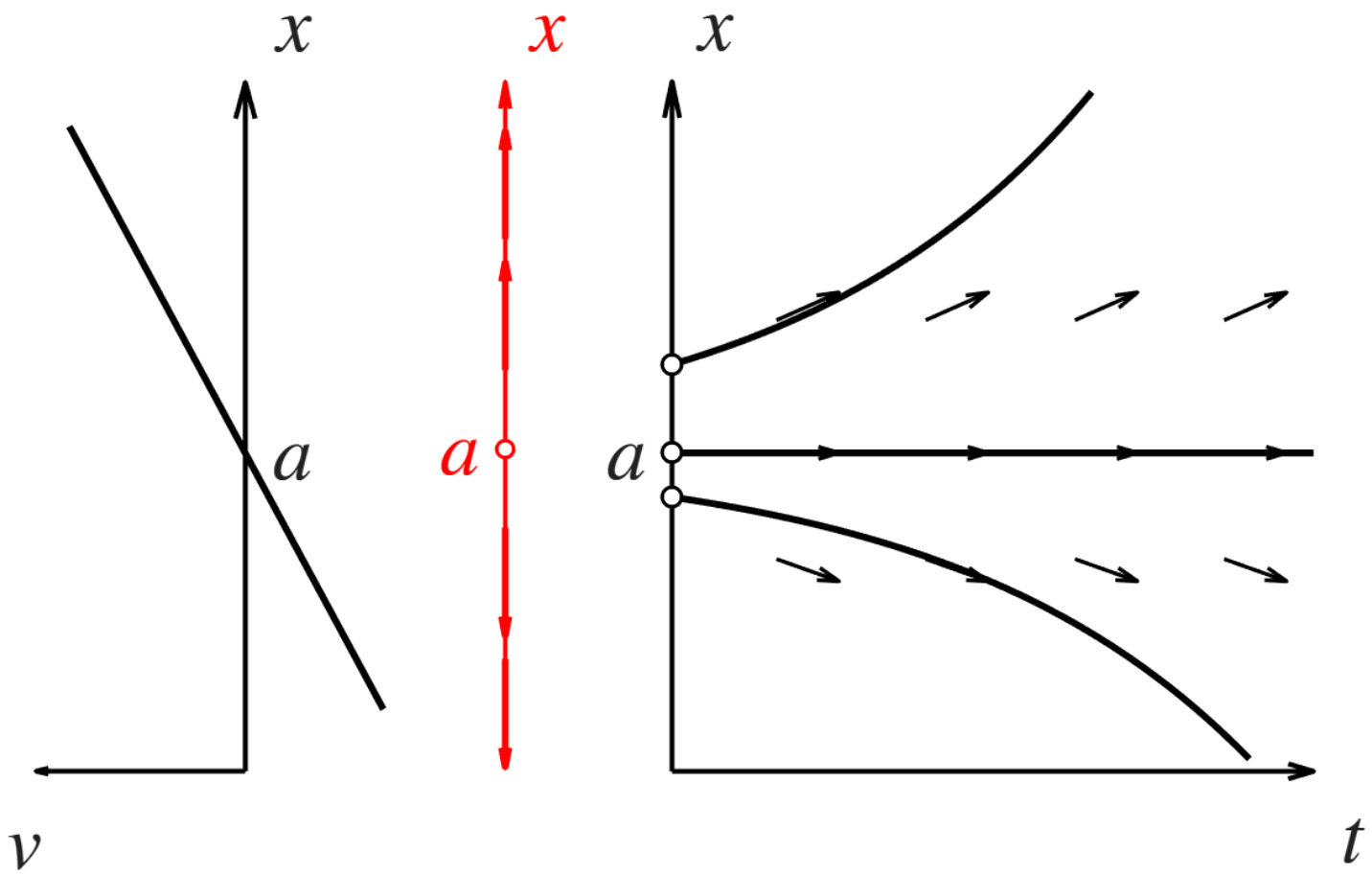


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f\big(x(t)\big)$$

状态的变化率只由当前状态决定，把这样的动力系统称为 **自治系统**

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

向量场的平衡点

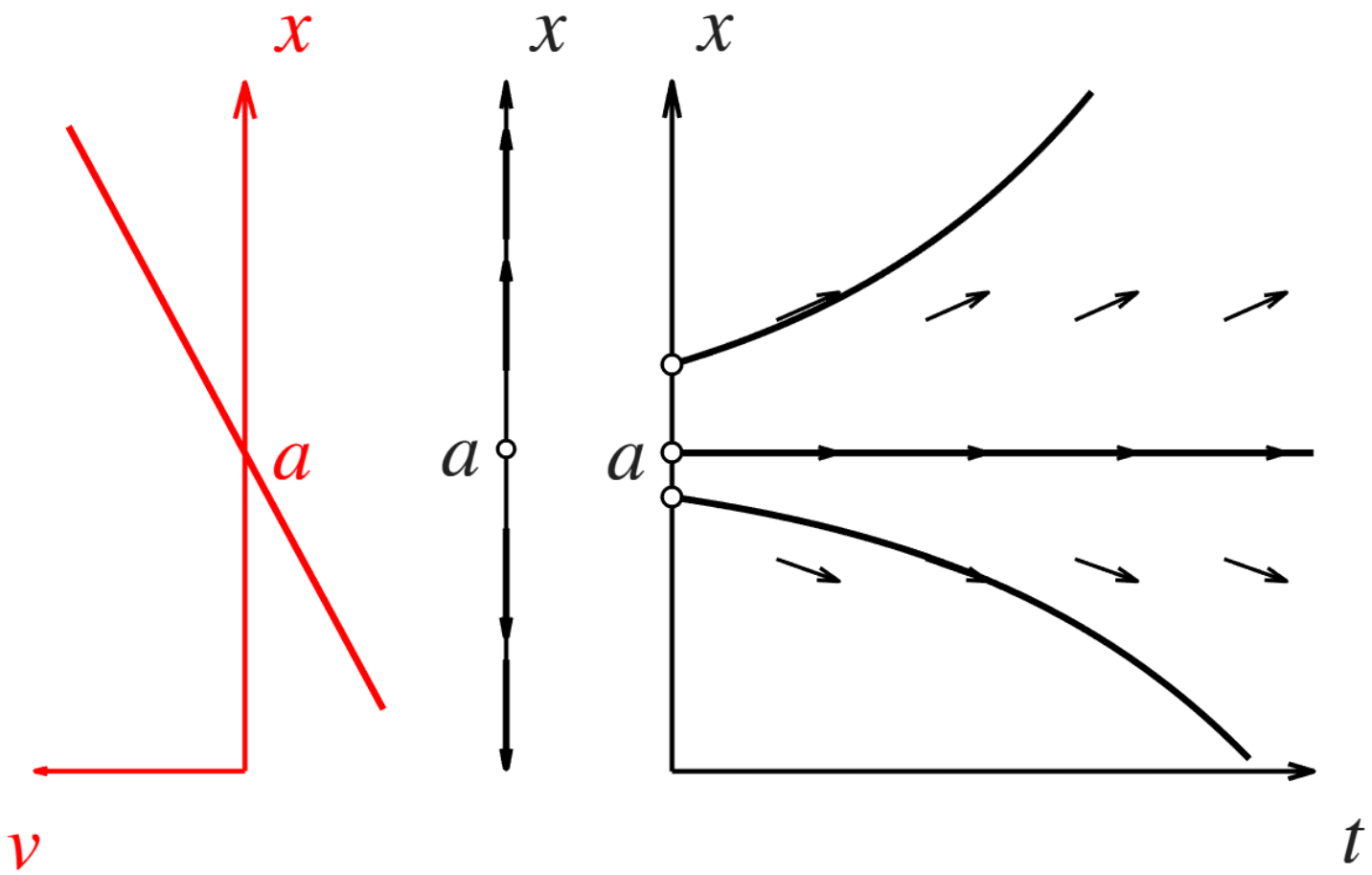


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f\big(x(t)\big)$$

在向量场中，状态 a 的变化率为零，即 $f(a) = 0$，称 a 为 **平衡点**

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

自治系统的状态变化与时间无关

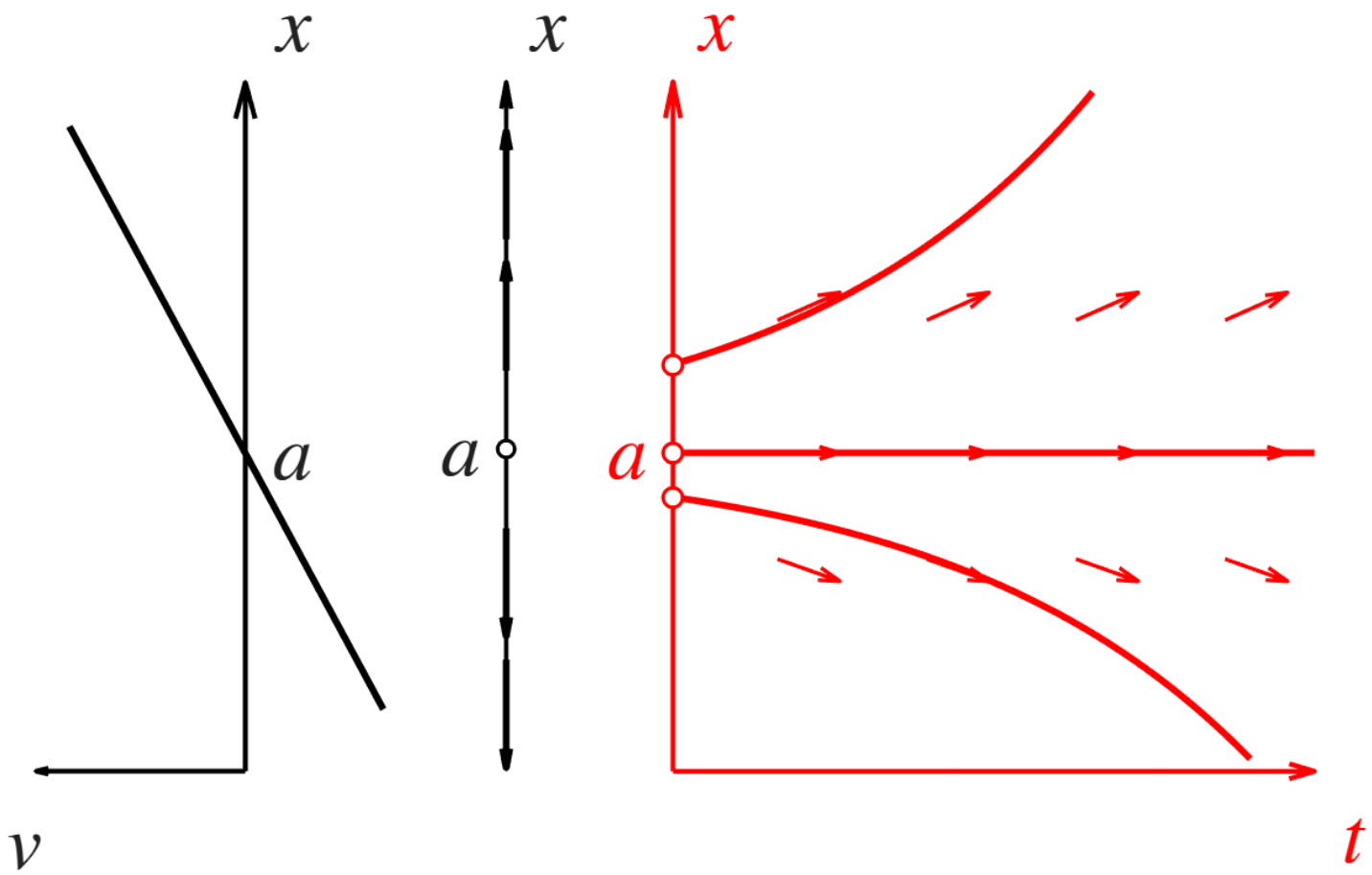


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f\big(x(t)\big)$$

变化率 v 仅由当前状态 x 决定，即 $\boldsymbol{v = f(x)}$

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

自治系统的状态变化与时间无关

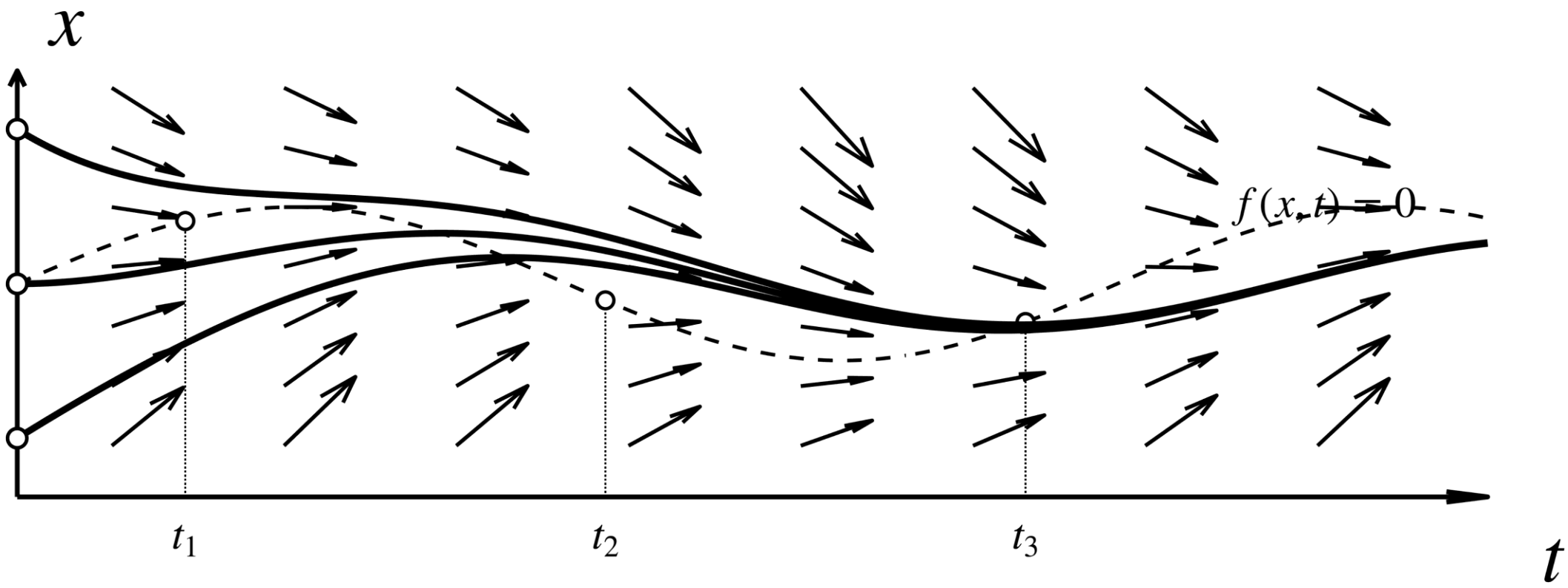


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f\big(x(t)\big)$$

在相同状态 x 处，变化率不随时间 t 改变，即 **向量场不显式依赖时间**

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

如果时间本身也影响变化率

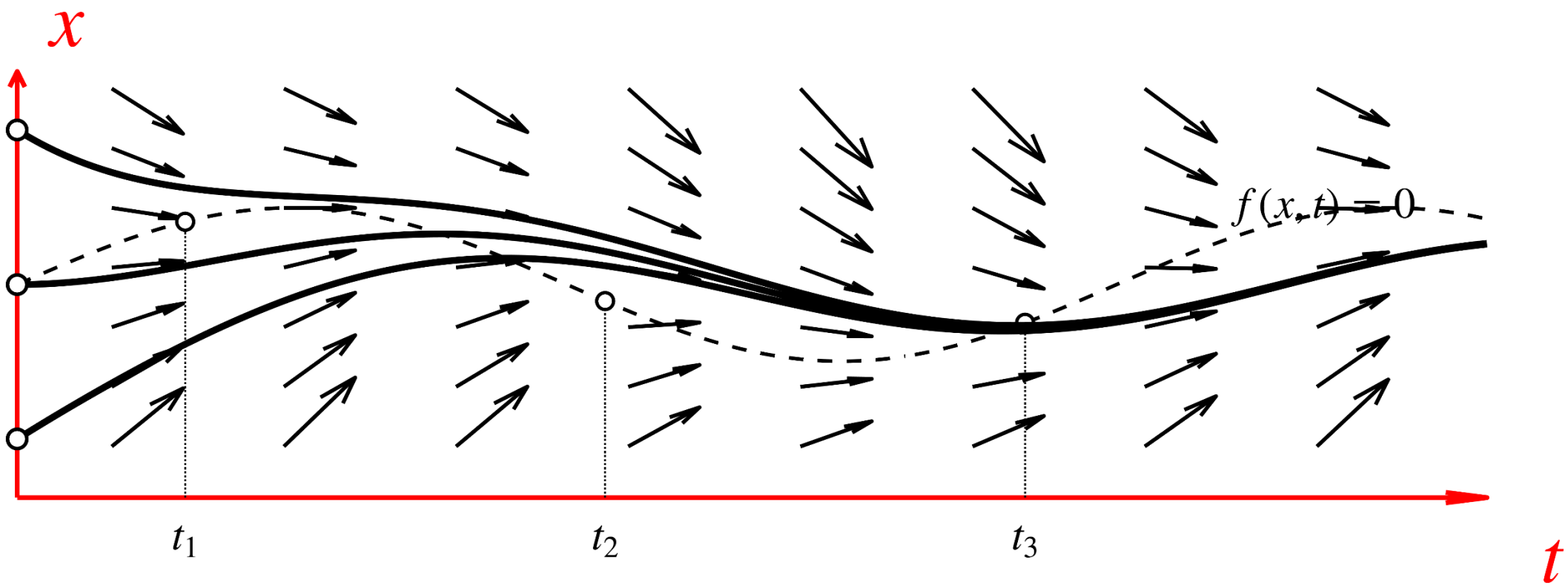


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f(x(t), t)$$

若状态的变化率还显式依赖时间，则称为 **非自治系统**

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

非自治系统

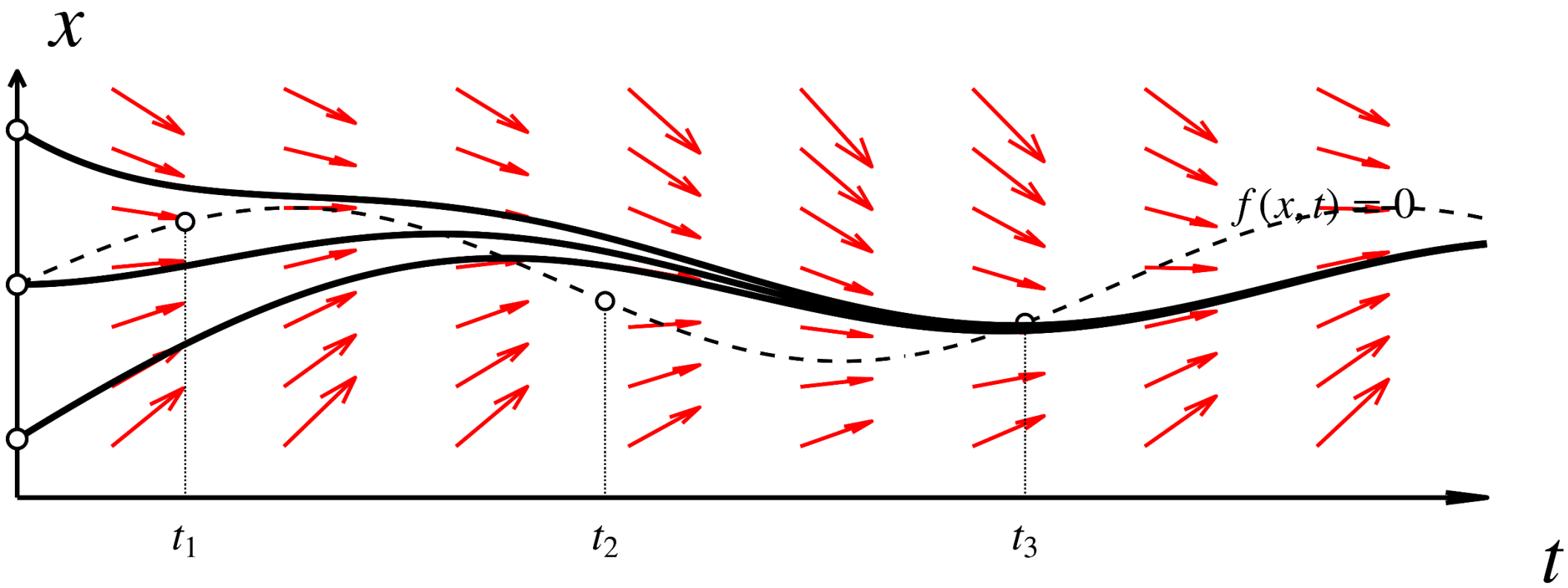


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f(x(t), t)$$

横轴表示时间 t，纵轴表示系统状态 x

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

非自治系统的向量场

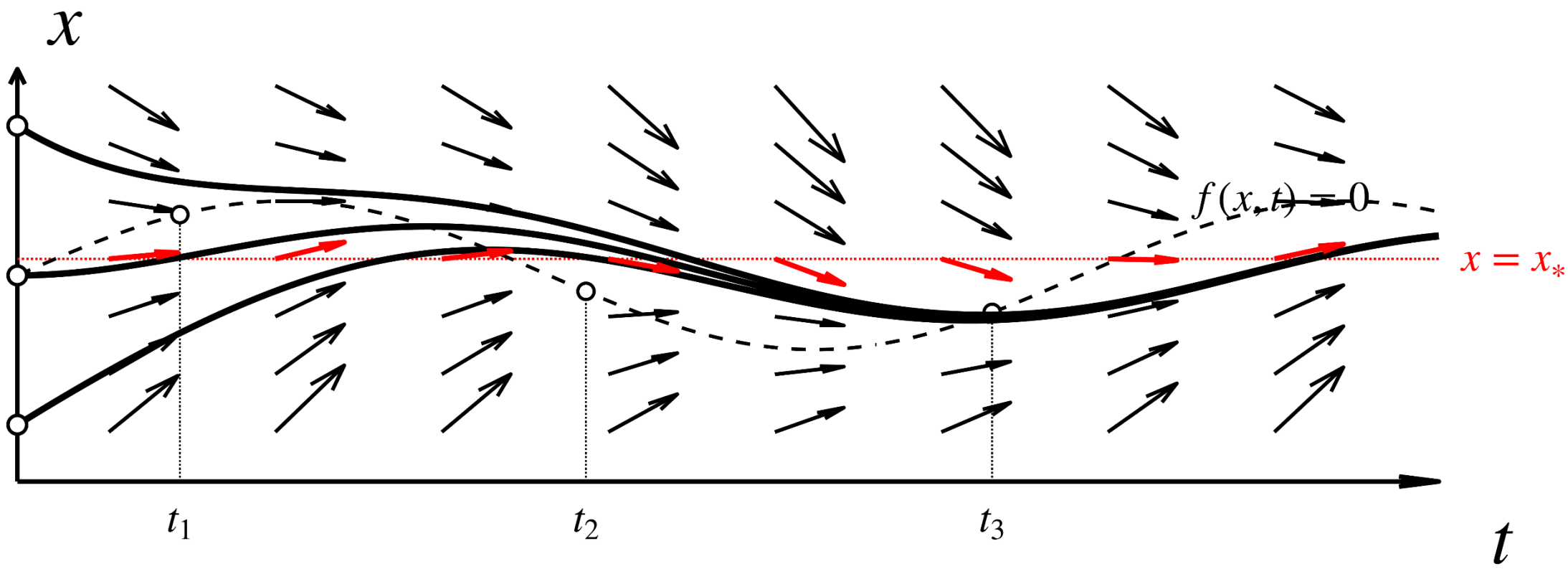


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f(x(t), t)$$

每个箭头表示系统在相应时刻和状态下的瞬时演化方向

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

非自治系统的向量场

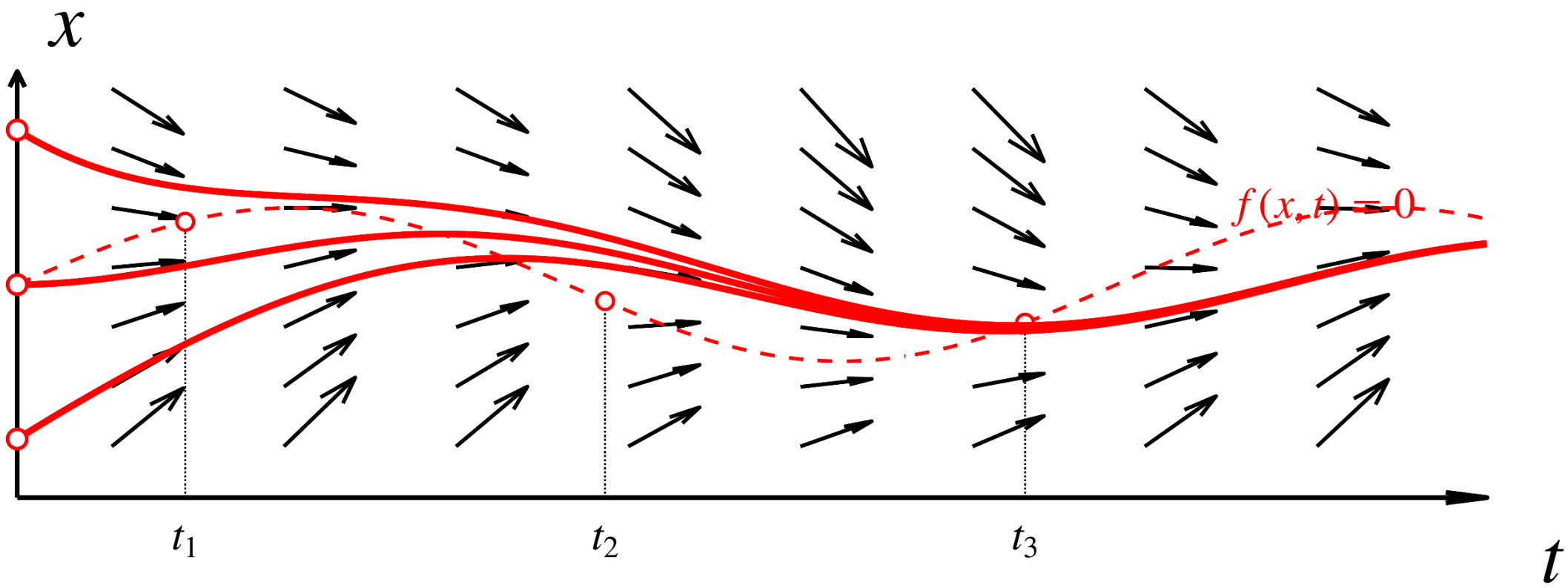


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f(x(t), t)$$

沿时间轴观察，同一状态位置的箭头会改变方向和倾角，说明 **向量场随时间变化**

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

非自治系统的轨迹

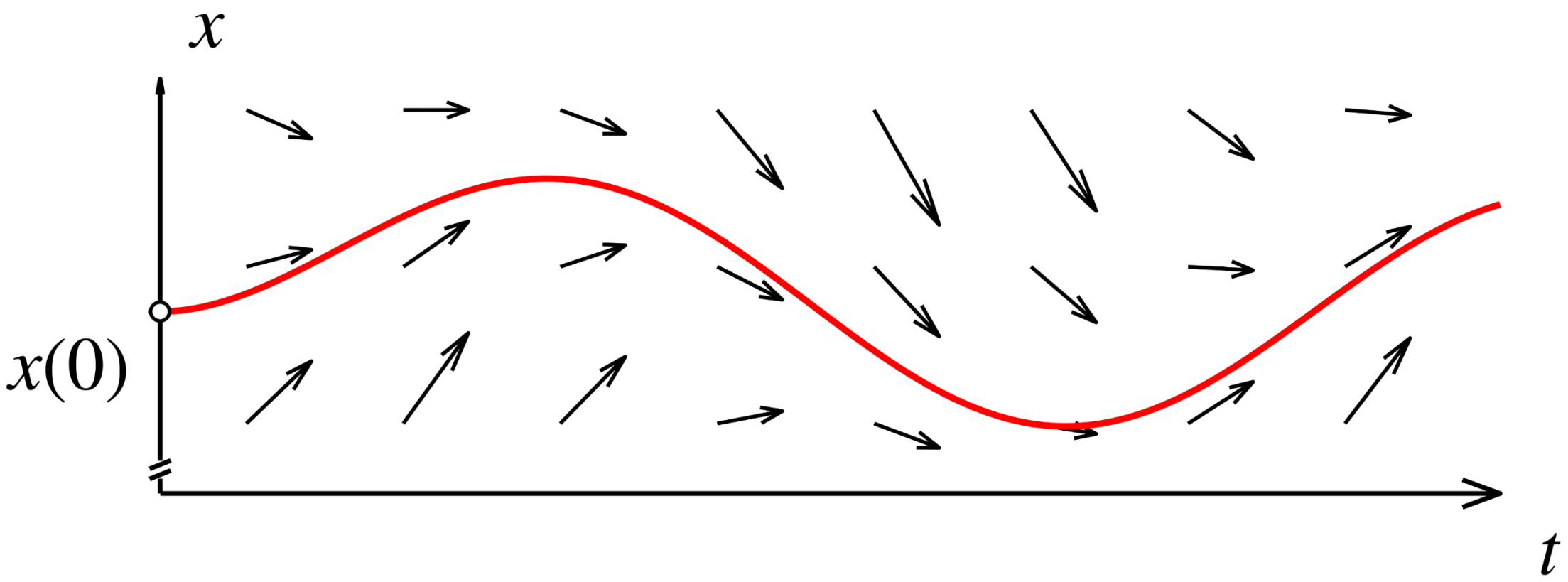


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f(x(t),t)$$

虚线显示瞬时零速度位置也在移动，实线则表示轨迹被时变向量场不断推动

VI A, Richard A S. Ordinary differential equations[M]. Cambridge, MA: MIT Press, 1973.

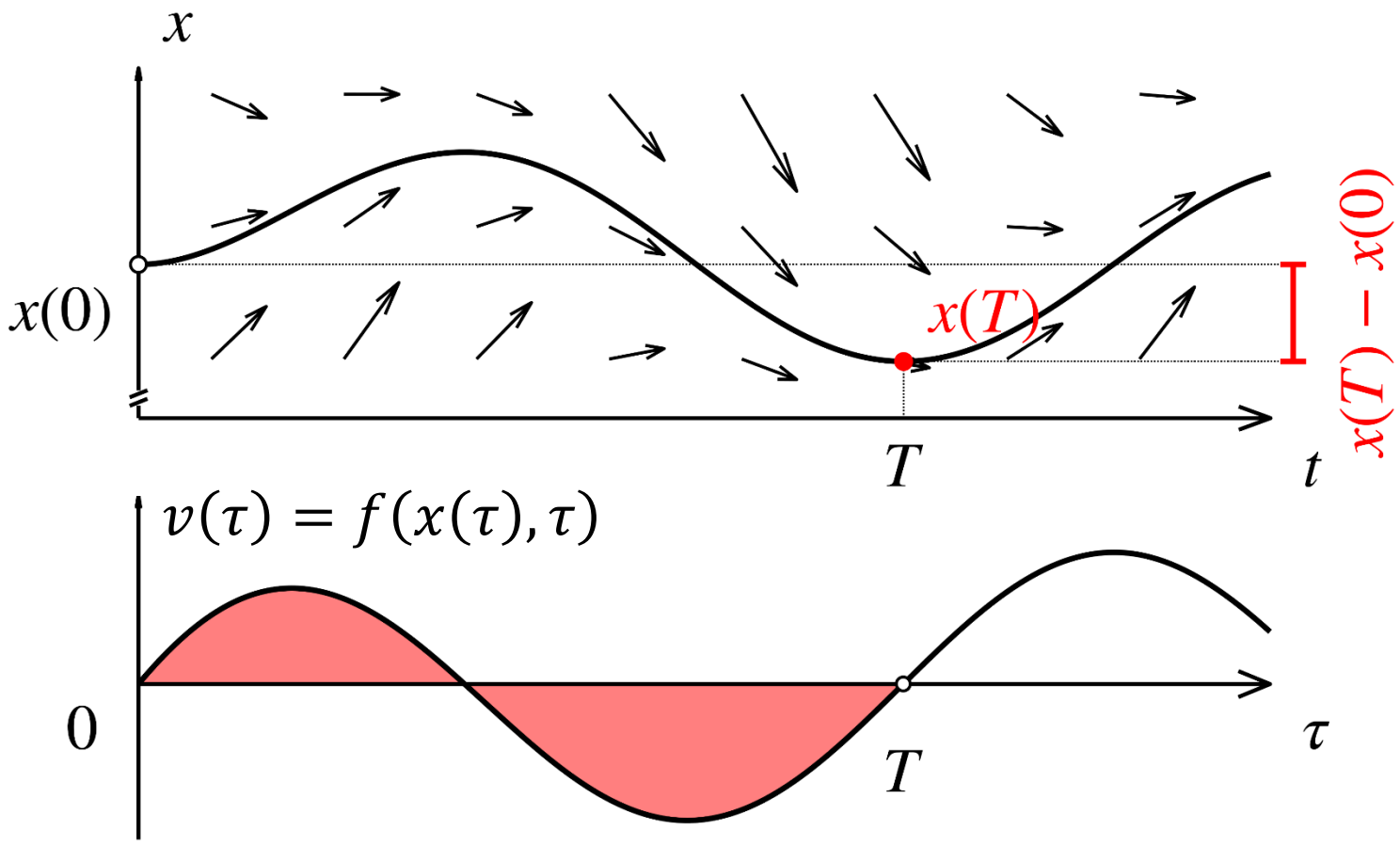
常微分方程的初值问题



$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f(x(t), t)$$

初值问题：已知初始状态 $x(0)$ 和向量场，求任意时刻 t 的状态 $x(t)$

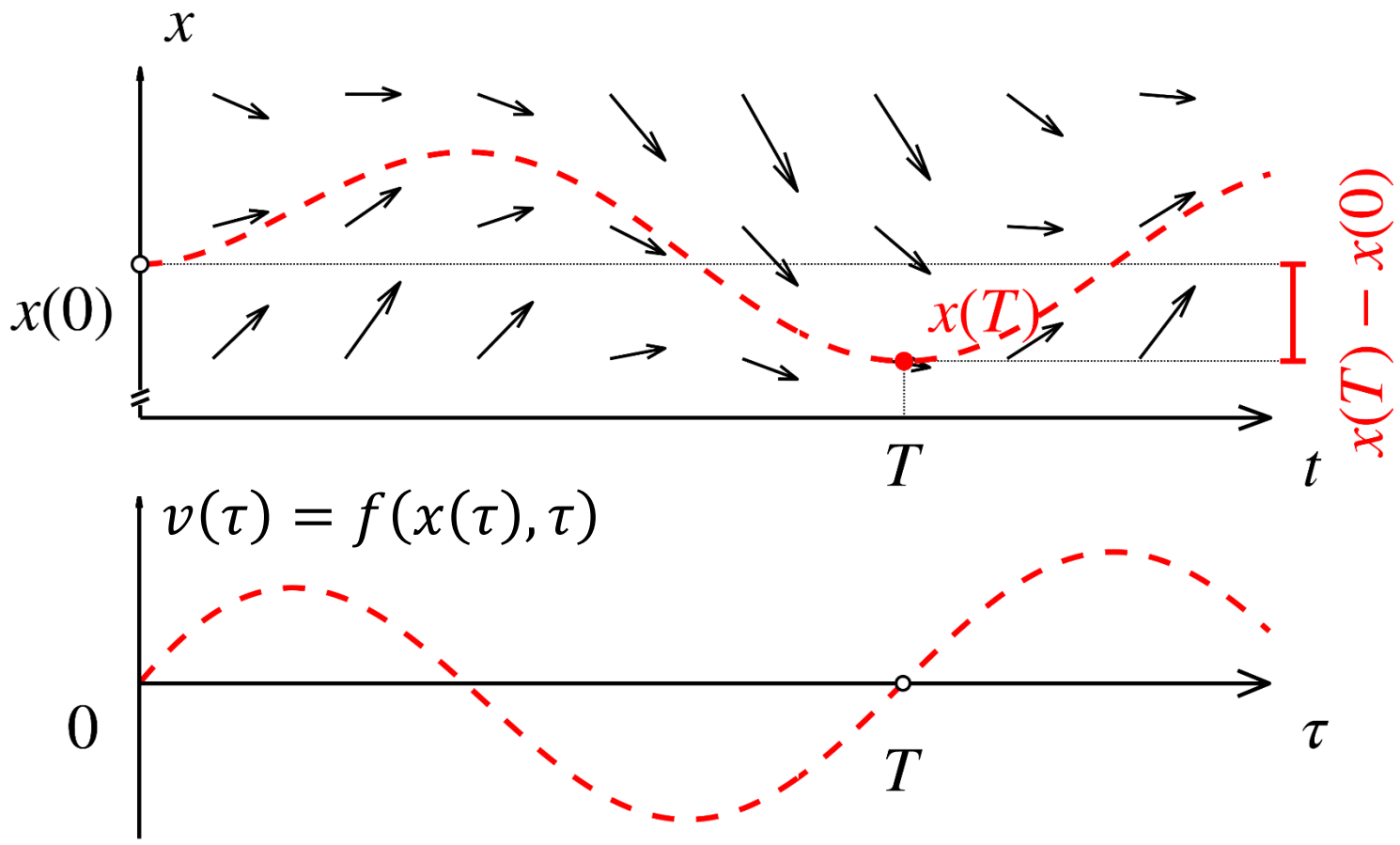
积分表示——常微分方程的连续化求解



$$x(t) = x(0) + \int_0^T f\,(x(\tau), \tau) d\tau$$

状态 = 初值 + **变化率对时间的积分**

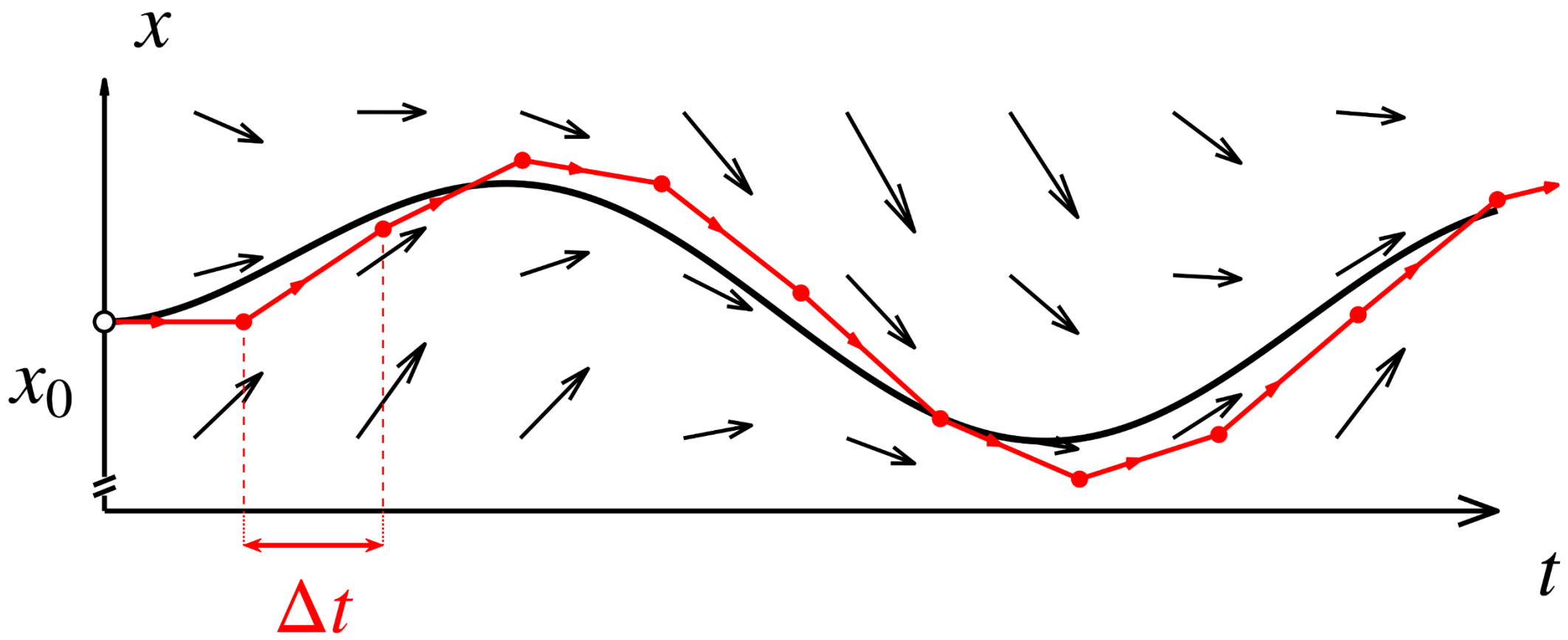
积分表示的困难——被积函数依赖未知轨迹



$$x(t) = x(0) + \int_0^T f\left(x(\tau), \tau\right) d\tau$$

被积函数 $f(x(\tau),\tau)$ 依赖于**未知的状态轨迹** $x(\tau)$ 本身，因此无法直接计算

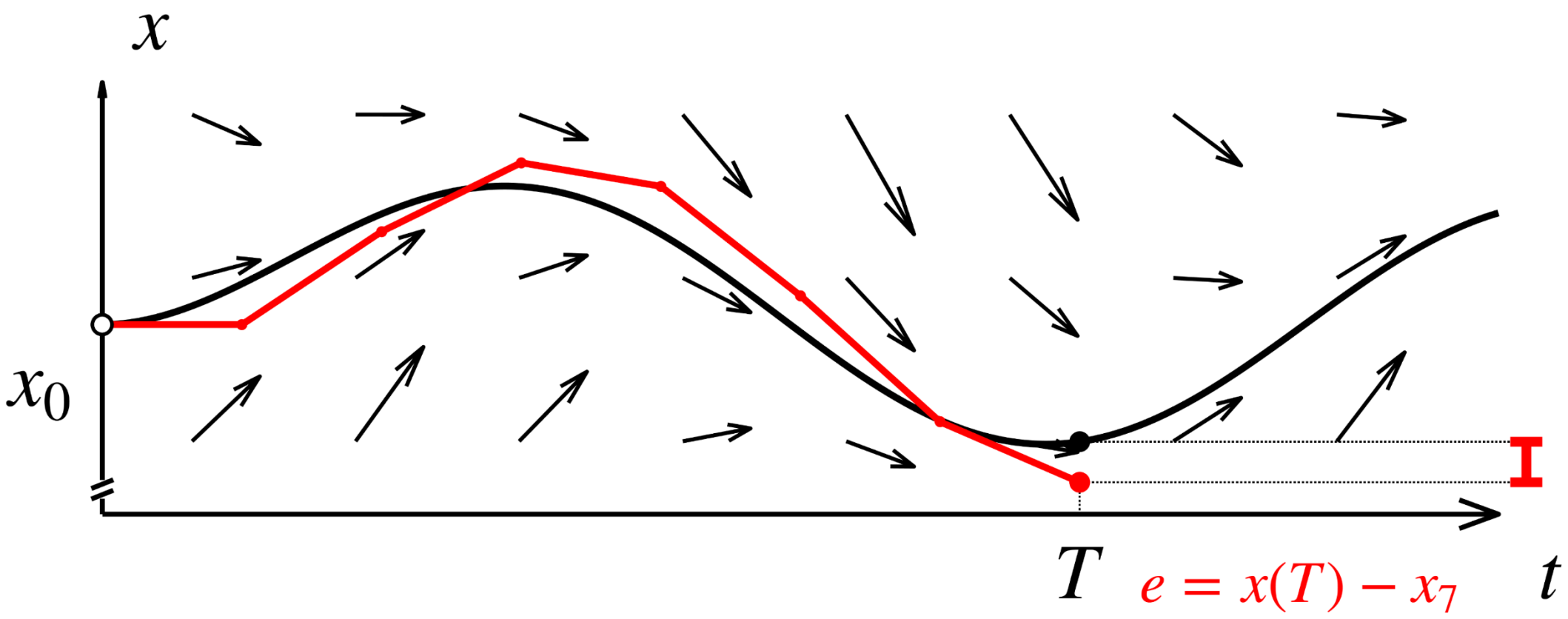
欧拉法——常微分方程的离散化求解



$$x_{n+1} = x_n + \Delta t \cdot f(x_n, t_n)$$

用局部变化率 $f(x_n, t_n)$ 近似短时间积分的数值方法被称为 **欧拉法**

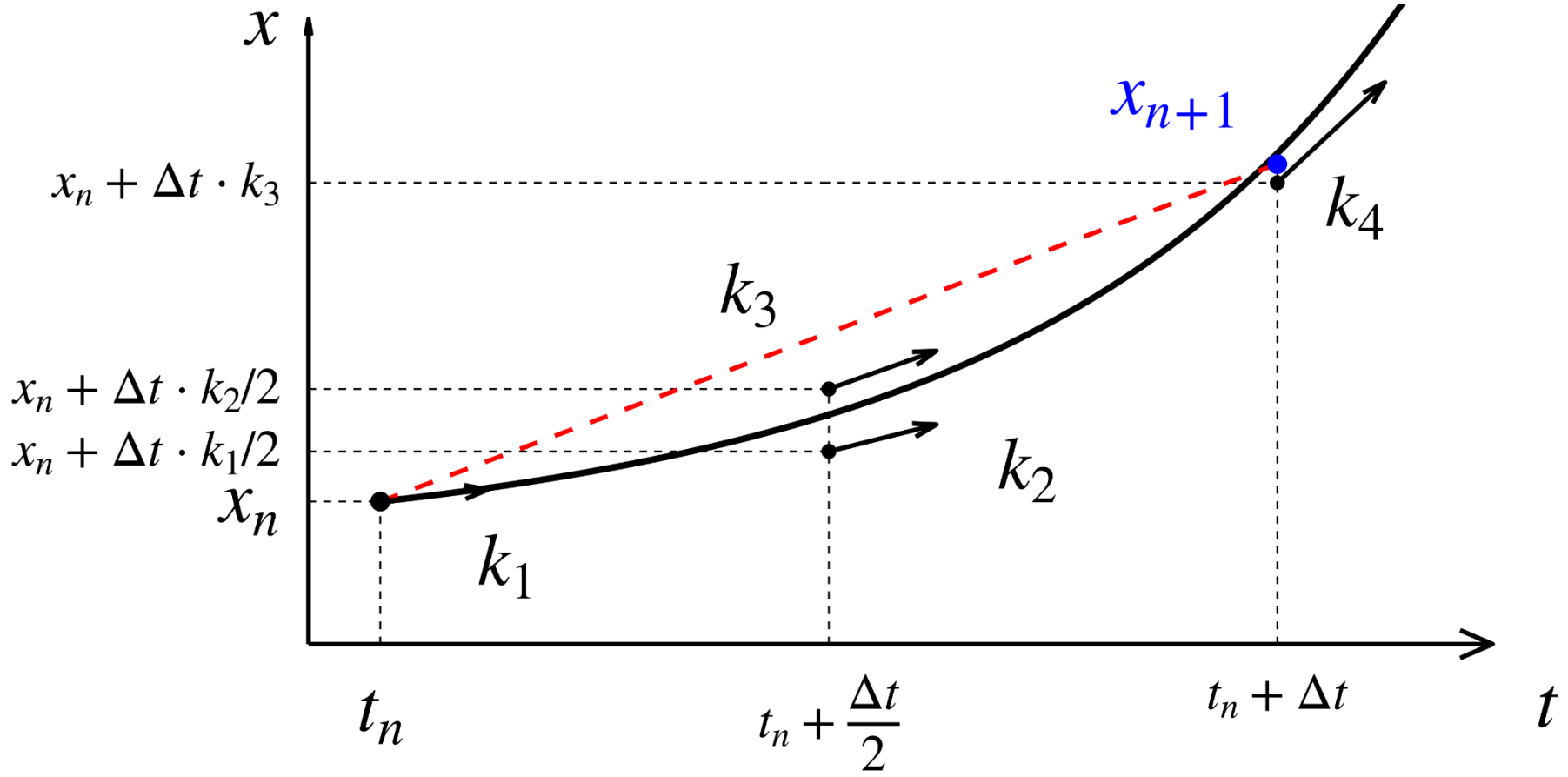
欧拉法的局限——步长大时误差累积明显



$$x_{n+1} = x_n + \Delta t \cdot f(x_n, t_n)$$

当步长 Δt 较大时，即离散化程度越大时，欧拉法的 **全局误差** 往往较大

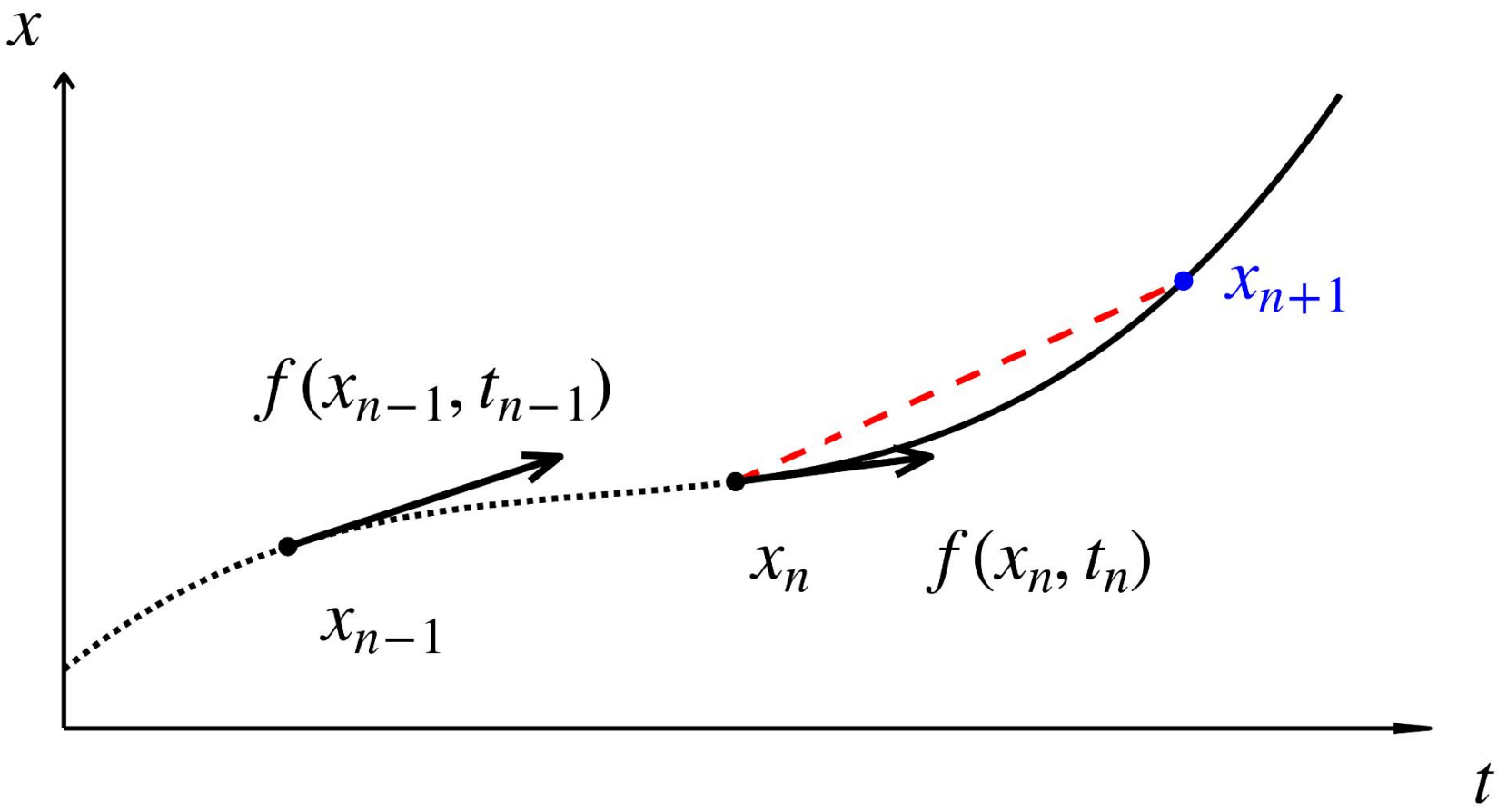
高阶单步法——在一个步长内多次采样向量场



$$t_{n+1} = t_n + \Delta t \qquad x_{n+1} = x_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

四阶 Runge–Kutta 方法利用一个步长内四个斜率的加权和来提高精度

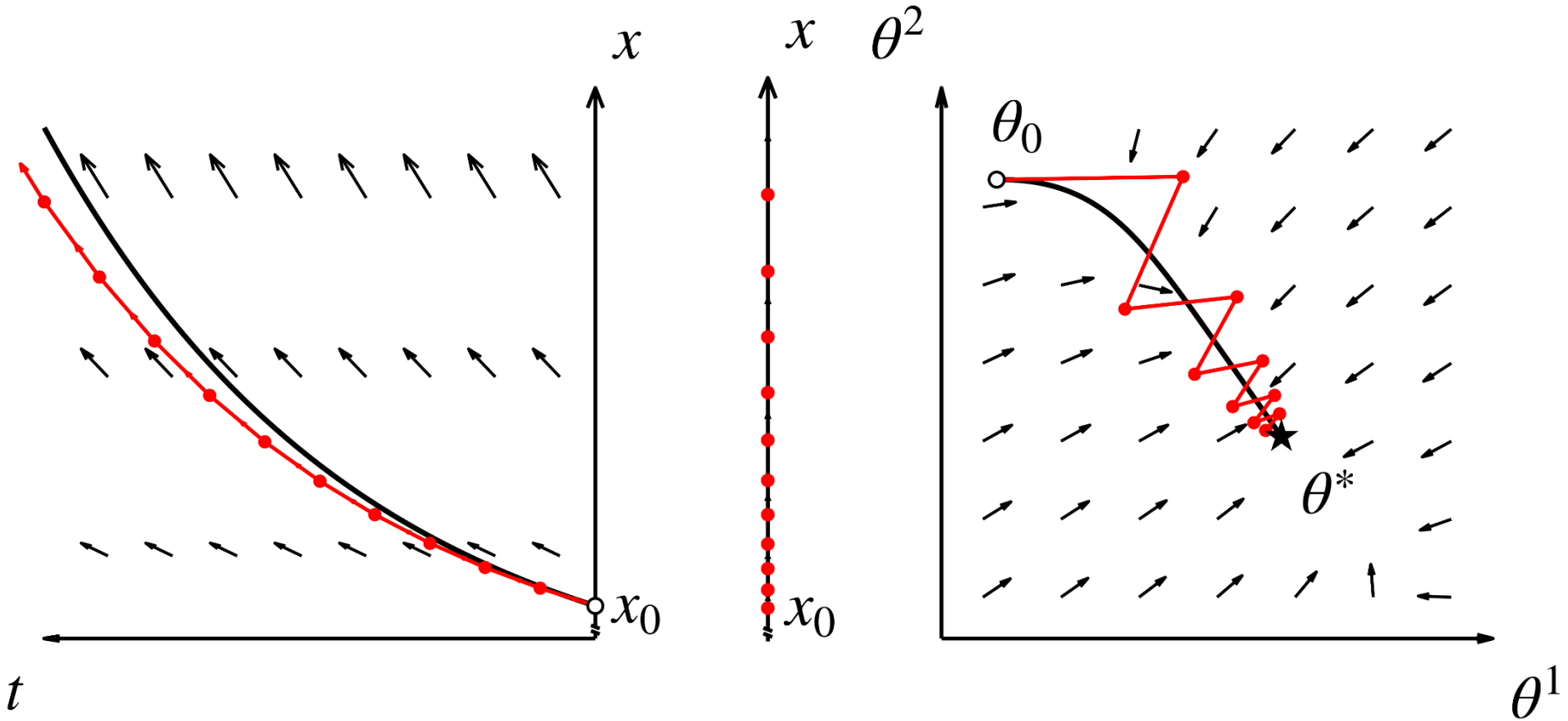
多步法——利用前几步的历史信息预测下一步



$$x_{n+1} = x_n + \frac{3}{2}\Delta t \cdot f(x_n, t_n) - \frac{1}{2}\Delta t \cdot f(x_{n-1}, t_{n-1})$$

二步 Adams–Bashforth 方法利用前两个时刻斜率的加权和来预测下一时刻状态

欧拉法与梯度下降法具有相同本质

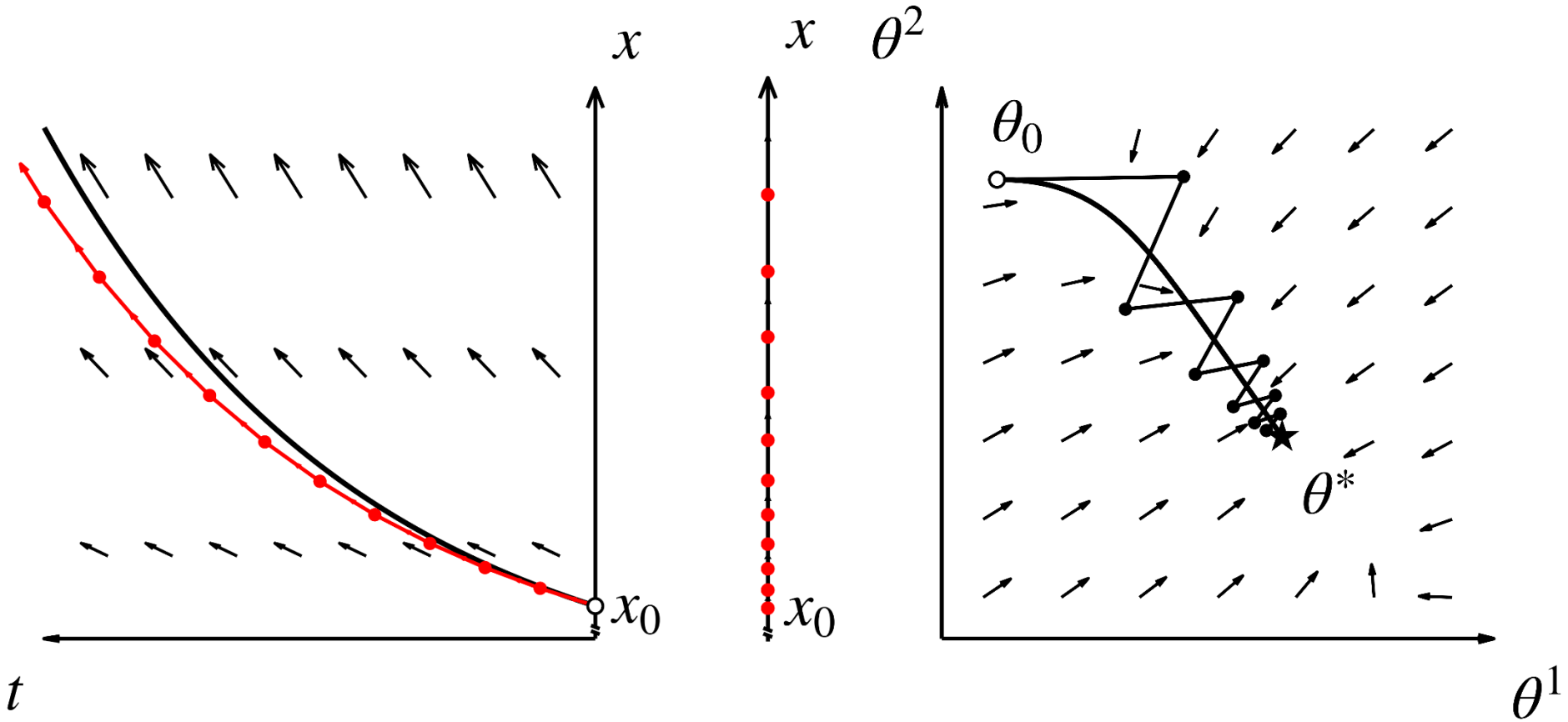


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f\big(x(t)\big)$$

$$\frac{\mathrm{d}\theta(t)}{\mathrm{d}t} = -\nabla_\theta L\big(\theta(t)\big)$$

二者本质上都是沿向量场对连续动力系统进行 **离散步进**

欧拉法与梯度下降法具有相同本质

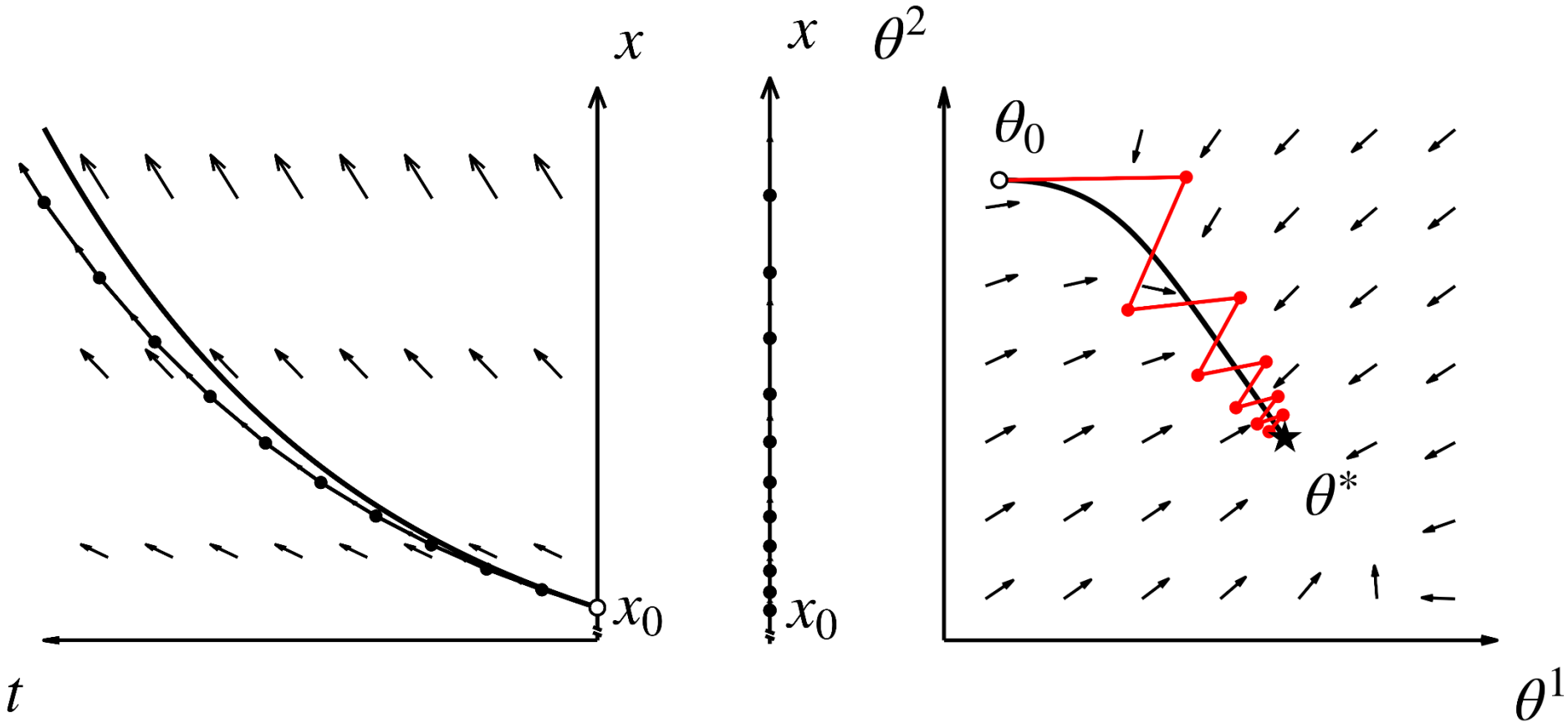


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f\big(x(t)\big)$$

$$\frac{\mathrm{d}\theta(t)}{\mathrm{d}t} = -\nabla_\theta L\big(\theta(t)\big)$$

欧拉法 沿向量场对常微分方程进行离散步进

欧拉法与梯度下降法具有相同本质

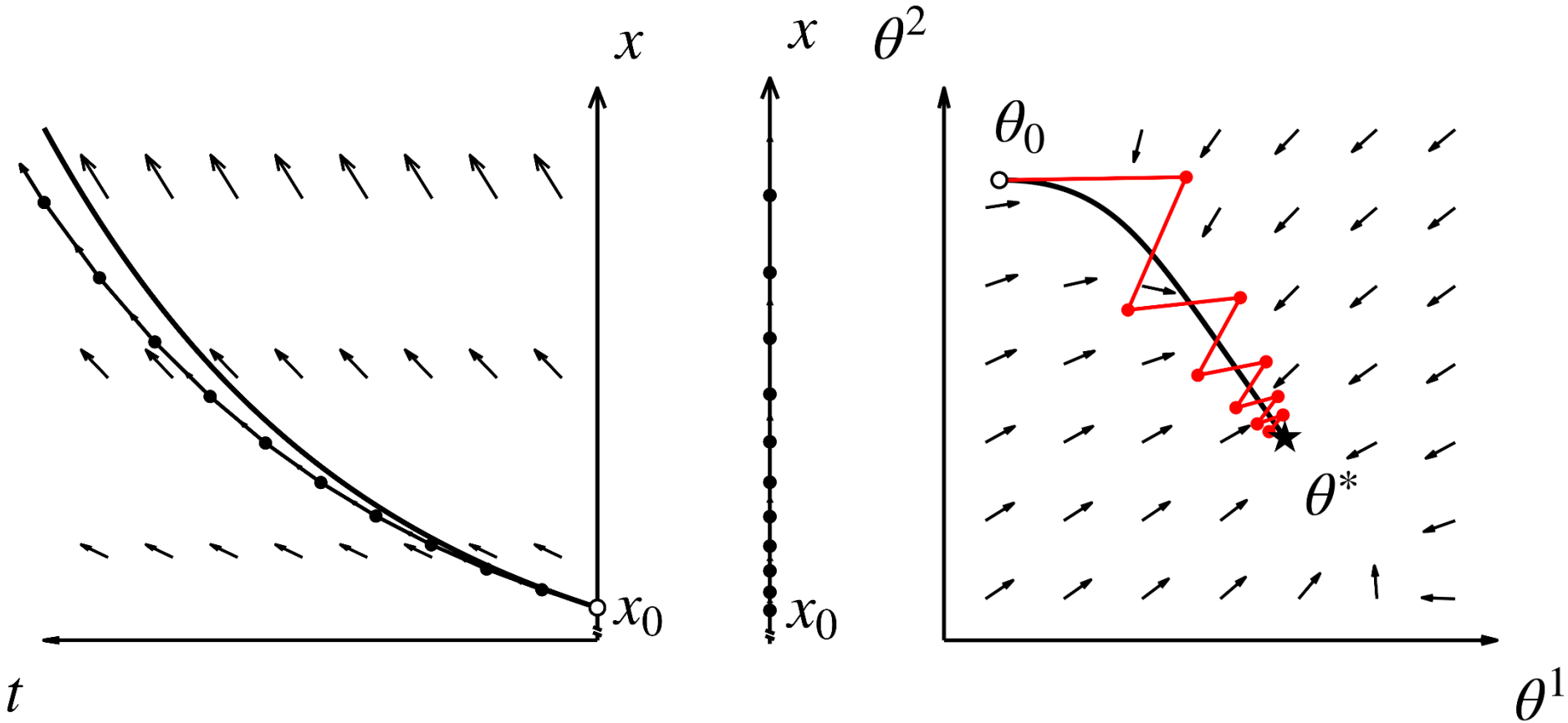


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f\big(x(t)\big)$$

$$\frac{\mathrm{d}\theta(t)}{\mathrm{d}t} = -\nabla_\theta L\big(\theta(t)\big)$$

梯度下降法 沿负梯度场对梯度流进行离散步进

欧拉法与梯度下降法具有相同本质

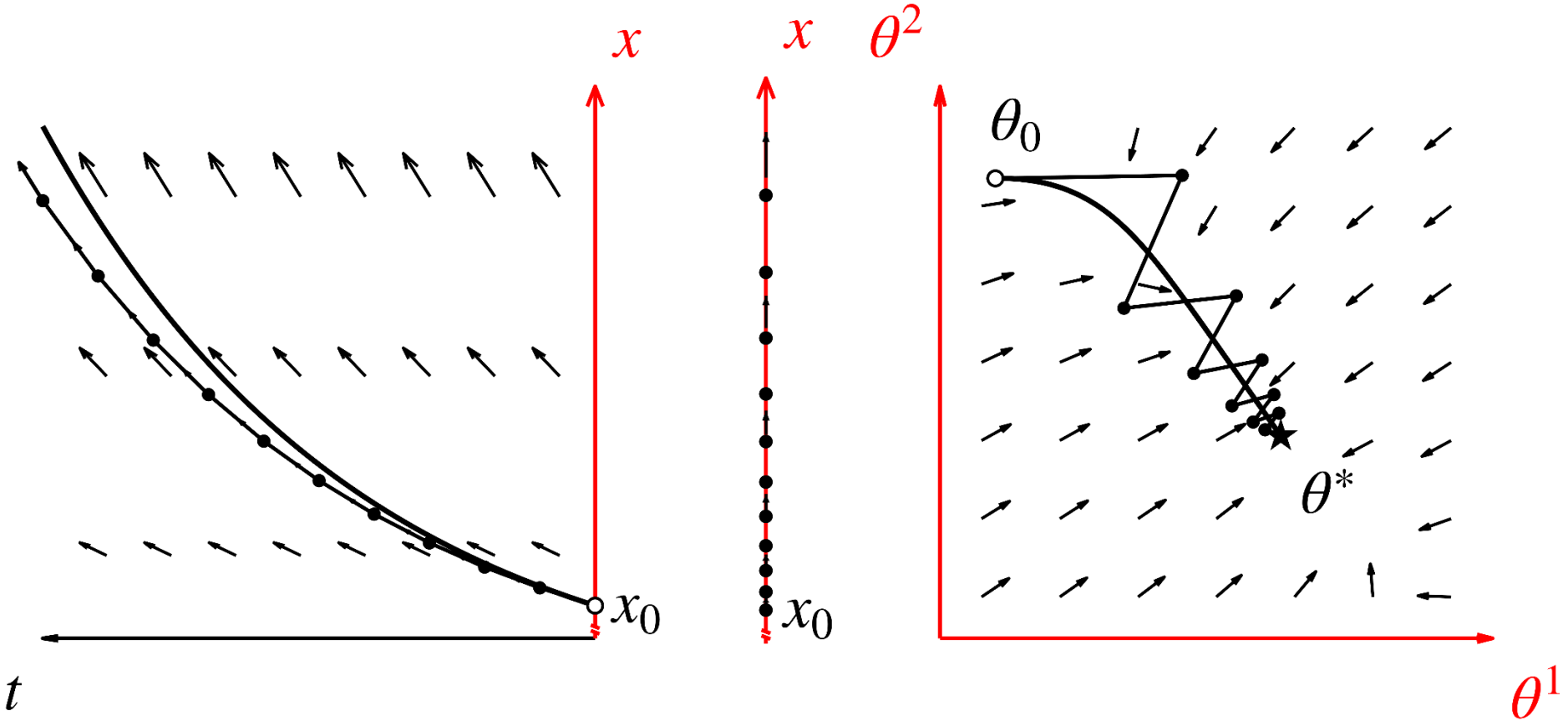


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f\big(x(t)\big)$$

$$\frac{\mathrm{d}\theta(t)}{\mathrm{d}t} = -\nabla_\theta L\big(\theta(t)\big)$$

梯度下降法是欧拉法在优化问题中的一个特例

欧拉法与梯度下降法的对应关系

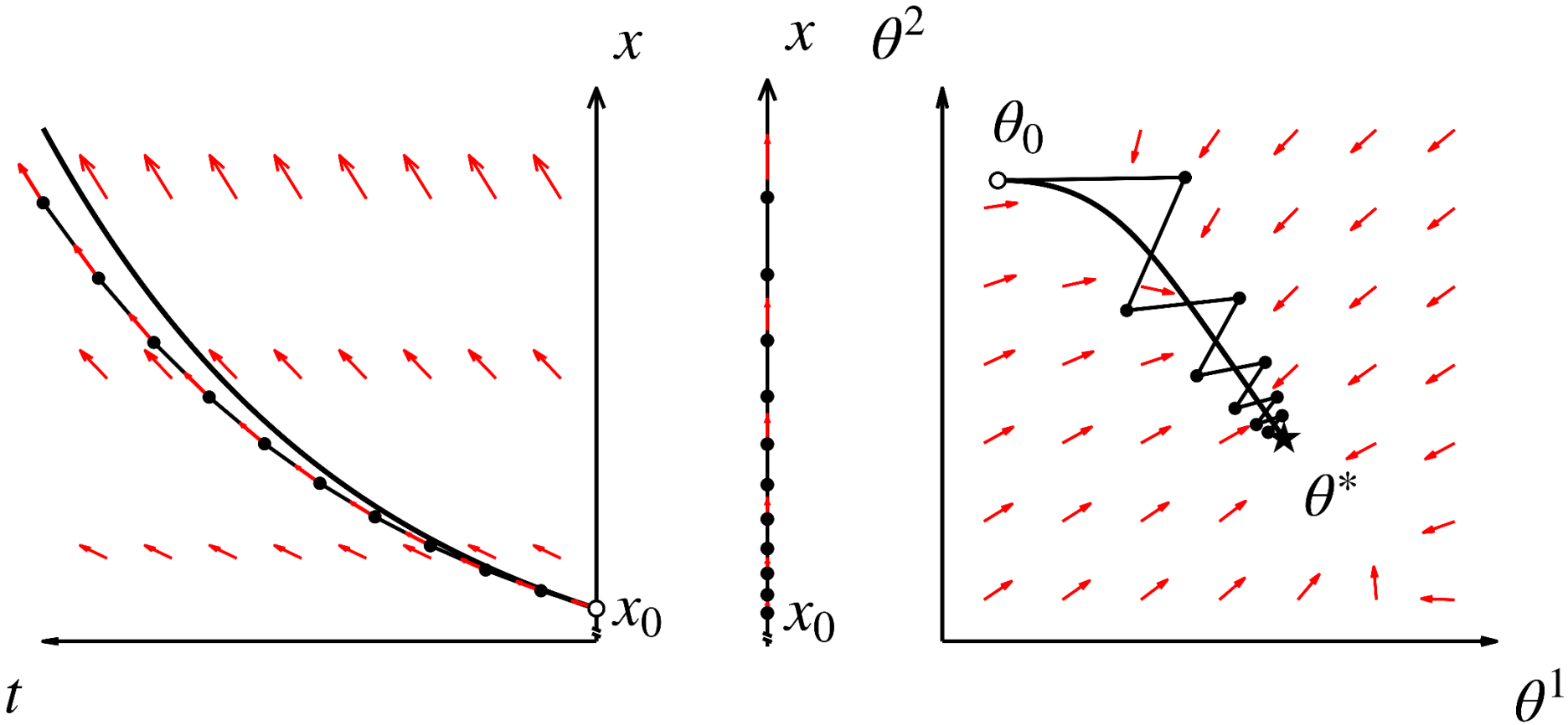


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f\big(x(t)\big)$$

$$\frac{\mathrm{d}\theta(t)}{\mathrm{d}t} = -\nabla_\theta L\big(\theta(t)\big)$$

系统状态：$x(t) \in \mathbb{R}$ 对应 $\theta(t) \in \mathbb{R}^2$

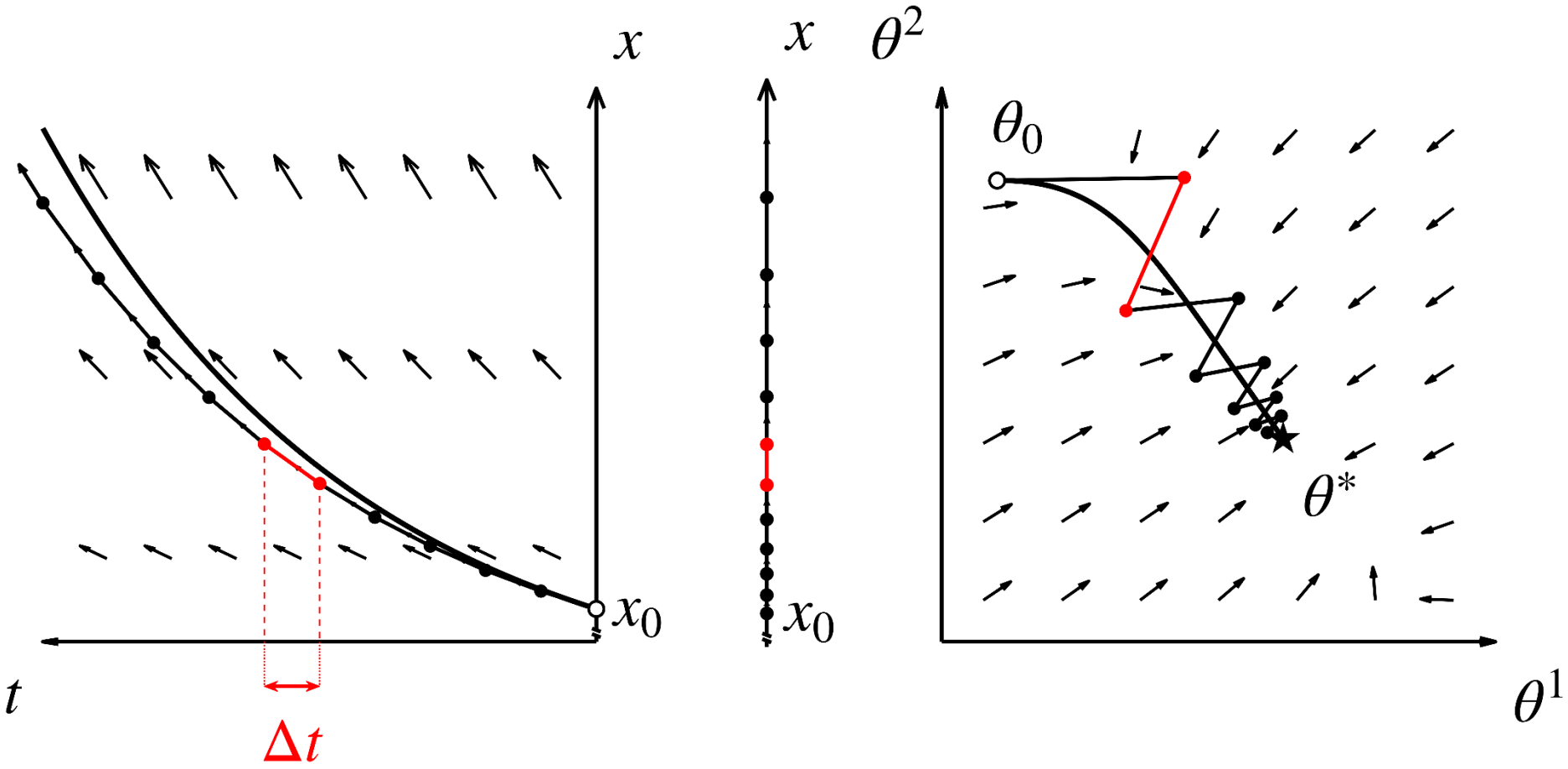
欧拉法与梯度下降法的对应关系



$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f\big(x(t)\big) \qquad \frac{\mathrm{d}\theta(t)}{\mathrm{d}t} = -\nabla_\theta L\big(\theta(t)\big)$$

动力学函数：向量场 $f\big(x(t)\big)$ 对应 负梯度场 $-\nabla_\theta L\big(\theta(t)\big)$

欧拉法与梯度下降法的对应关系



$$x_{k+1} = x_k + \Delta t f(x_k)$$

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta L(\theta_k)$$

离散步长：时间步长 Δt 对应 学习率 η

深度学习中的连续动力学机制

4.1 连续动力学的数学基础

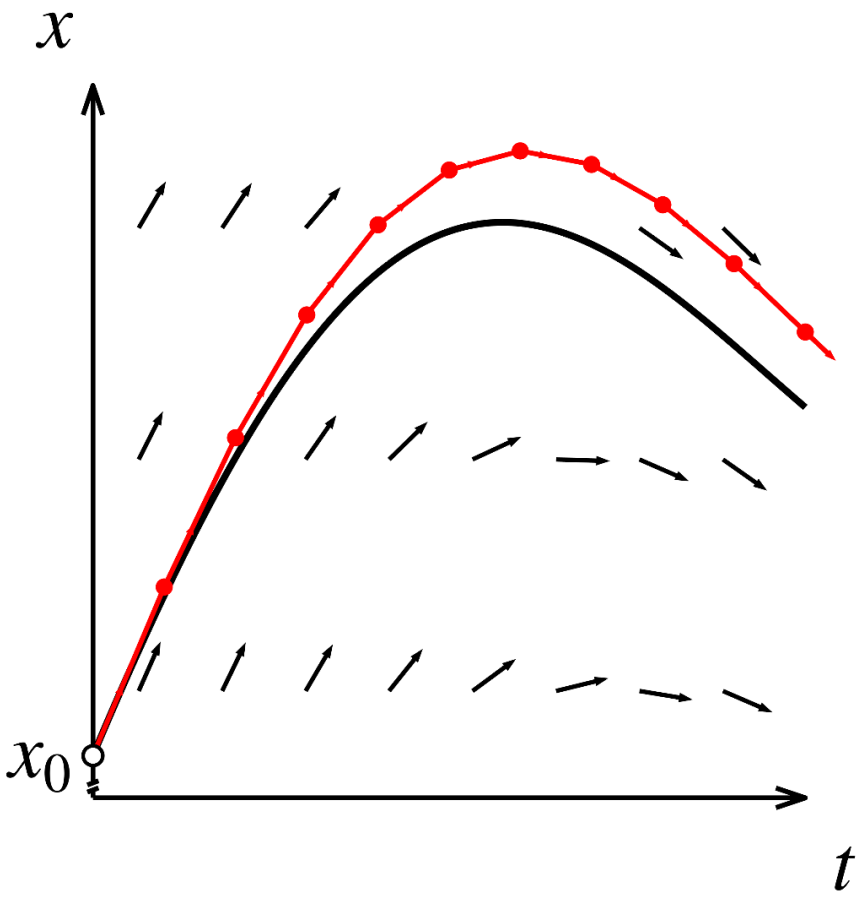
4.2 表示空间中的连续动力学

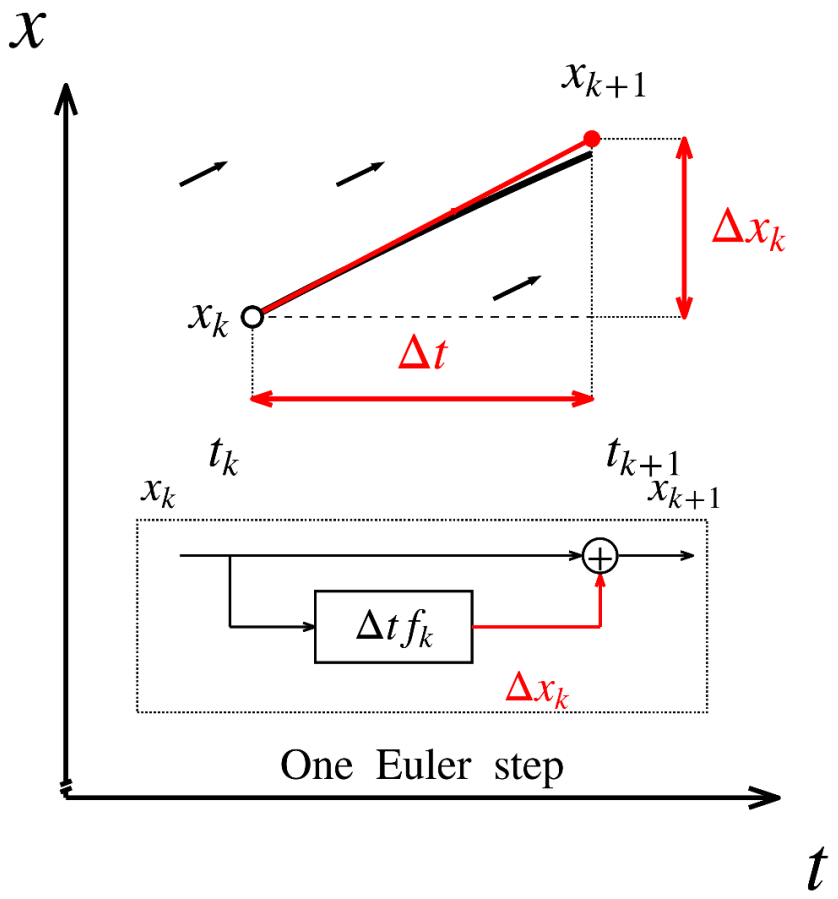
4.3 确定性概率动力学

4.4 随机概率动力学

4.5 离散状态上的连续时间动力学

从连续动力学到离散状态步进



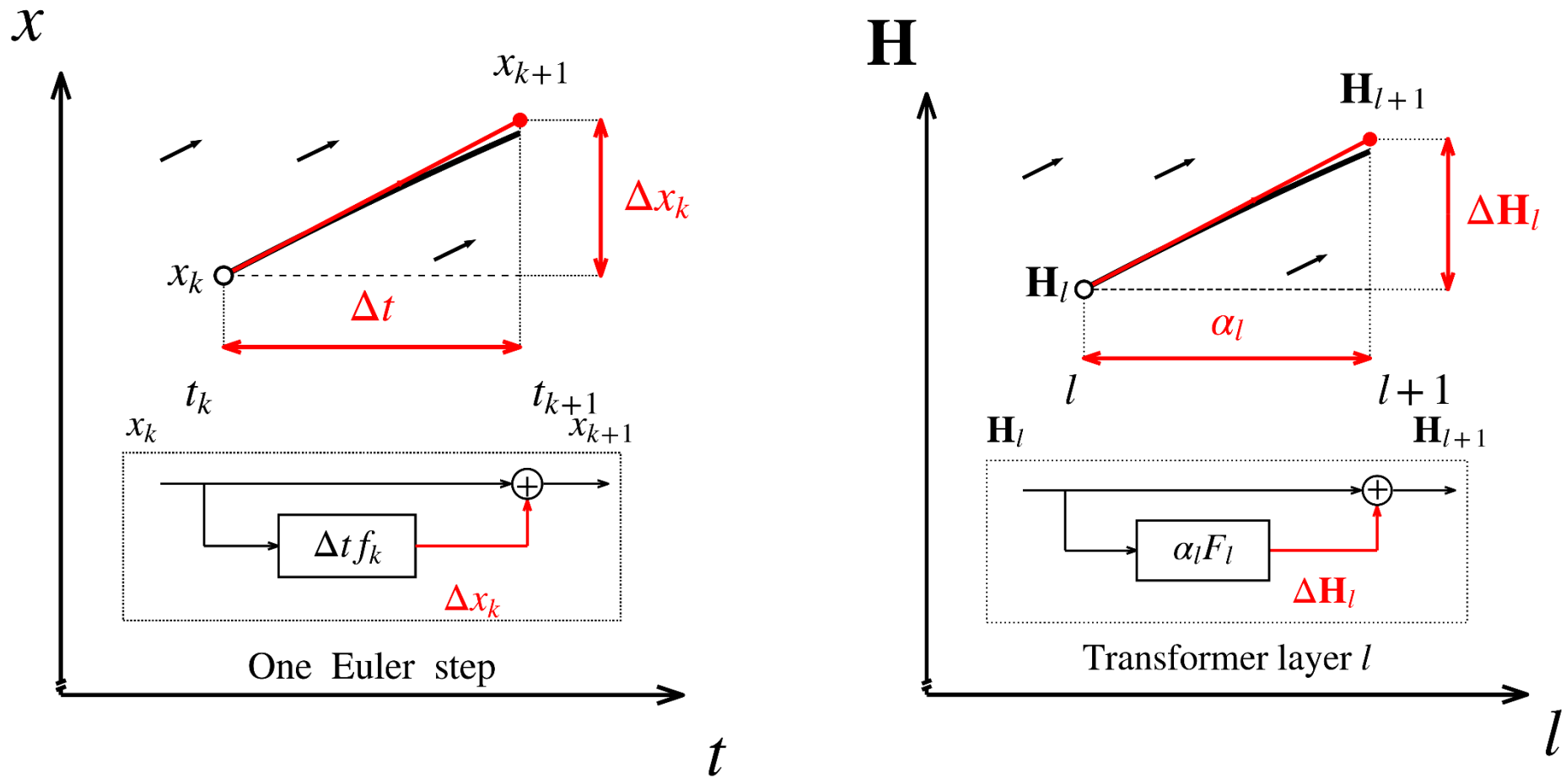


$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f(x(t), t)$$

$$x_{k+1} = x_k + \Delta t f(x_k, t_k)$$

欧拉法把连续动力学转化为“**当前状态 + 局部增量**”的离散步进

Transformer 残差连接具有相同结构

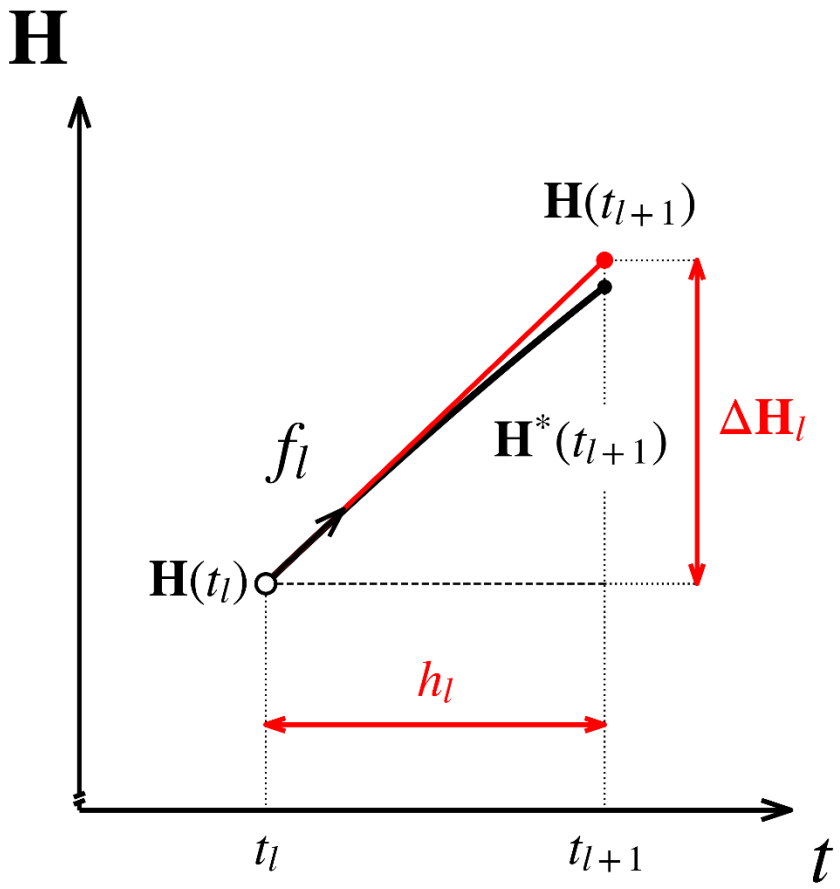


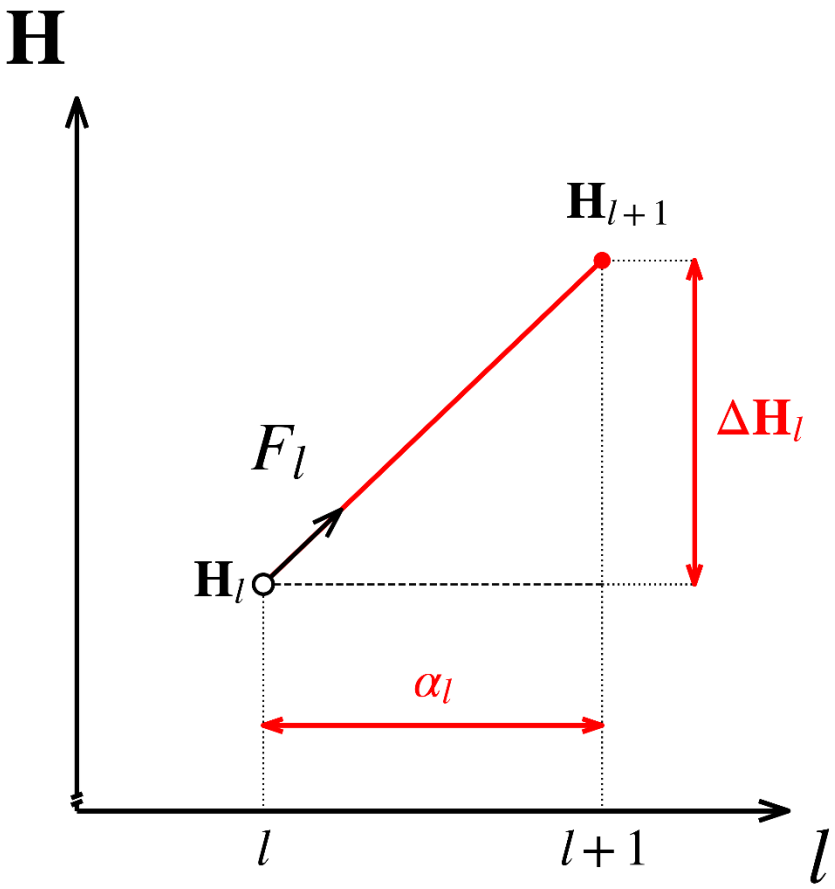
$$x_{k+1} = x_k + \Delta t f(x_k, t_k)$$

$$\boldsymbol{H}_{l+1} = \boldsymbol{H}_l + \alpha_l F_l(\mathrm{LN}(H_l); \theta_l)$$

残差连接 可被理解为隐藏表示 $\boldsymbol{H}$ 在网络深度 l 上的一次 **离散状态步进**

层索引可以视为离散深度时间

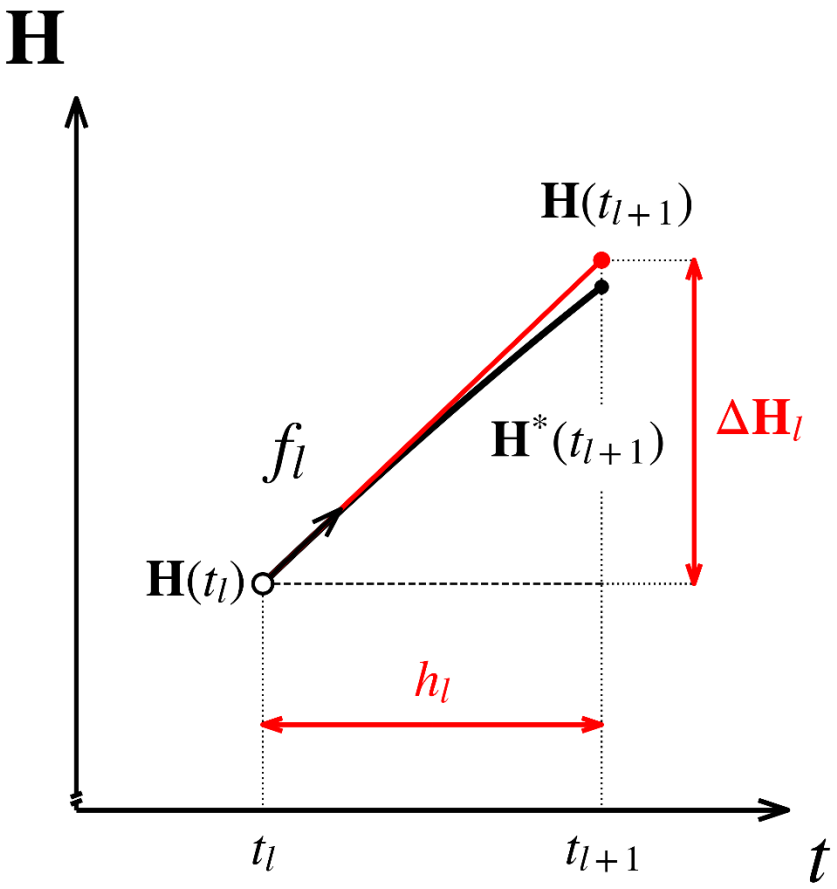


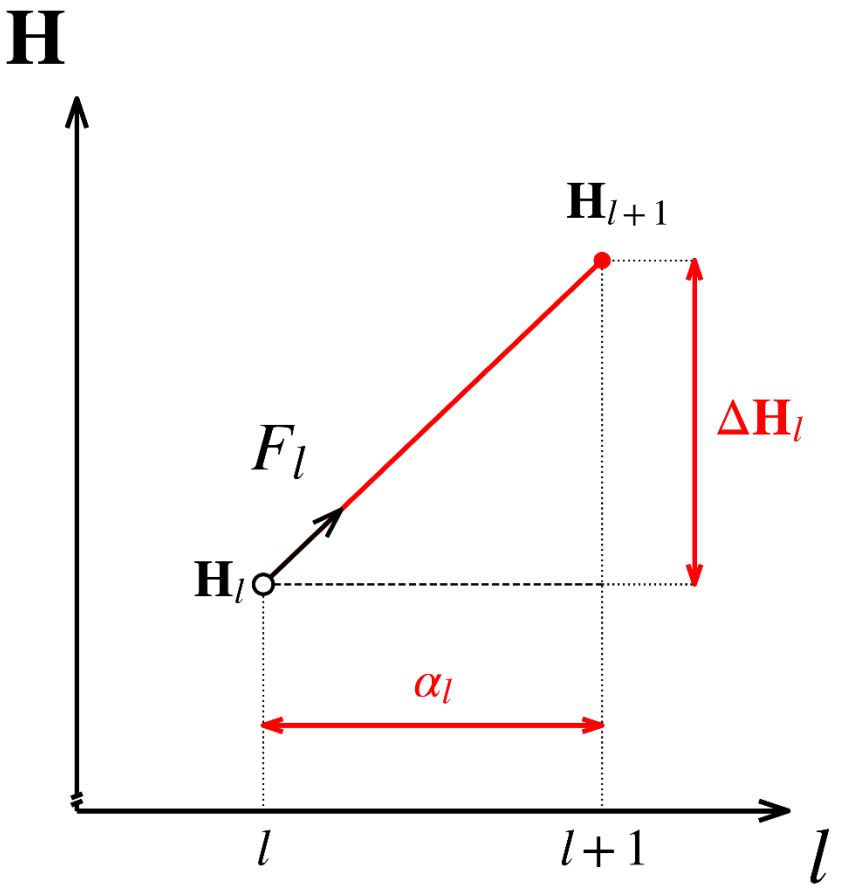


$$\boldsymbol{H}(t_{l+1}) \approx \boldsymbol{H}(t_l) + h_l f\left(H(t_l), t_l; \theta(t_l)\right) \qquad \boldsymbol{H}_{l+1} = \boldsymbol{H}_l + \alpha_l F_l(\mathrm{LN}(H_l); \theta_l)$$

Transformer 层 可视为连续表示动力学在离散深度网络上的 **状态步进**

结构对应 ≠ 严格等价



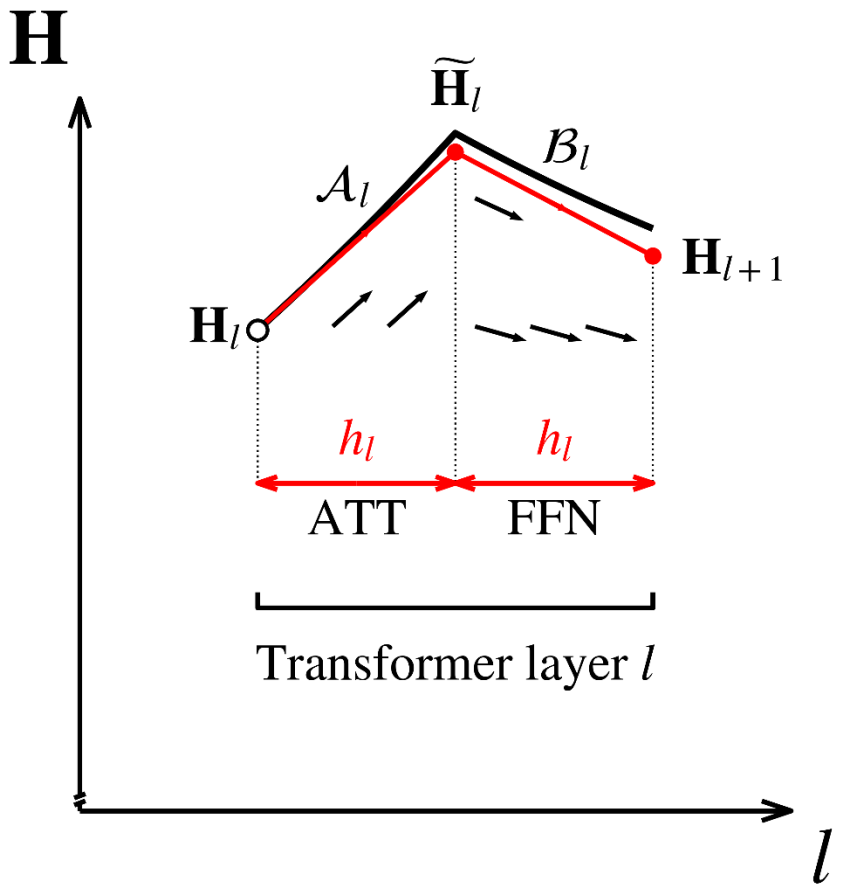


$$\boldsymbol{H}(t_{l+1}) \approx \boldsymbol{H}(t_l) + h_l f_l$$

$$\boldsymbol{H}_{l+1} = \boldsymbol{H}_l + \alpha_l F_l$$

ODE 视角提供分析 Transformer 表示演化的数学框架，而非网络结构的微分方程等价

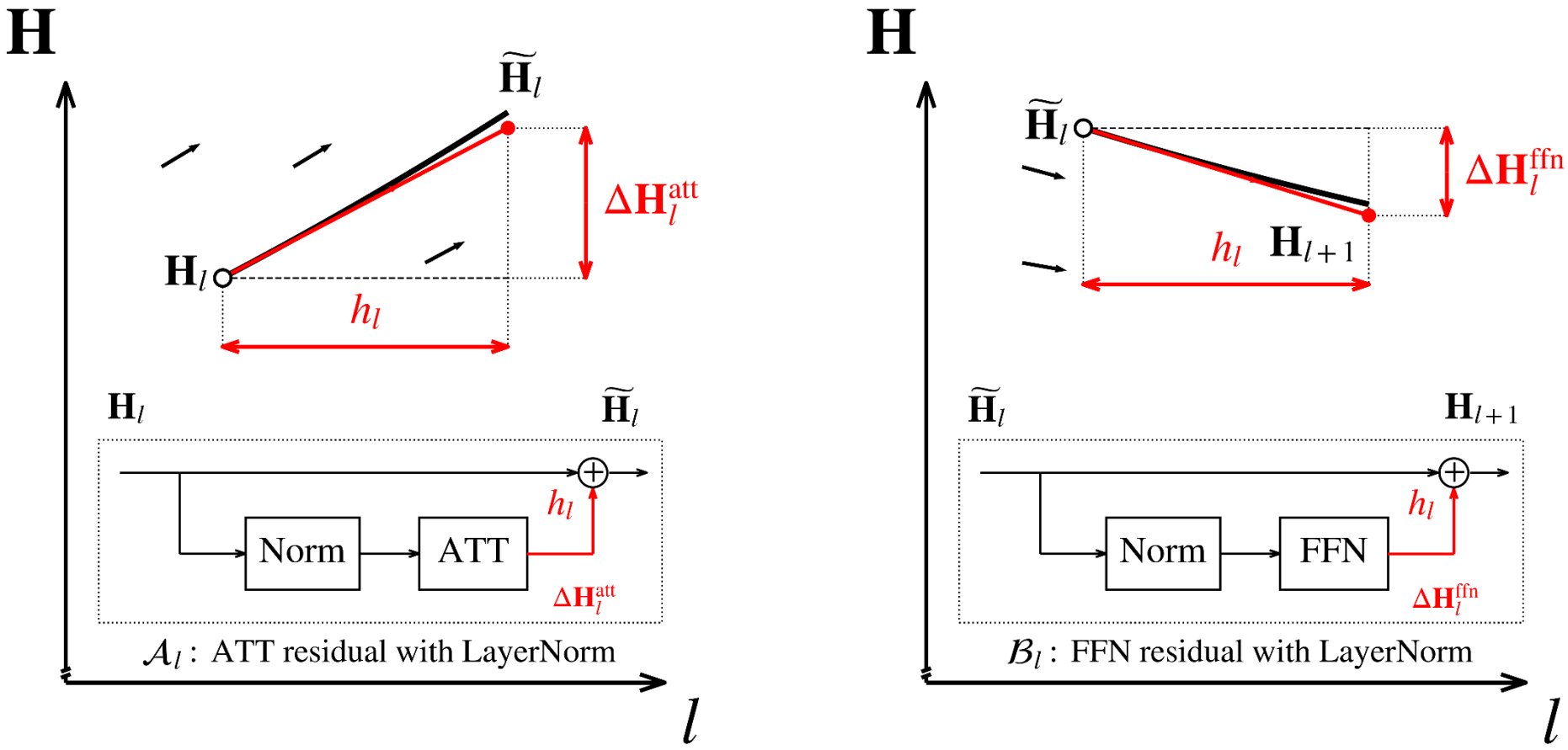
完整 Transformer 层包含两次子步进



$$\boldsymbol{H}_l \overset{\mathcal{A}_l}{\rightarrow} \widetilde{\boldsymbol{H}}_l \overset{\mathcal{B}_l}{\rightarrow} \boldsymbol{H}_{l+1}$$

一个 Transformer 层可以视为自注意力和前馈网络依次执行的两次 **离散状态步进**

ATT 与 FFN 对应不同的局部更新

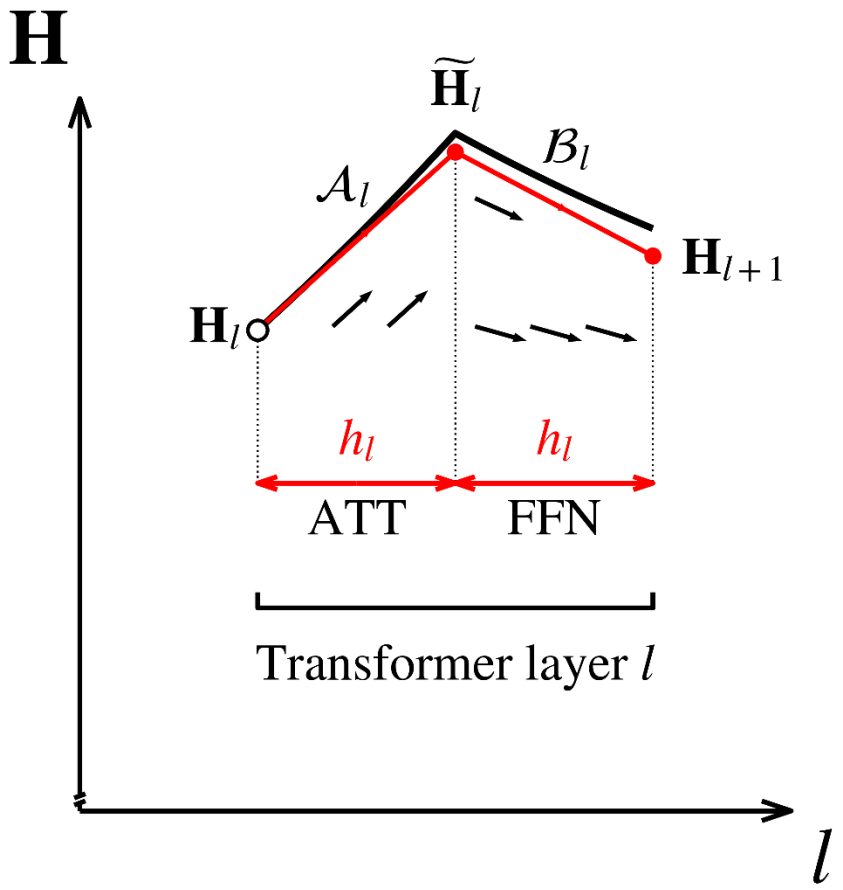


$$\widetilde{\boldsymbol{H}}_l = \boldsymbol{H}_l + h_l \mathcal{A}_l\left(H_l; \theta_l^{\text{att}}\right)$$

$$\boldsymbol{H}_{l+1} = \widetilde{\boldsymbol{H}}_l + h_l \mathcal{B}_l\left(\widetilde{H}_l; \theta_l^{\text{ffn}}\right)$$

自注意力 负责跨 token 的信息耦合，**前馈网络** 负责位置内的非线性变换

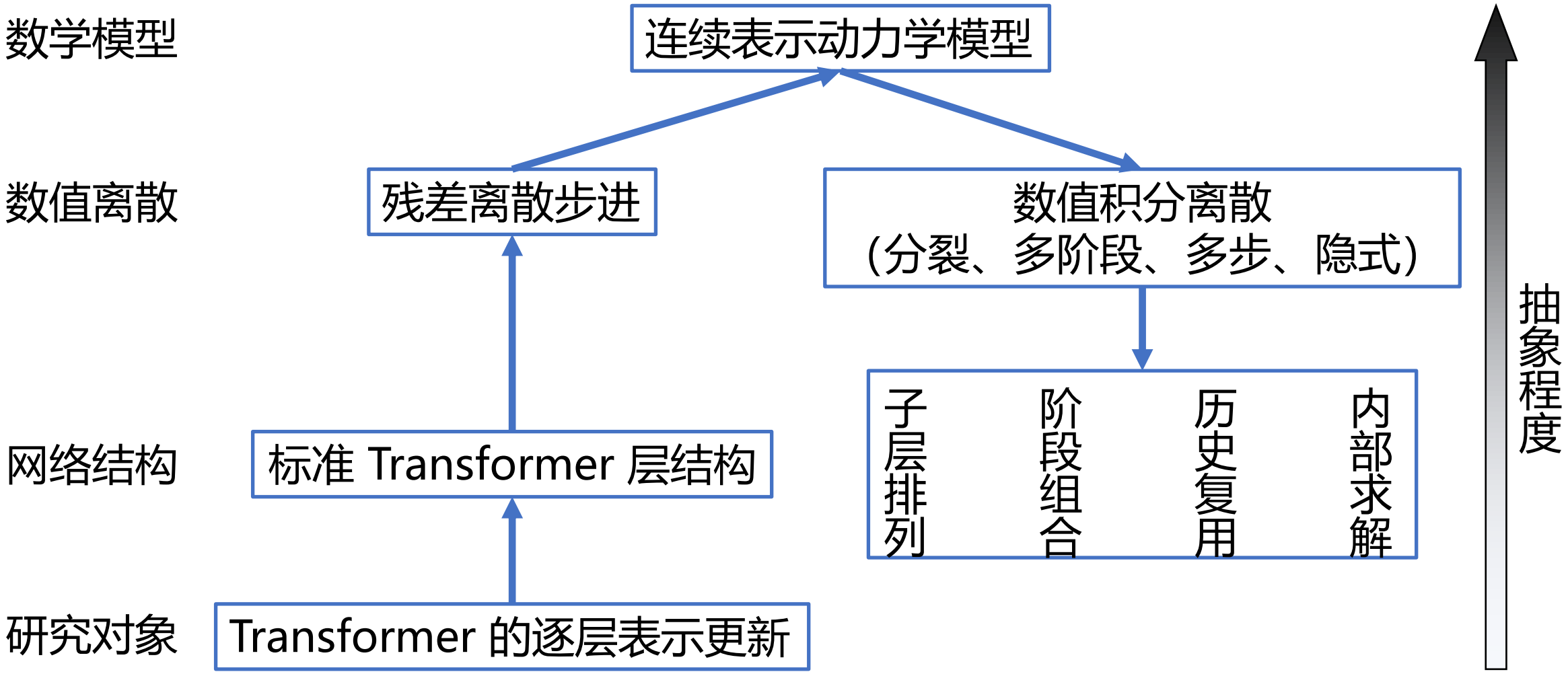
从残差网络走向算子分裂



$$\frac{\mathrm{d}\boldsymbol{H}}{\mathrm{d}t} = \mathcal{A}(\boldsymbol{H}, t) + \mathcal{B}(\boldsymbol{H}, t)$$

改变积分与分裂格式，是否能构造具有不同精度与稳定性的 Transformer 模块?

连续表示动力学下 Transformer 的分析与架构设计



算子分裂：一种求解微分方程的数值方法

与逼近、离散化、迭代等思想类似，**分裂** 也是数值分析的一种核心思想。

该思想主张将复杂整体拆分成若干简单部分，分别处理，再以受控的方式重新组合。

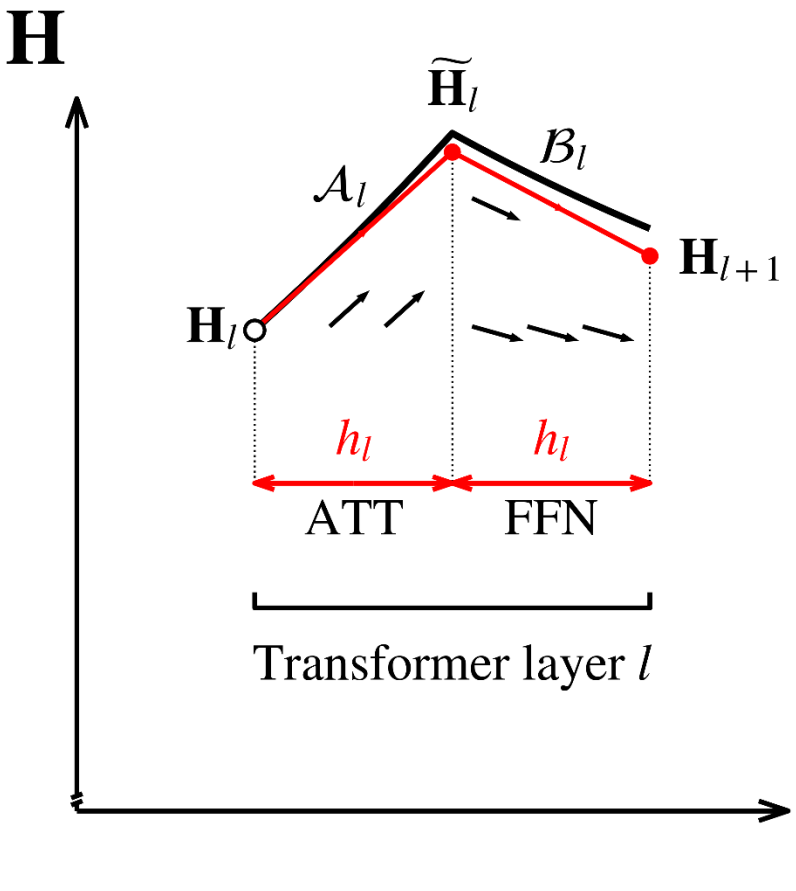
在所有应用中，计算分裂项的解比计算合并后的解更快。

然而，这种优势是以分裂引入的误差为代价的。

因此，人们设计了各种策略来控制这种误差。

S. MacNamara and G. Strang (2016), Operator splitting, in Splitting Methods in Communication, Imaging, Science, and Engineering (R. Glowinski, S. J. Osher and W. Yin,eds), Springer, pp. 95–114.

Lie–Trotter 分裂：标准 Transformer 的串行子层



$$\frac{\mathrm{d}\boldsymbol{H}}{\mathrm{d}t} = \mathcal{A}(\boldsymbol{H}) + \mathcal{B}(\boldsymbol{H}) \quad \Rightarrow \quad \boldsymbol{H}_{l+1} = \Phi^{\mathcal{B}}_{h_l} \circ \Phi^{\mathcal{A}}_{h_l}(\boldsymbol{H}_l)$$

注意力与前馈网络的顺序组合对应一阶算子分裂

Strang 分裂：从 Transformer 到 Macaron 结构

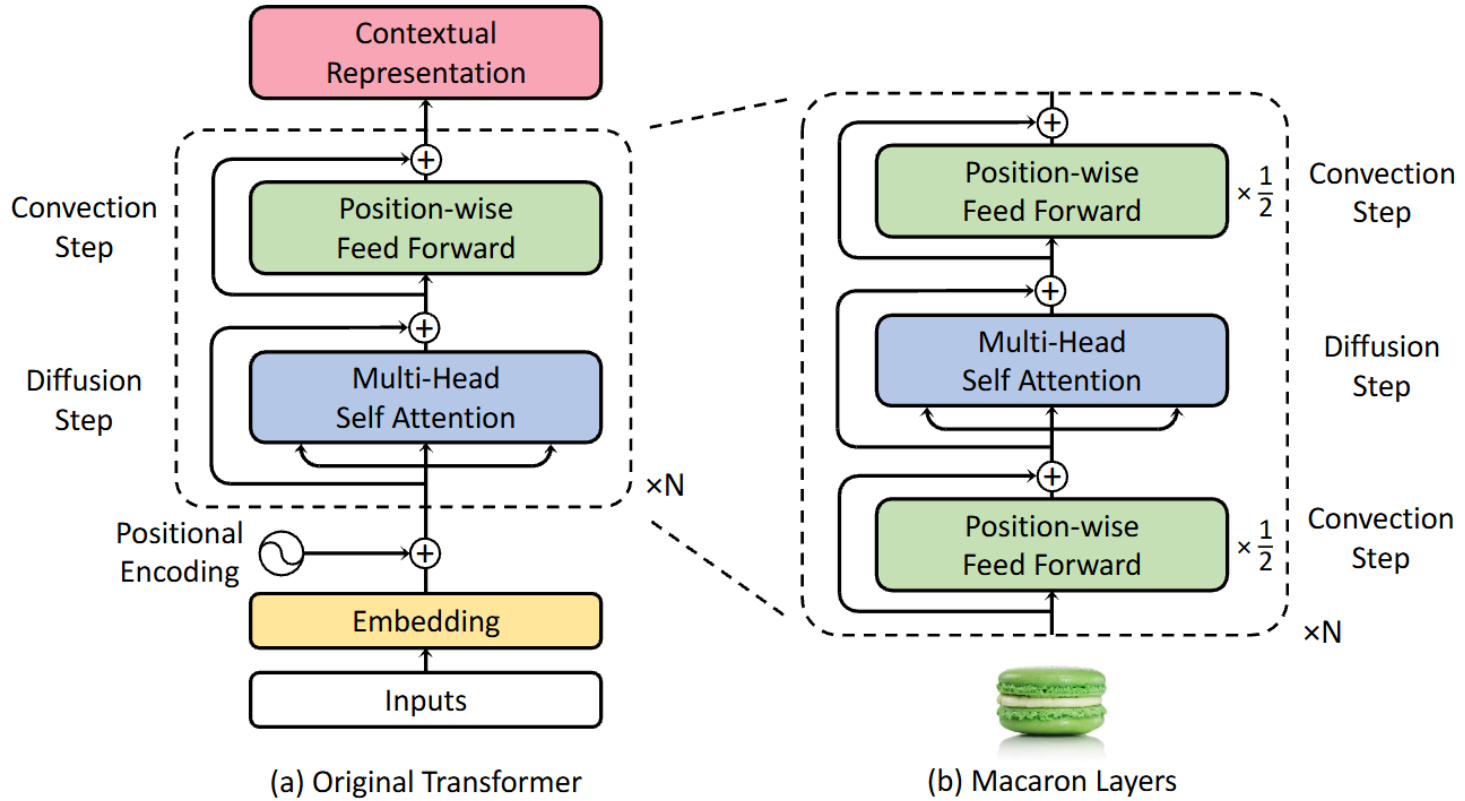


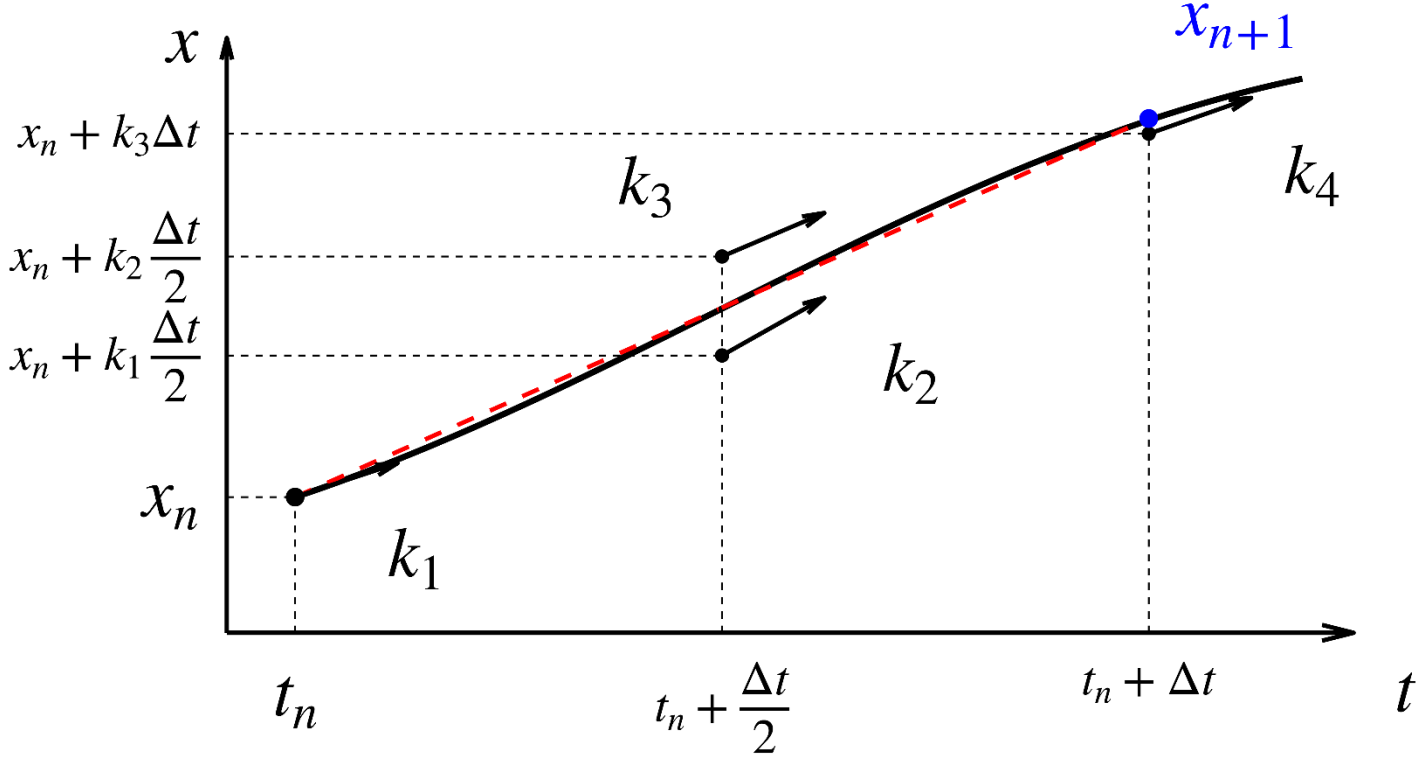
Figure 2: The Transformer and our Macaron architectures.

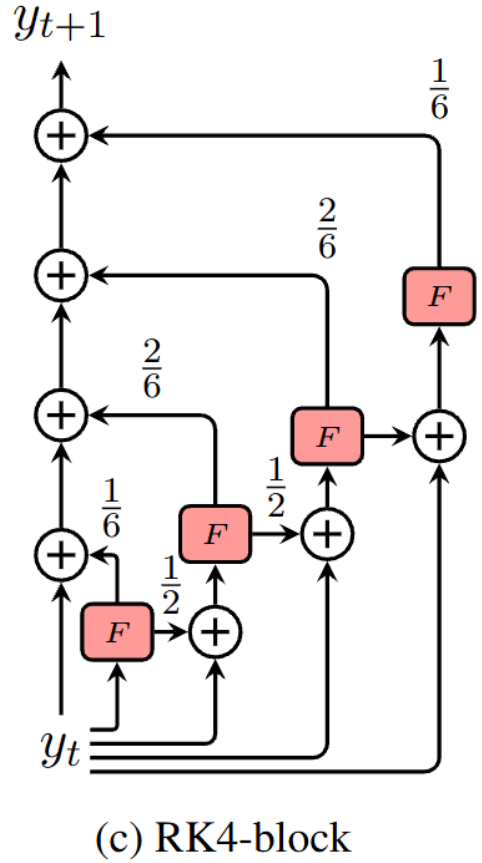
$$\boldsymbol{H}_{l+1} = \Phi^{\mathcal{B}}_{h_l} \circ \Phi^{\mathcal{A}}_{h_l}(\boldsymbol{H}_l) \quad \Rightarrow \quad \boldsymbol{H}_{l+1} = \Phi^{\mathcal{B}}_{h_l/2} \circ \Phi^{\mathcal{A}}_{h_l} \circ \Phi^{\mathcal{B}}_{h_l/2}(\boldsymbol{H}_l)$$

对称的“前馈–注意力–前馈”结构借鉴二阶算子分裂思想

Lu et al., "Understanding and Improving Transformer from a Multi-Particle Dynamic System Point of View.", ICLR 2020 Workshops: DeepDiffEq, 2020.

Runge–Kutta 高阶更新





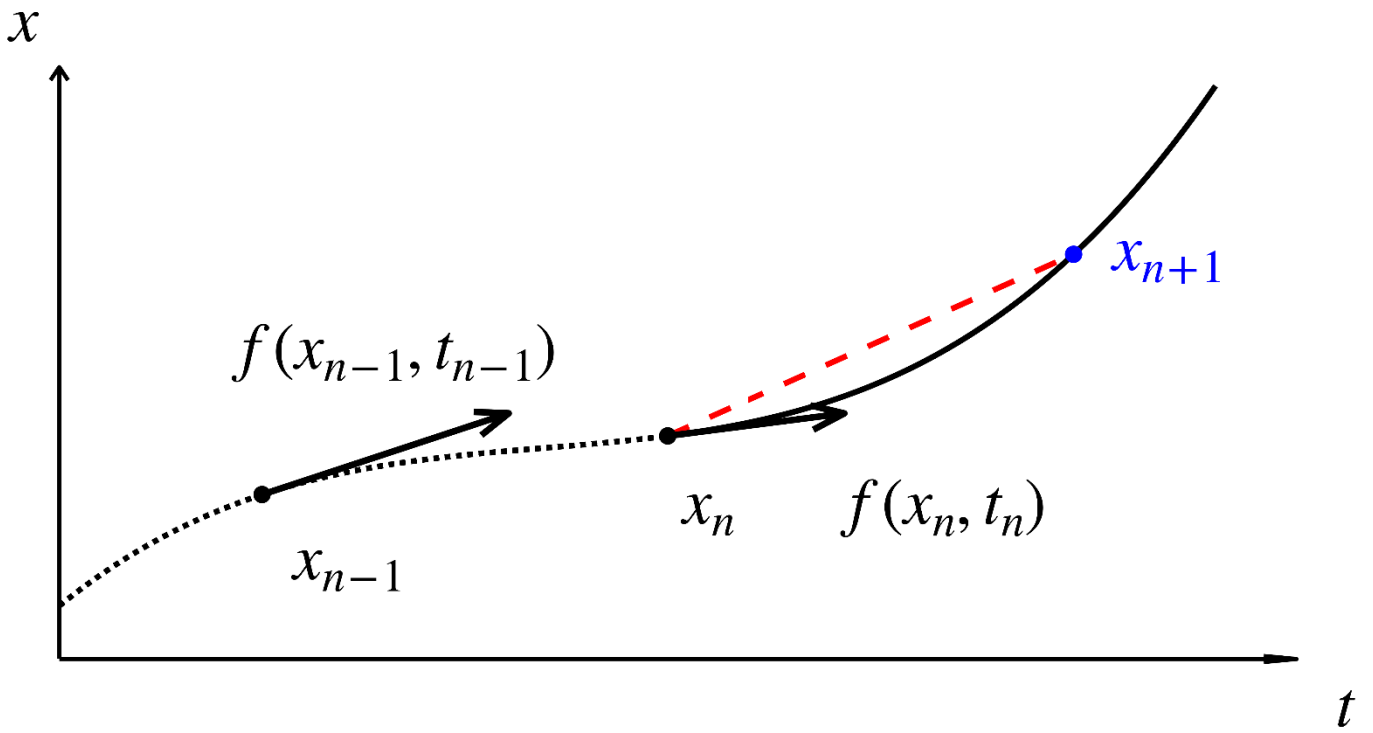
(c) RK4-block

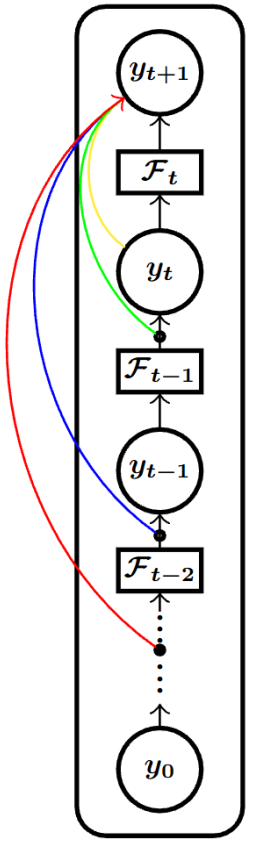
$$x_{n+1} = x_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4), \qquad \Delta t = 1$$

ODE Transformer 通过共享参数的四阶段斜率加权组合实现高阶表示更新

Li, Bei, et al., " ODE Transformer: An Ordinary Differential Equation-Inspired Model for Sequence Generation.", ACL 2022.

Adams–Bashforth 多步更新





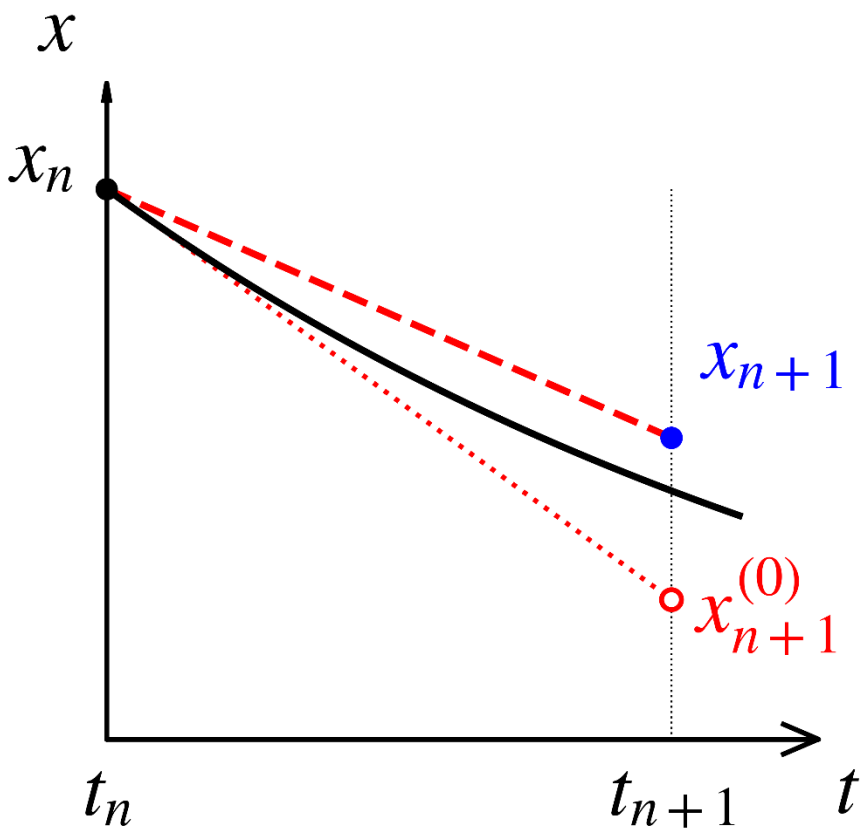
1-Order, Multi-Step

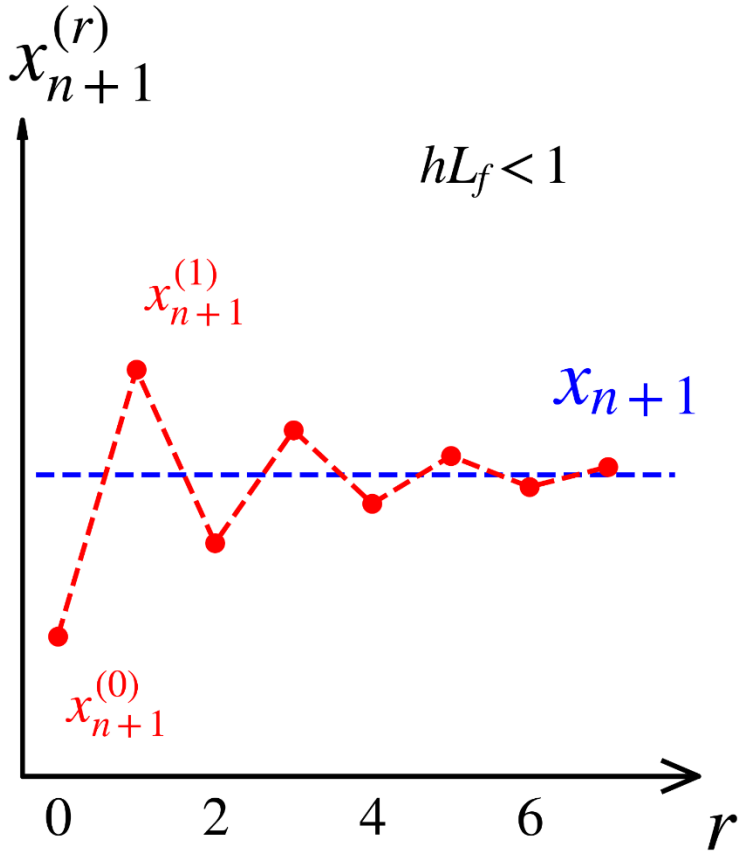
$$\boldsymbol{x_{n+1}} = x_n + \frac{\mathbf{3}}{\mathbf{2}}\boldsymbol{\Delta t \cdot f(x_n, t_n)} - \frac{\mathbf{1}}{\mathbf{2}}\boldsymbol{\Delta t \cdot f(x_{n-1}, t_{n-1})}$$

通过复用历史层的变化率并加权组合实现多步表示更新

Li, Bei, et al., "Predictor-corrector enhanced transformers with exponential moving average coefficient learning.", NeurIPS 2024.

隐式更新与迭代校正

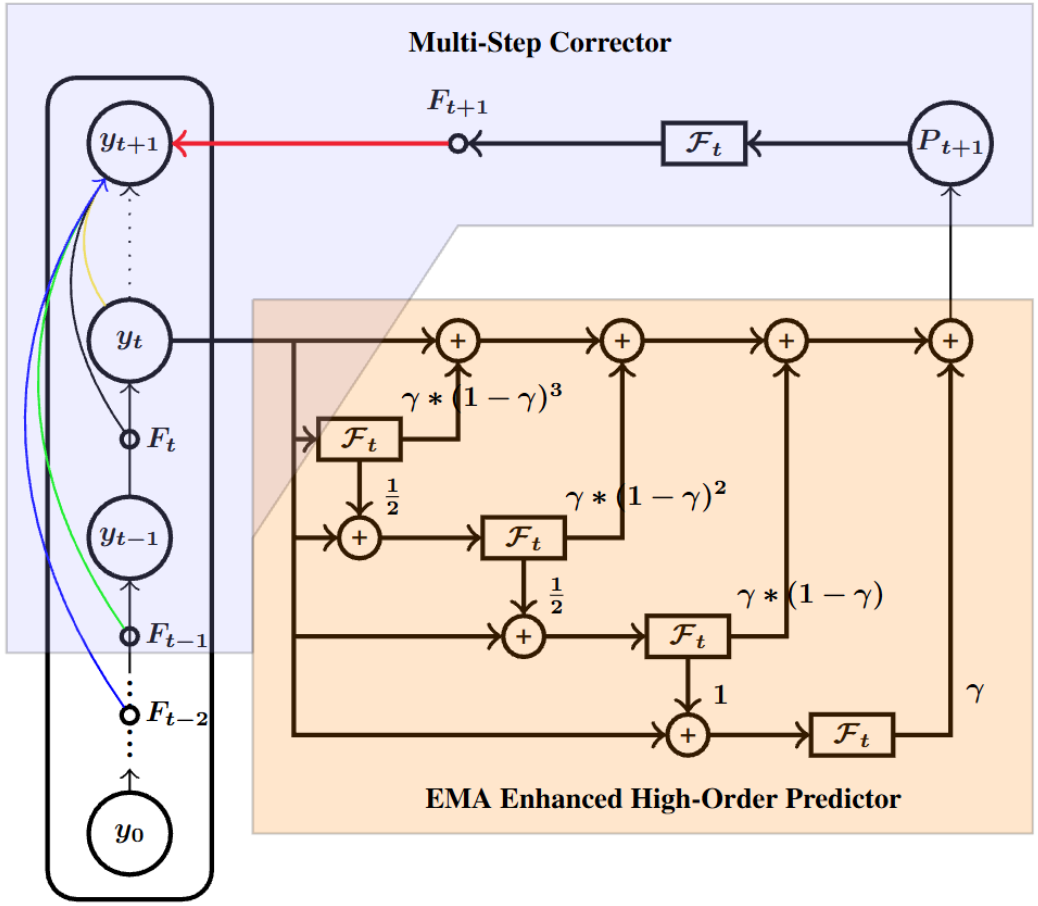




$$x_{n+1}^{(0)} = x_n + hf(x_n, t_n) \qquad x_{n+1}^{(r+1)} = x_n + hf\left(x_{n+1}^{(r)}, t_{n+1}\right), \quad r = 0,1,\ldots$$

通过显式预测与定点迭代逐步逼近隐式更新的解

预测–校正的网络实现



High-Order Predictor, Multi-Step Corrector

(b) Predictor-Corrector Paradigm

通过 EMA 高阶预测与历史多步校正实现 Transformer 表示更新

Li, Bei, et al., "Predictor-corrector enhanced transformers with exponential moving average coefficient learning.", NeurIPS 2024.

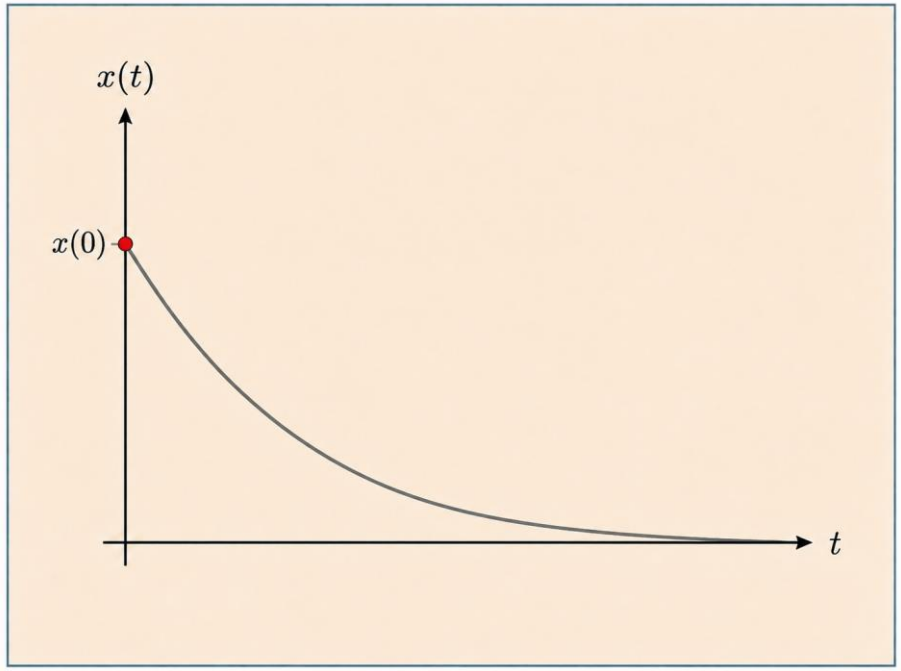
4.2.3 表示动力学的刚性与数值稳定性

- ODE数值解的稳定性
  - 考虑以下ODE：

$$\frac{dx(t)}{dt} = -\lambda x(t), \qquad \lambda > 0$$

  - 其解析解为

$$x(t) = x(0)e^{-\lambda t}$$



4.2.3 表示动力学的刚性与数值稳定性

- ODE数值解的稳定性

$$\frac{dx(t)}{dt} = -\lambda x(t), \qquad \lambda > 0$$

- 离散化后该ODE是否还稳定?

- 考虑前向欧拉离散化

$$\begin{aligned} x_{k+1} &= x_k - h\lambda x_k \\ &= (1 - h\lambda)x_k \end{aligned}$$

- 要使数值解稳定，需要有

$$|1 - h\lambda| < 1$$

4.2.3 表示动力学的刚性与数值稳定性

- ODE数值解的稳定性

$$\frac{dx(t)}{dt} = -\lambda x(t), \qquad \lambda > 0$$

- 离散化后该ODE是否还稳定？

数值方法的稳定域：在该范围内，
数值迭代不会把扰动不断放大

- 考虑前向欧拉离散化

$$\begin{aligned} x_{k+1} &= x_k - h\lambda x_k \\ &= (1 - h\lambda)x_k \end{aligned}$$

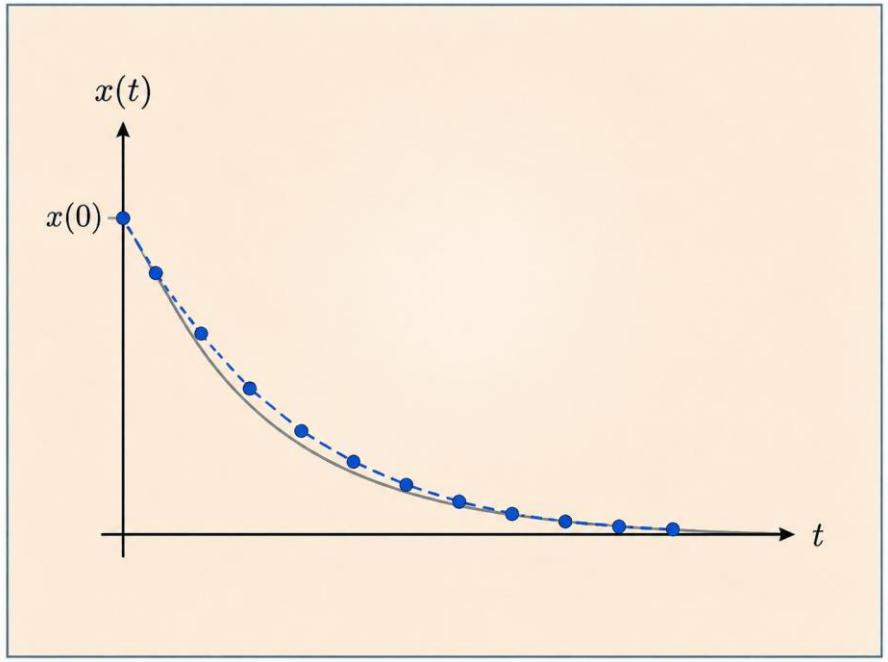
- 要使数值解稳定，需要有

$$|1 - h\lambda| < 1$$

4.2.3 表示动力学的刚性与数值稳定性

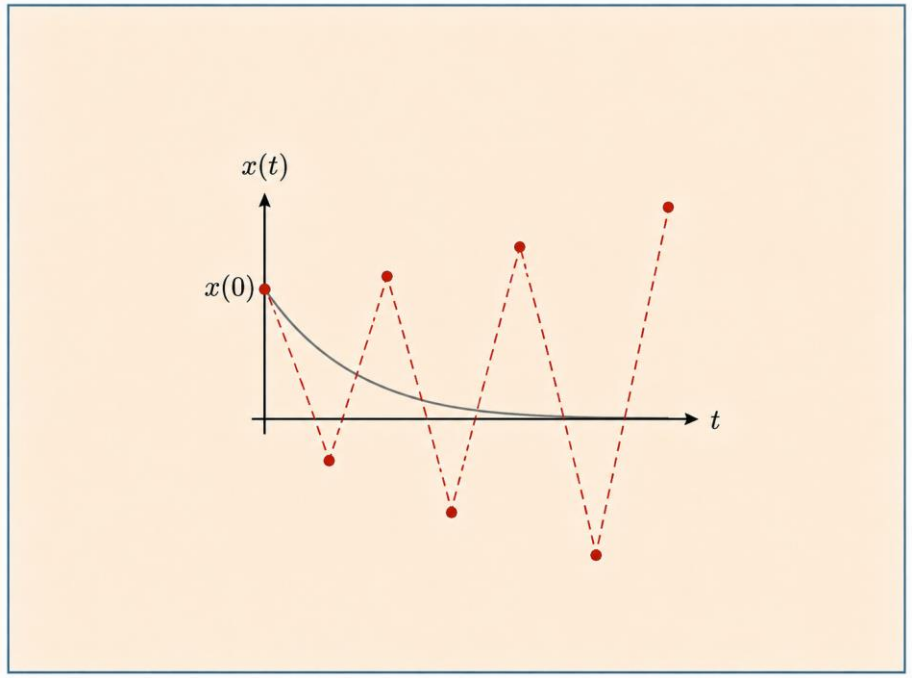
- ODE数值解的稳定性

$$\frac{dx(t)}{dt} = -\lambda x(t), \qquad \lambda > 0$$



$|1 - h\lambda| < 1$

数值稳定



$|1 - h\lambda| > 1$

数值不稳定

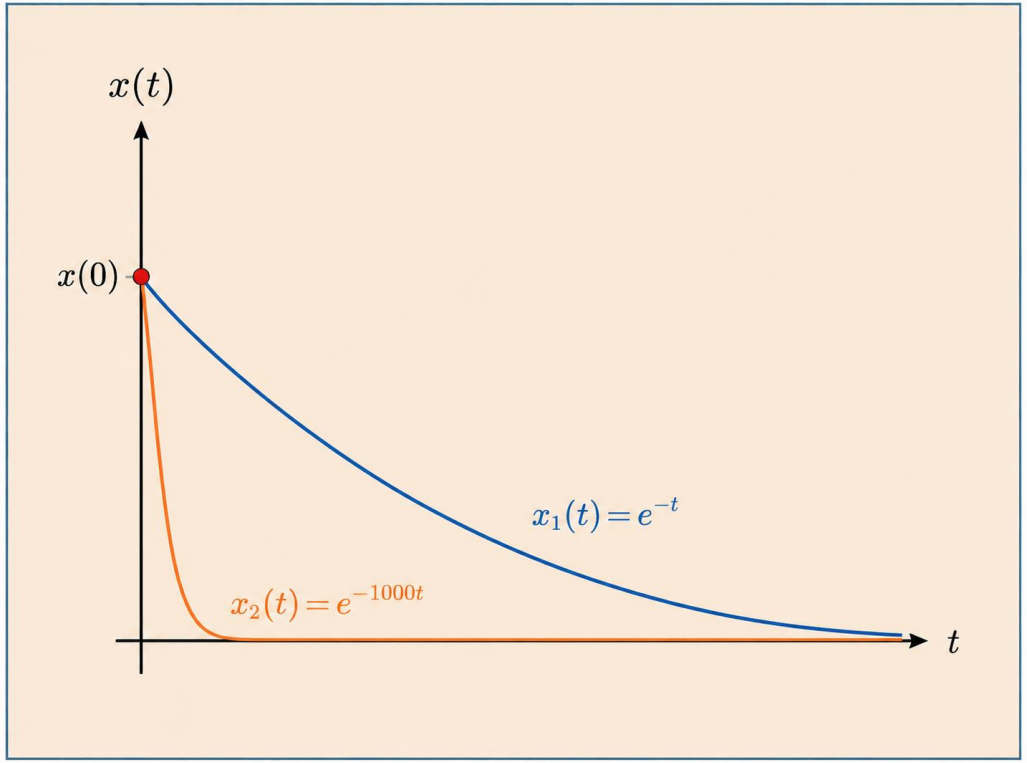
4.2.3 表示动力学的刚性与数值稳定性

- 刚性
  - 动力系统中，不同分量在时间上的变化尺度可能相差很大
  - 例如以下的系统：

$$\begin{cases} \frac{dx_1(t)}{dt} = -x_1(t), \\ \frac{dx_2(t)}{dt} = -1000x_2(t). \end{cases}$$

  - 解析解：

$$\begin{cases} x_1(t) = x_1(0)e^{-t}, \\ x_2(t) = x_2(0)e^{-1000t}. \end{cases}$$



x_1衰减较慢，因此可以接受较大步长；

x_2变化很快，因此需要使用大步长

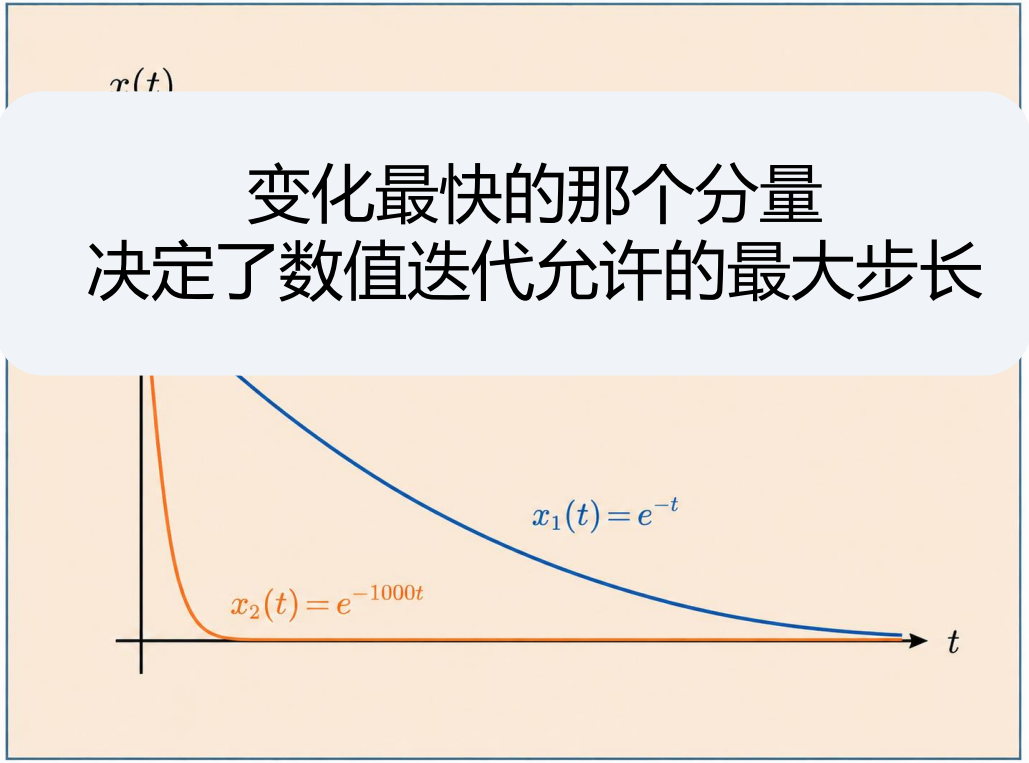
4.2.3 表示动力学的刚性与数值稳定性

- 刚性
  - 动力系统中，不同分量在时间上的变化尺度可能相差很大
  - 例如以下的系统：

$$\begin{cases} \dfrac{dx_1(t)}{dt} = -x_1(t), \\ \dfrac{dx_2(t)}{dt} = -1000x_2(t). \end{cases}$$

  - 解析解：

$$\begin{cases} x_1(t) = x_1(0)e^{-t}, \\ x_2(t) = x_2(0)e^{-1000t}. \end{cases}$$



x_1衰减较慢，因此可以接受较大步长；

x_2变化很快，因此需要使用大步长

4.2.3 表示动力学的刚性与数值稳定性

- Transformer网络中的刚性

  - Transformer中隐藏状态的不同分量也可能具有显著不同的变化速率：部分分量快速变化，部分分量缓慢变化。

  - 在刚性系统中，Euler、Runge–Kutta等显式方法的稳定域有限，往往需要很小的步长

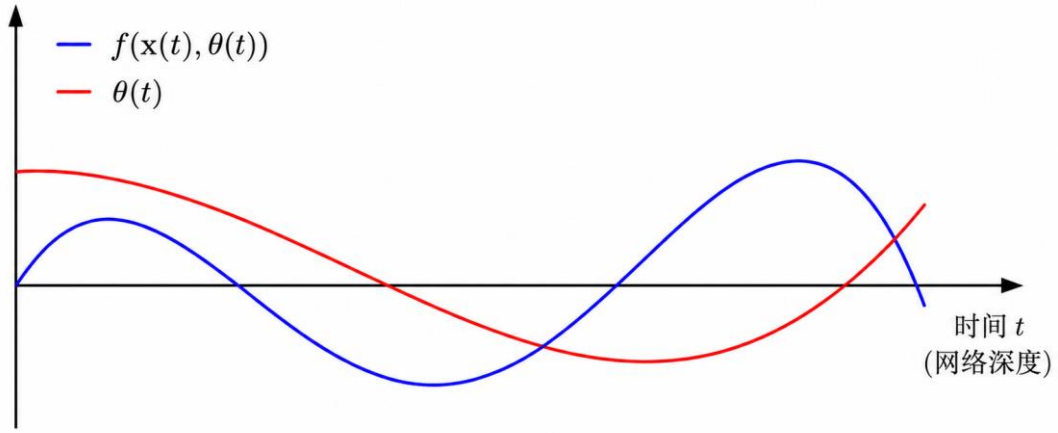
  - 在建模刚性问题时，隐式建模方法具有更好的稳定性

4.2.3 表示动力学的刚性与数值稳定性

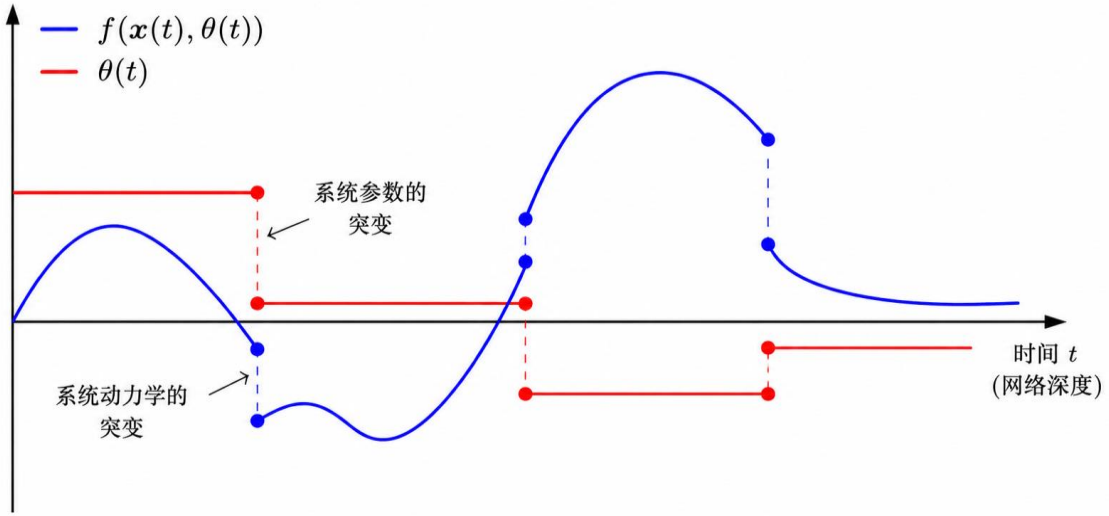
- Transformer层间参数的变化也可能导致数值不稳定

$$\frac{d\mathbf{x}(t)}{dt} = \begin{cases} F\big(\mathbf{x}(t), \theta_1\big), & t \in [t_0, t_1), \\ F\big(\mathbf{x}(t), \theta_2\big), & t \in [t_1, t_2), \\ \dots \\ F\big(\mathbf{x}(t), \theta_L\big), & t \in [t_{L-1}, t_L]. \end{cases}$$

- 由于Transformer的层是离散的，不同层具有不同的参数，因此其动力学可看作关于层的分段函数



随时间平滑演化的系统动力学与系统参数



由层间参数突变引起的系统动力学剧烈变化

4.2.3 表示动力学的刚性与数值稳定性

- 如何解决Transformer中由刚性和层间参数突变导致的数值不稳定?
  - Lipschitz约束

$$\|F(\mathbf{x}_1) - F(\mathbf{x}_2)\| \leq K \|\mathbf{x}_1 - \mathbf{x}_2\| .$$

  - 参数正则化

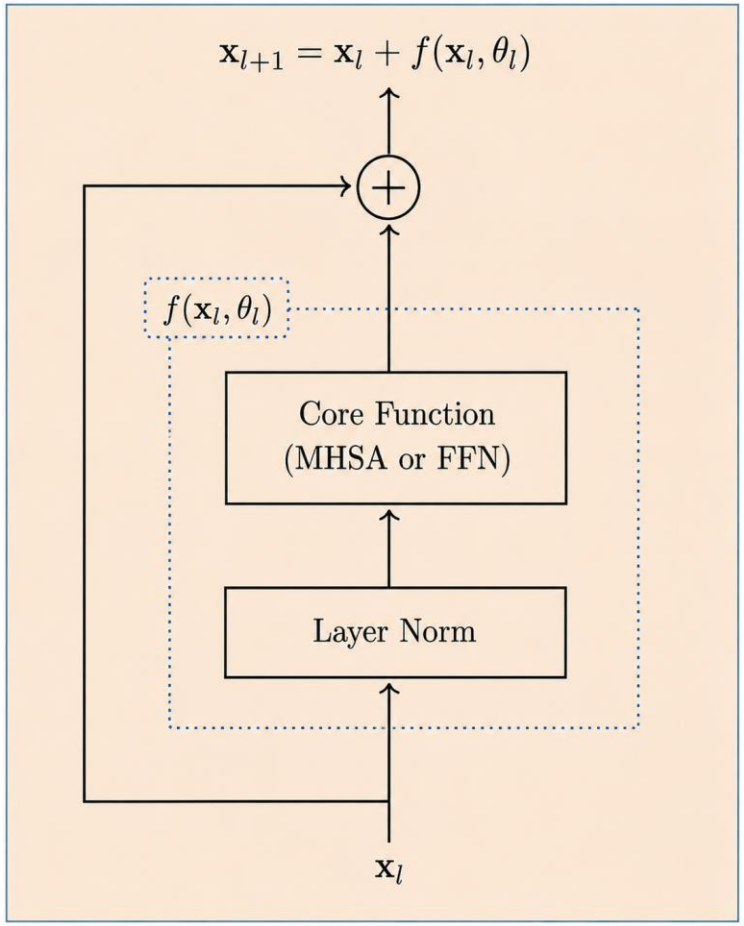
$$\mathcal{L}_{\text{param}} = \lambda_{\text{param}} \sum_{k=1}^{L-1} \|\theta_{k+1} - \theta_k\|_2^2 .$$

  - 状态正则化

$$\mathcal{L}_{\text{state}} = \lambda_{\text{state}} \sum_{k=1}^{L-1} \|\mathbf{x}_{k+1} - \mathbf{x}_k\|_2^2 .$$

4.2.4 Neural ODE

- 从离散深度到连续深度
  - 残差网络中，隐藏状态的更新可以写成ODE的前向欧拉离散化：

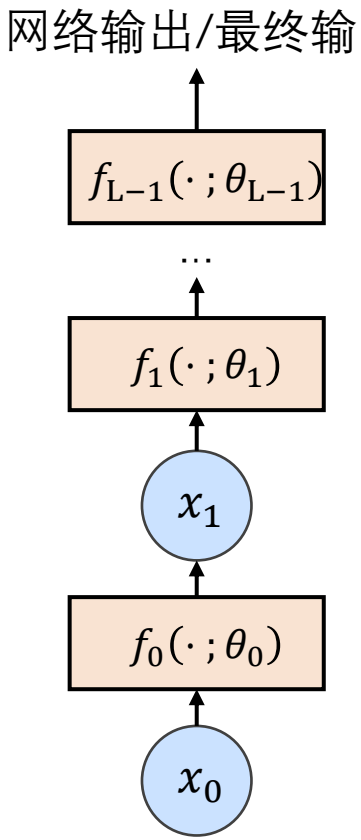


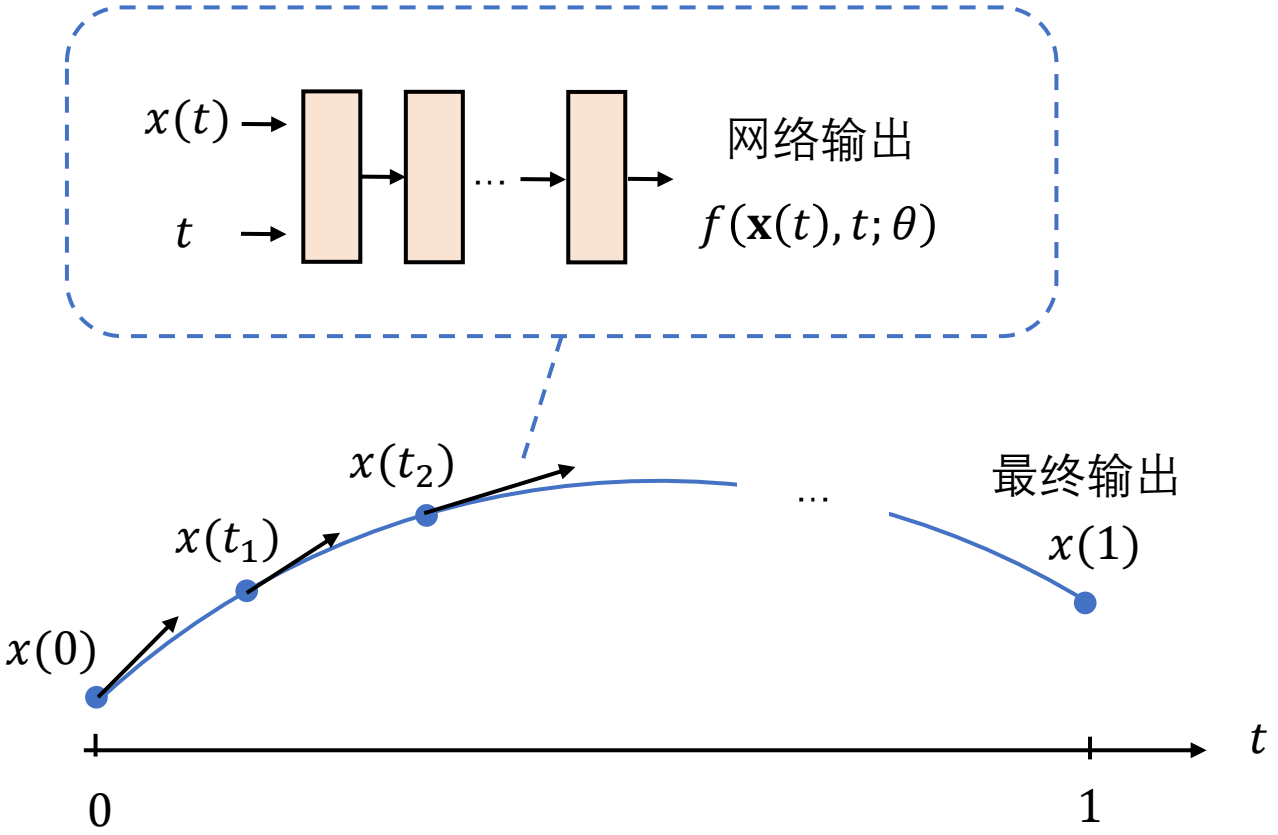
$$\frac{d\mathbf{x}(t)}{dt} = f\big(\mathbf{x}(t), \theta(t)\big)$$

  - 无法刻画连续的状态变化

4.2.4 Neural ODE

- 从离散深度到连续深度
  - Neural ODE：直接使用网络建模连续动力学





$$\mathbf{x}_{l+1} = \mathbf{x}_l + f_l(\mathbf{x}_l;\theta_l).$$

传统网络（离散深度）

$$\frac{d\mathbf{x}(t)}{dt} = f(\mathbf{x}(t),t;\theta)$$

Neural ODE（连续深度）

4.2.4 Neural ODE

- Neural ODE的推理
  - 前向过程：积分法求解ODE

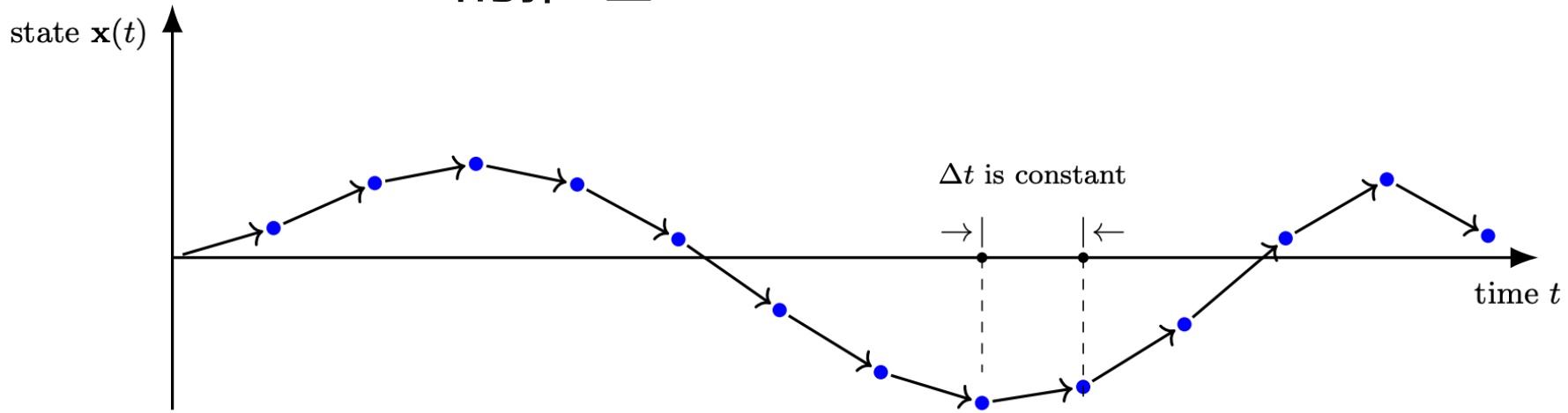
$$\mathbf{x}(t_1) = \mathbf{x}(t_0) + \int_{t_0}^{t_1} f\left(\mathbf{x}(\tau), \tau; \theta\right) d\tau$$

  - 当f是复杂网络时，以上积分无法解析计算，因此需要使用ODE的数值方法（例如欧拉法/龙格库塔法等）求解

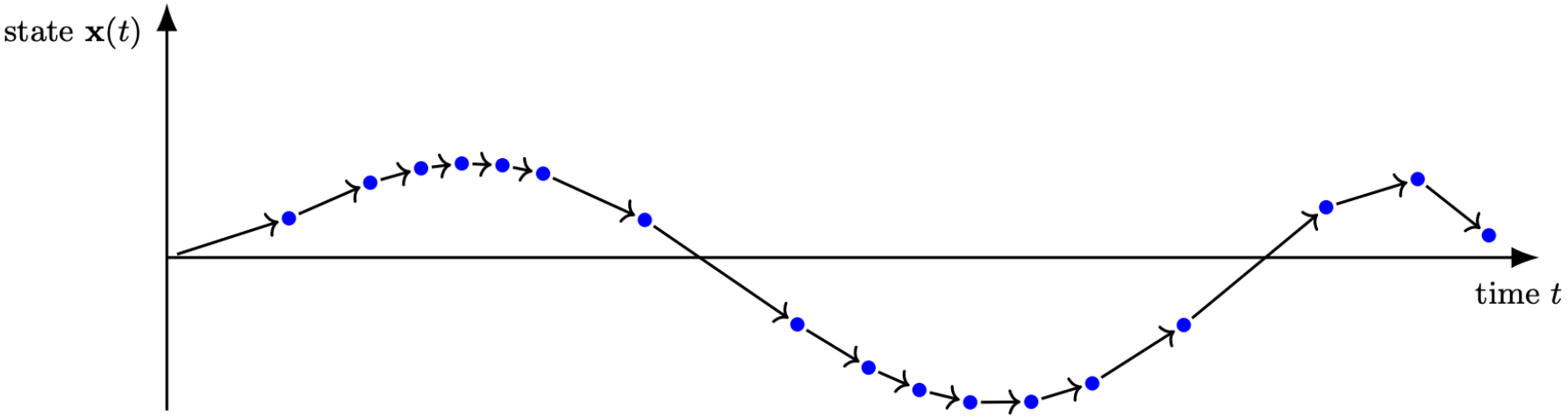
  - Neural ODE允许自适应步长的ODE求解

4.2.4 Neural ODE

- Neural ODE的推理



残差网络：只允许固定时间步的求解（受离散的层数限制）



Neural ODE：允许连续或动态步长的求解

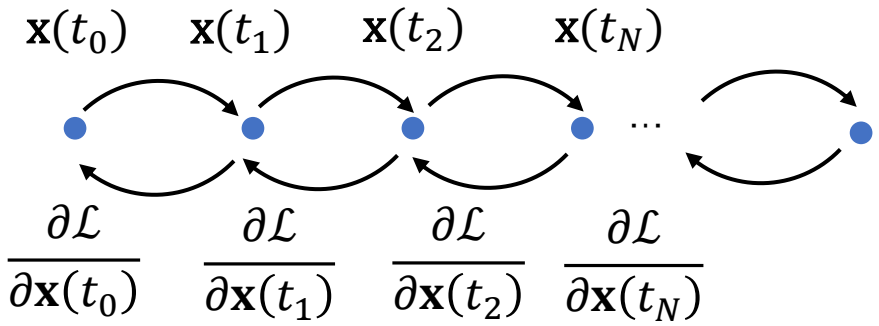
4.2.4 Neural ODE

- 传统深层网络与Neural ODE的对比

比较维度	固定深度残差网络	Neural ODE
网络建模对象	最终输出（即建模目标本身）	给定时刻t，轨迹上点$x(t)$处的速度
状态更新	残差层	由网络输出速度， ODE数值求解器进行状态更新
参数形式	通常每一层具有独立参数 θ_l	通常共享动力学参数 θ，并可显式依赖时间 t
计算步数	网络结构设计时即固定	可以根据需要动态调整
中间状态	对应固定层输出	ODE求解过程中不同时间节点对应的状态
误差控制	由网络结构和参数训练等影响	可通过ODE求解器的步长等参数显式控制

4.2.5 Neural ODE的训练与伴随灵敏度法

- Neural ODE的训练
  - 如何对Neural ODE进行反向传播?
  - 一个直接的方法是把ODE前向的所有状态记录下来，逐步进行反向传播，在此过程中累积参数梯度



  - 需要保存ODE求解过程中每一步的状态以及反向传播所需的中间计算结果，开销过大

4.2.5 Neural ODE的训练与伴随灵敏度法

- 伴随变量
  - 如何避免保存全部中间状态?
  - 引入伴随变量

$$\mathbf{a}(t) = \frac{\partial \mathcal{L}}{\partial \mathbf{x}(t)}$$

  - $\mathbf{x}(t)$ 、 $\mathbf{a}(t)$均为列向量
  - 伴随变量描述最终损失对当前状态的导数
  - 终点t_1处伴随变量可以直接由损失函数得到 $\mathbf{a}(t_1) = \frac{\partial \mathcal{L}}{\partial \mathbf{x}(t_1)}$

4.2.5 Neural ODE的训练与伴随灵敏度法

- 伴随变量随时间的演化

  - **使用ODE来描述伴随变量随时间的变化**

  - 以前向欧拉求解器为例，离散更新规则为：

$$\mathbf{x}(t+\Delta t)=\mathbf{x}(t)+\Delta t\, f\left(\mathbf{x}(t),\theta,t\right)$$

  - 由链式法则

$$\begin{aligned}\mathbf{a}(t)&=\frac{\partial \mathcal{L}}{\partial \mathbf{x}(t)}\\&=\left(\frac{\partial \mathbf{x}(t+\Delta t)}{\partial \mathbf{x}(t)}\right)^{T}\frac{\partial \mathcal{L}}{\partial \mathbf{x}(t+\Delta t)}\\&=\left(\frac{\partial \mathbf{x}(t+\Delta t)}{\partial \mathbf{x}(t)}\right)^{T}\mathbf{a}(t+\Delta t)\end{aligned}$$

    - $\left(\frac{\partial \mathbf{x}(t+\Delta t)}{\partial \mathbf{x}(t)}\right)$：雅可比矩阵

4.2.5 Neural ODE的训练与伴随灵敏度法

- 伴随变量随时间的演化
  - 代入离散化规则 $\mathbf{x}(t+\Delta t)=\mathbf{x}(t)+\Delta t\, f\left(\mathbf{x}(t),\theta,t\right)$ ：

$$\mathbf{a}(t)=\left(\frac{\partial\left(\mathbf{x}(t)+\Delta t\cdot f\left(\mathbf{x}(t),\theta,t\right)\right)}{\partial\mathbf{x}(t)}\right)^{T}\mathbf{a}(t+\Delta t)$$

$$=\left(\frac{\partial\mathbf{x}(t)+\partial\left[\Delta t\cdot f\left(\mathbf{x}(t),\theta,t\right)\right]}{\partial\mathbf{x}(t)}\right)^{T}\mathbf{a}(t+\Delta t)$$

$$=\left(\mathbf{I}+\Delta t\cdot\frac{\partial f\left(\mathbf{x}(t),\theta,t\right)}{\partial\mathbf{x}(t)}\right)^{T}\mathbf{a}(t+\Delta t)$$

  - 这里建立了 $\mathbf{a}(t)$ 和 $\mathbf{a}(t+\Delta t)$ 的关系

4.2.5 Neural ODE的训练与伴随灵敏度法

- 伴随变量随时间的演化
  - 上式整理可得：

$$\frac{\mathbf{a}(t+\Delta t)-\mathbf{a}(t)}{\Delta t}=-\left(\frac{\partial f(\mathbf{x}(t),\theta,t)}{\partial \mathbf{x}(t)}\right)^{T}\mathbf{a}(t+\Delta t)$$

  - 上式即为离散情形下 $\mathbf{a}(t)$ 的变化规律。在适当条件下，令 $\Delta t \to 0$，得到伴随状态满足的ODE：

$$\frac{d\mathbf{a}(t)}{dt}=-\left(\frac{\partial f(\mathbf{x}(t),\theta,t)}{\partial \mathbf{x}(t)}\right)^{T}\mathbf{a}(t)$$

4.2.5 Neural ODE的训练与伴随灵敏度法

- 使用伴随状态计算参数梯度
  - 参数θ在每个时刻都会影响动力学f，进而影响$x(t)$，以及最终损失

$$\theta \longrightarrow f\left(\mathbf{x}(t), \theta, t\right) \longrightarrow \mathbf{x}(t) \longrightarrow \mathcal{L}$$

  - 时刻t的参数梯度贡献为：

$$\left(\frac{\partial f\left(\mathbf{x}(t), \theta, t\right)}{\partial \theta}\right)^{T} \mathbf{a}(t)$$

  - 沿整条轨迹累积各时刻贡献，得到总参数梯度

$$\frac{\partial \mathcal{L}}{\partial \theta} = -\int_{t_1}^{t_0} \left(\frac{\partial f\left(\mathbf{x}(t), \theta, t\right)}{\partial \theta}\right)^{T} \mathbf{a}(t)\, dt$$

4.2.5 Neural ODE的训练与伴随灵敏度法

- 增广ODE
  - 为了不同时保存所有的中间状态$x(t)$，定义增广状态，同时记录中间状态、伴随状态和参数梯度的变化：

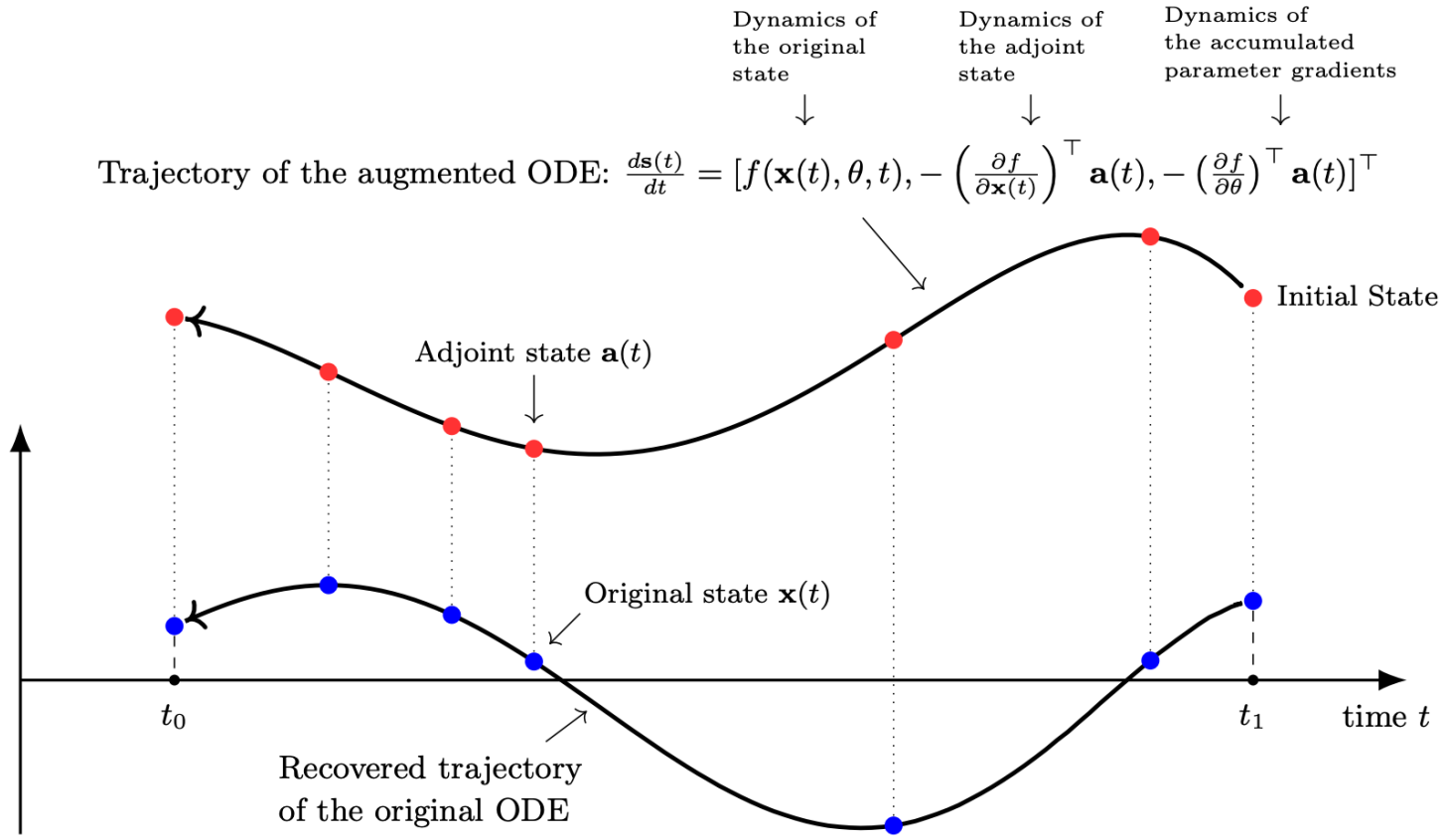
$$\mathbf{s}(t) = \begin{bmatrix} \mathbf{x}(t) \\ \mathbf{a}(t) \\ \frac{\partial \mathcal{L}}{\partial \theta}(t) \end{bmatrix}$$

  - 从终点出发，在反向积分过程中逐步还原$x(t)$，同时计算伴随状态和参数梯度：

$$\mathbf{s}(t_1) = \begin{bmatrix} \mathbf{x}(t_1) \\ \frac{\partial \mathcal{L}}{\partial \mathbf{x}(t_1)} \\ \mathbf{0} \end{bmatrix} \qquad \frac{d\mathbf{s}(t)}{dt} = \begin{bmatrix} f\left(\mathbf{x}(t), \theta, t\right) \\ -\left(\frac{\partial f}{\partial \mathbf{x}(t)}\right)^T \mathbf{a}(t) \\ -\left(\frac{\partial f}{\partial \theta}\right)^T \mathbf{a}(t) \end{bmatrix}$$

4.2.5 Neural ODE的训练与伴随灵敏度法

- 伴随灵敏度法总结



- 前向过程：从t_0到t_1使用离散ODE求解器，前向积分得到终点状态$\mathbf{x}(t_1)$
- 反向过程：构造反向过程的起始状态，即t_1处的增广状态

$$\mathbf{s}(t_1) = \left[\mathbf{x}(t_1), \frac{\partial \mathcal{L}}{\partial \mathbf{x}(t_1)}, \mathbf{0}\right]^T$$

- 使用如下动力学反向积分到 t_0 ：

$$\frac{d\mathbf{s}(t)}{dt} = \left[f(\mathbf{x}(t),\theta,t), -\left(\frac{\partial f}{\partial \mathbf{x}(t)}\right)^T \mathbf{a}(t), -\left(\frac{\partial f}{\partial \theta}\right)^T \mathbf{a}(t)\right]^T$$

深度学习中的连续动力学机制

4.1 连续动力学的数学基础

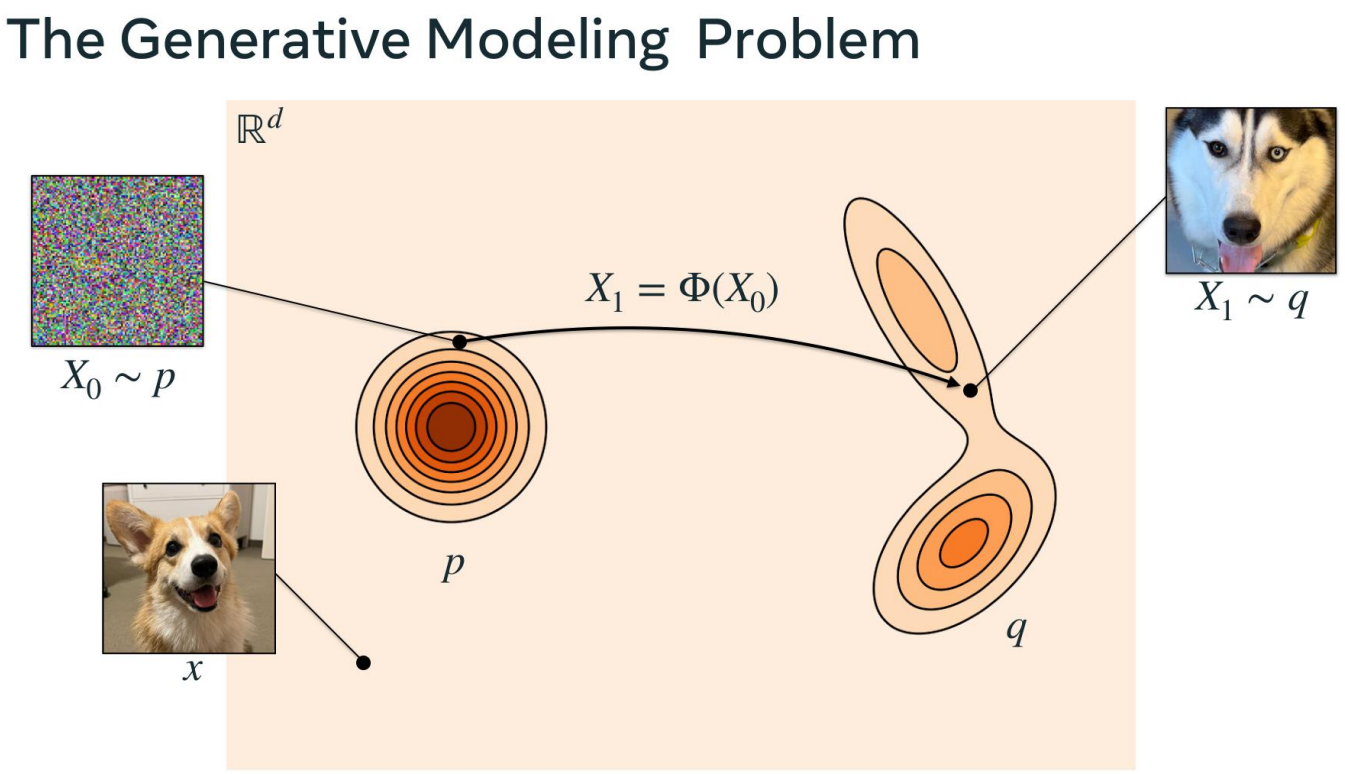
4.2 表示空间中的连续动力学

4.3 确定性概率动力学

4.4 随机概率动力学

4.5 离散状态上的连续时间动力学

4.3.1 从样本轨迹到概率输运



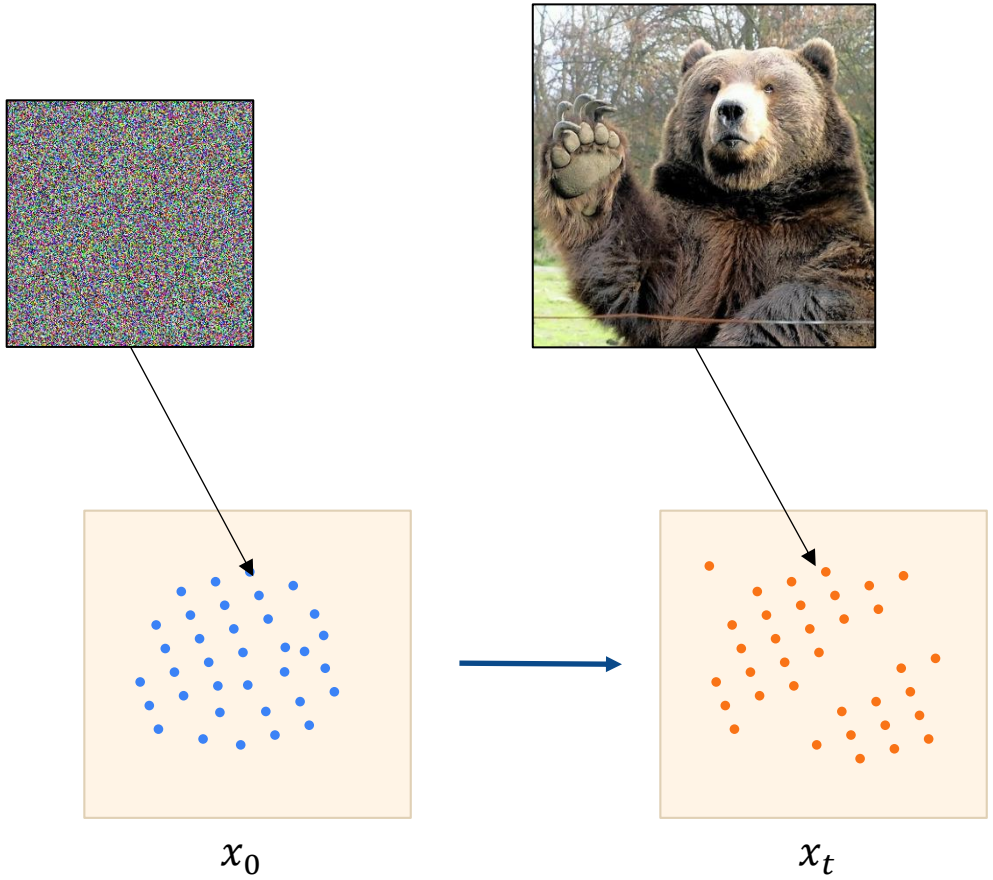
- 生成模型建模的目标不是只生成一个样本点，而是学习两个分布之间的转换。

p_prior ⇄ p_data

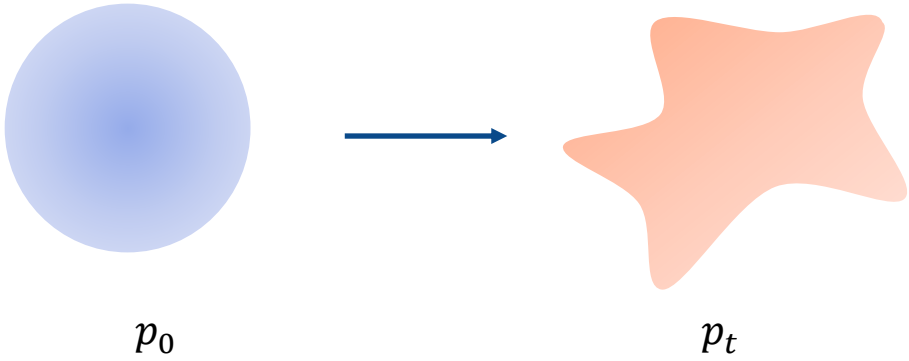
- 先验分布：易于采样
- 数据分布：结构复杂
- 生成过程：从先验分布到数据分布

4.3.1 从样本轨迹到概率输运

- 从“一个状态”到“一群样本”



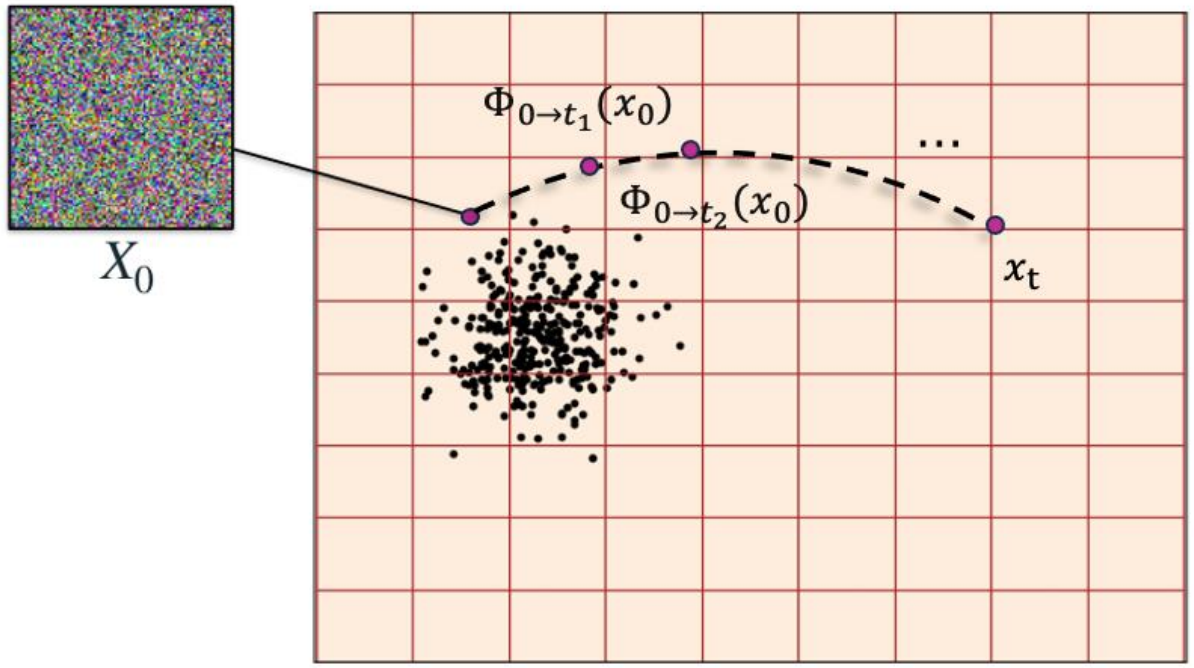
单个样本层面：位置变化



分布层面：概率质量重新分布

4.3.2 流映射与连续性方程

- 流映射



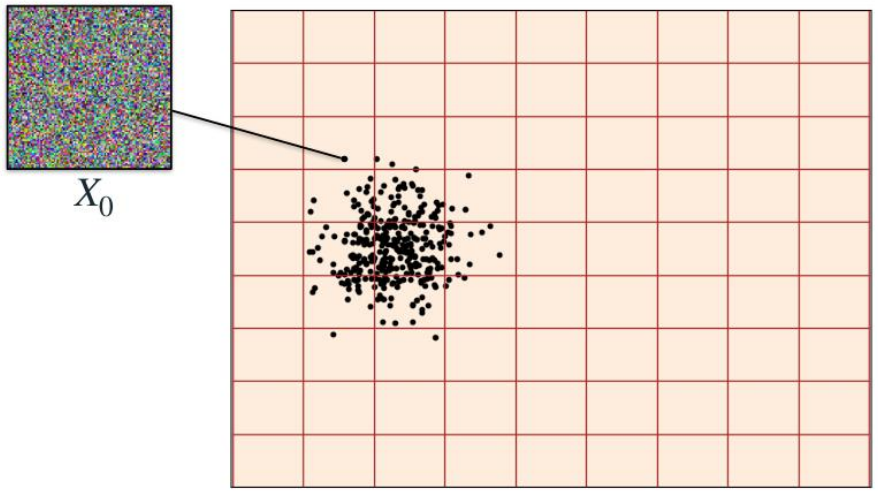
- 流映射$\Phi_{0\to t}$描述任意初始状态空间中的样本x_0在t时刻运动到的位置x_t

$$x(t) = \Phi_{0\to t}(x(0))$$

- 流映射$\Phi(x_0)$像一张地图，记录x_0在每个时刻的位置

4.3.2 流映射与连续性方程

- 流与向量场



$$\frac{\mathrm{d}}{\mathrm{d}t}\psi_t(x) = u_t(\psi_t(x))$$

- 流映射对应的向量场$f(x(t),t)$ 描述流上每个点x(t)在某一时刻t的“瞬时速度”
- 给定向量场f，对应的流映射可以由向量场的积分描述：

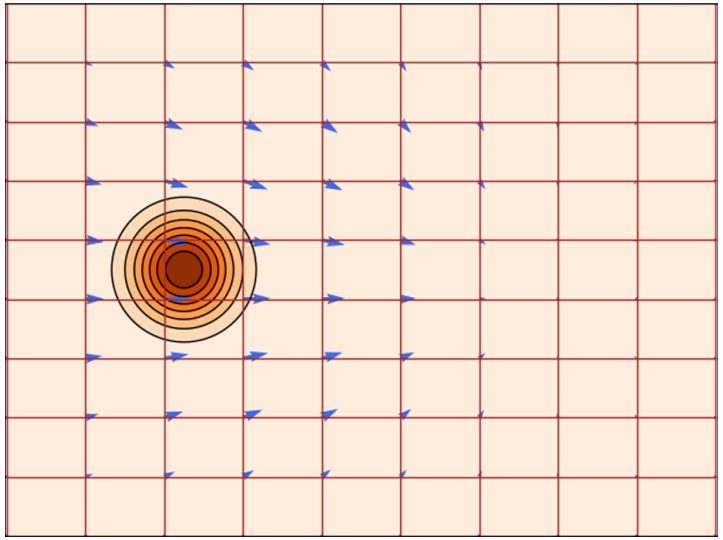
$$\Phi_{s\to t}(\mathbf{x}_s) = \mathbf{x}_s + \int_s^t f\big(\mathbf{x}(\tau),\tau\big)\,\mathrm{d}\tau, \qquad \mathbf{x}(s) = \mathbf{x}_s$$

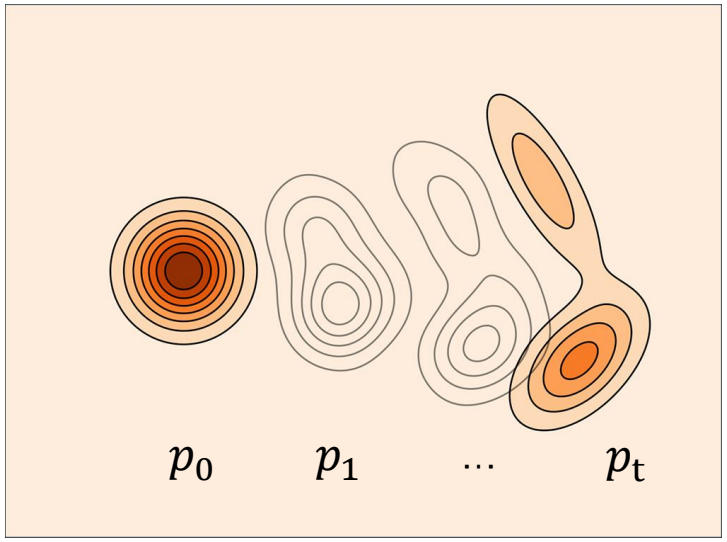
- 满足恒等和复合关系

$$\Phi_{s\to s} = \mathrm{Id}, \qquad \Phi_{r\to t} = \Phi_{s\to t}\circ\Phi_{r\to s}, \quad r \le s \le t.$$

4.3.2 流映射与连续性方程

- 流与概率分布





- 流映射描述单个样本点如何在状态空间中变化

$$x(t) = \Phi_{0\to t}(x(0)) = x(0) + \int_0^t f(x(s), s)\, ds.$$

- 假设初始分布为p_0，则流映射导出一族概率分布

$$p_t = \left(\Phi_{0\to t}\right)_{\#} p_0$$

样本点的运动
导致分布的变化

4.3.2 流映射与连续性方程

- 连续性方程
  - 已知每个样本点的运动速度$f(x(t),t)$，怎样描述整个概率密度p_t随时间的变化?
    - 连续性方程

$$\frac{\partial p_t(\mathbf{x})}{\partial t} + \nabla_{\mathbf{x}} \cdot (p_t(\mathbf{x}) f(\mathbf{x}, t)) = 0$$

- 其中∇_x为散度算子:

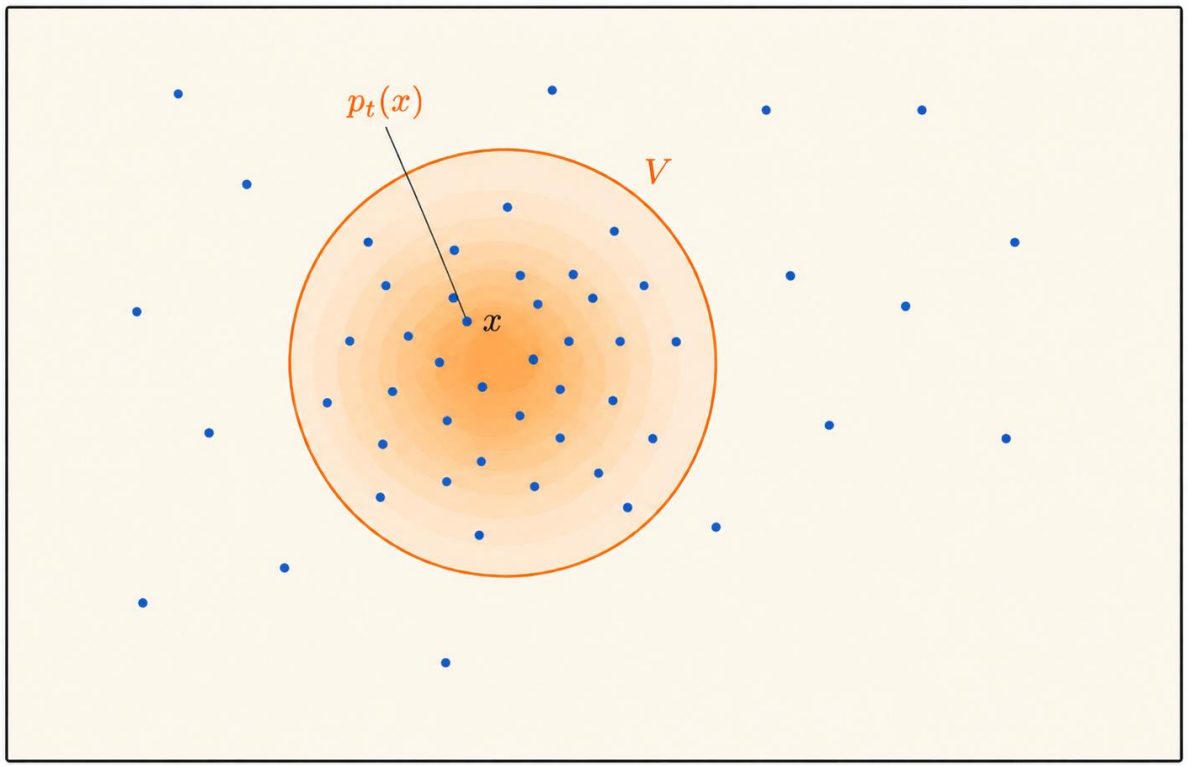
$$\nabla_x \cdot f(x,t) = \sum_{i=1}^{d} \frac{\partial f_i(x,t)}{\partial x_i}$$

- $\mathrm{p_t(x)}$为点x处的概率密度

4.3.2 流映射与连续性方程

- 连续性方程的推导
  - 考虑状态空间中的固定区域V，其中含有的概率质量为：

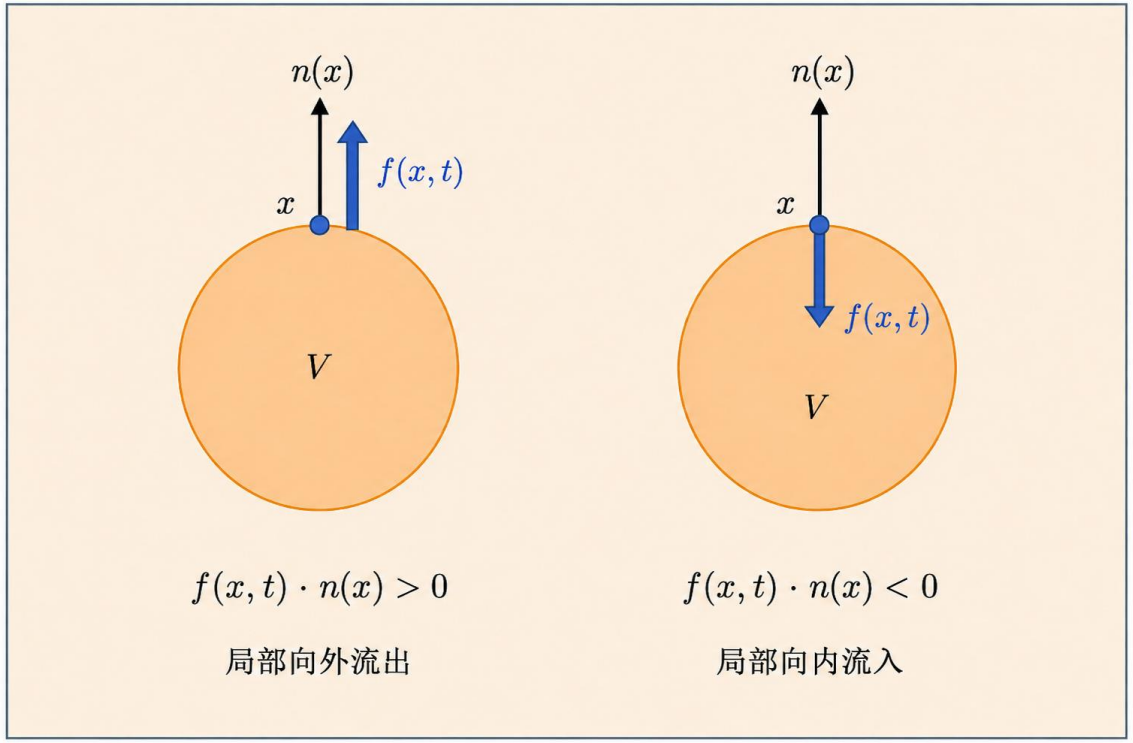
$$P_t(V) = \int_V p_t(x)\,dx$$



4.3.2 流映射与连续性方程

- 连续性方程的推导
  - 由于概率质量只进行空间输运，而不会凭空产生或消失，对状态空间内的固定区域V，概率质量守恒给出：

$$\frac{d}{dt}\int_V p_t(x)\,dx = -\int_{\partial V} p_t(x) f(x,t)\cdot n(x)\,dS$$



- f(x, t): 点x处的速度向量
- p_t(x)f(x, t): 点x处的概率通量
- n(x): 边界的单位外法向量

4.3.2 流映射与连续性方程

- 连续性方程的推导

  - 由于概率质量只进行空间输运，而不会凭空产生或消失，对状态空间内的固定区域V，概率质量守恒给出：

$$\frac{d}{dt}\int_V p_t(x)\,dx = -\int_{\partial V} p_t(x) f(x,t)\cdot n(x)\,dS \tag{1}$$

  - 由散度定理，边界上通量的积分可以转换成区域内散度的积分：

$$\int_{\partial V} p_t(x) f(x,t)\cdot n(x)\,dS = \int_V \nabla_x\cdot\left(p_t(x) f(x,t)\right)\,dx \tag{2}$$

  - 代入式（1）可得：

$$\frac{d}{dt}\int_V p_t(x)\,dx = -\int_V \nabla_x\cdot\left(p_t(x) f(x,t)\right)\,dx \tag{3}$$

4.3.2 流映射与连续性方程

- 连续性方程的推导
  - 固定区域V上，积分和求导可以交换（假设 p_t连续可微），因此式（3）变为：

$$\int_V \frac{\partial p_t(x)}{\partial t}\, dx = -\int_V \nabla_x \cdot (p_t(x) f(x,t))\, dx \tag{4}$$

  - 整理得：

$$\int_V \left[\frac{\partial p_t(x)}{\partial t} + \nabla_x \cdot (p_t(x) f(x,t))\right] dx = 0 \tag{5}$$

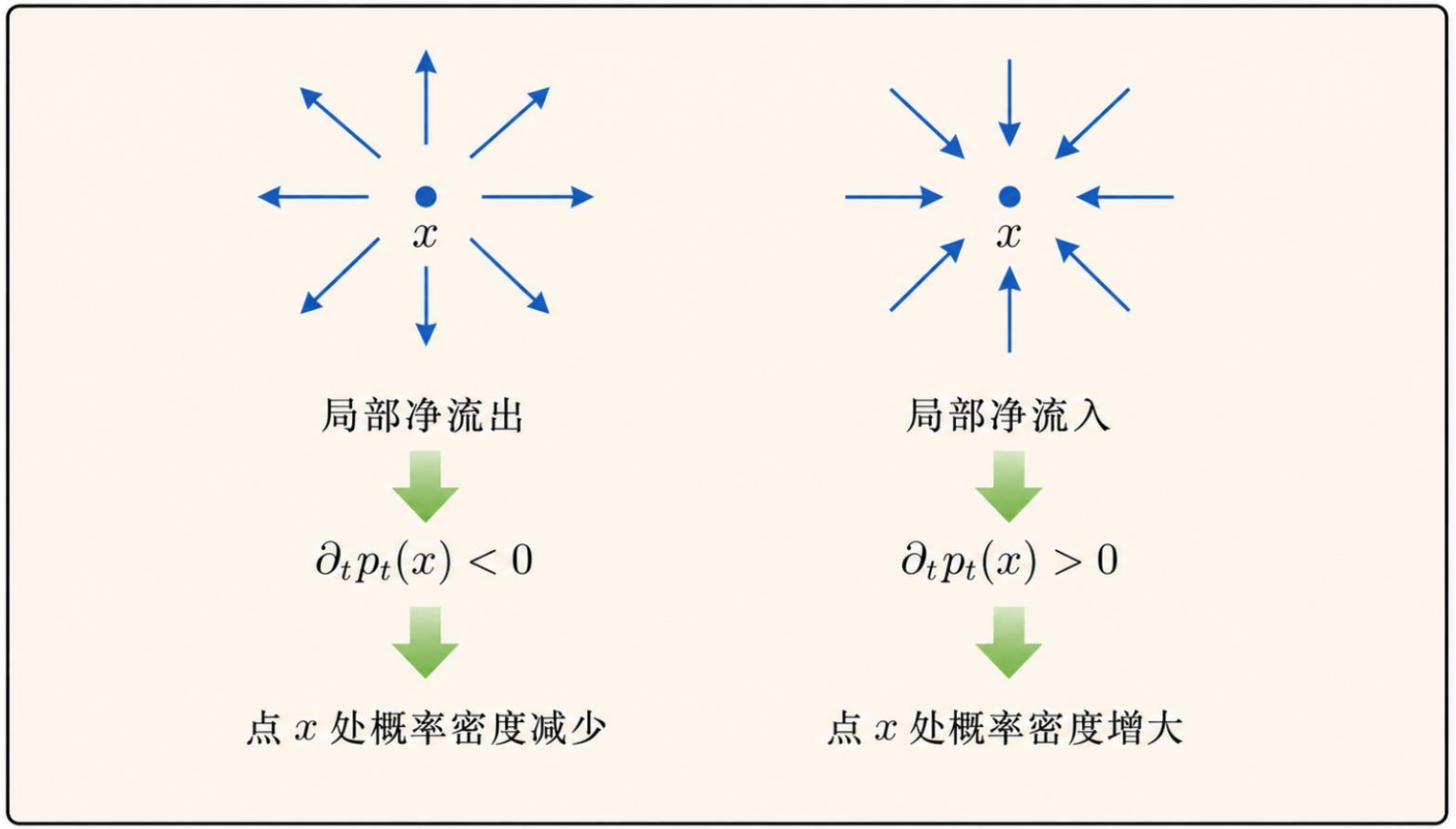
  - 因为上式对任意区域V都成立，所以必须有：

$$\frac{\partial p_t(x)}{\partial t} + \nabla_x \cdot (p_t(x) f(x,t)) = 0 \tag{6}$$

4.3.2 流映射与连续性方程

- 连续性方程的推导

$$\frac{\partial p_t(x)}{\partial t} + \nabla_x \cdot (p_t(x) f(x,t)) = 0$$



局部的流出/流入
导致了该点概率密度的变化

4.3.3 连续归一化流

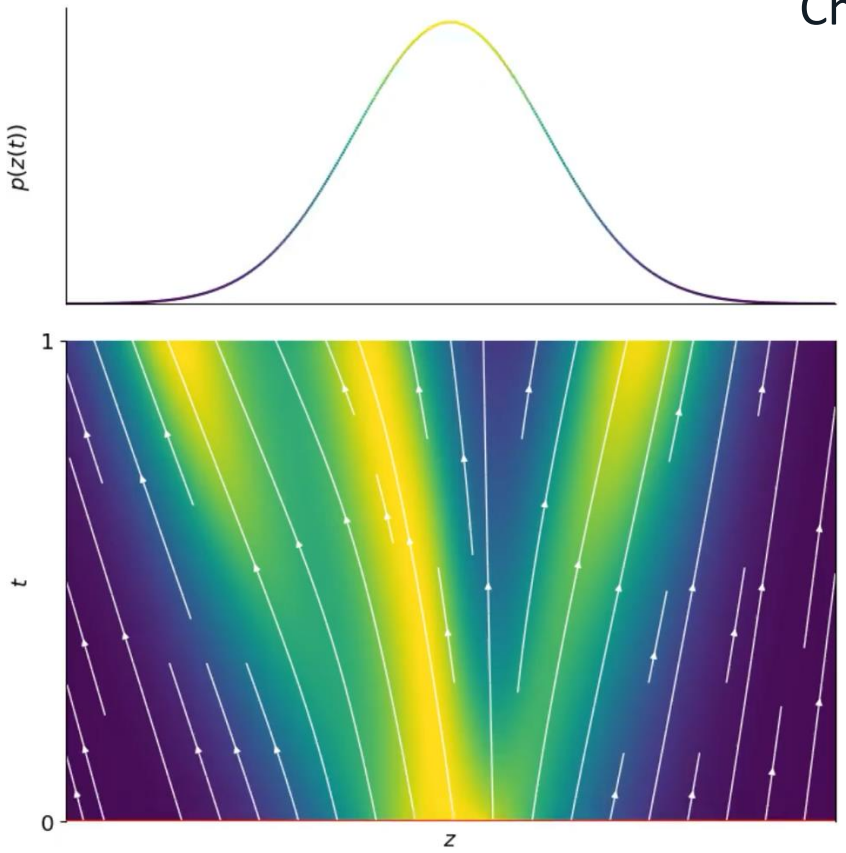
- Neural ODE：用神经网络参数化速度场

$$\frac{\mathrm{d}\mathbf{x}_t}{\mathrm{d}t} = u_t\ \mathbf{x}_t, \theta$$

Changes in position　Vector field

$t = 1$

$p_{\text{base}}(\mathbf{z})$



$t = 0$

$p_\theta(\mathbf{x})$

[1] 引用网址及参考文献等

4.3.3 连续归一化流

- 同一速度场同时决定样本轨迹与概率密度

$$\frac{\mathrm{d}\mathbf{x}_t}{\mathrm{d}t} = u_t\ \mathbf{x}_t, \theta$$

Changes in position　Vector field

$$\frac{\mathrm{d}p_t(\mathbf{x}_t)}{\mathrm{d}t} = ?$$

Changes in density

- 与普通 Neural ODE 只关注状态轨迹不同，CNF 需要追踪轨迹对应的概率密度变化

$$\frac{\mathrm{d}p_t(\mathbf{x}_\mathrm{t})}{\mathrm{d}t} + \frac{\mathrm{d}}{\mathrm{dx}}\left(p_t(\mathbf{x}_\mathrm{t})u_t(\mathbf{x}_\mathrm{t})\right) = 0$$

Continuity Equation

$$\frac{\mathrm{d}p_t(\mathbf{x}_\mathrm{t})}{\mathrm{d}t} = -\nabla\cdot\left(p_t(\mathbf{x}_\mathrm{t})u_t(\mathbf{x}_\mathrm{t})\right)$$

[1] 引用网址及参考文献等

4.3.3 连续归一化流

- 训练方式-最大似然

$$\log p_\theta(\mathbf{x}) = \log p_{\text{base}}(\mathbf{z}) + \int_{t=0}^{1} -\text{div}(u_t\ \mathbf{x}_t\ \theta\ )\ \mathrm{d}t$$

似然计算的过程中： 需要不断的计算散度项

$$\mathbf{z} = \mathbf{x} - \int_{t=0}^{1} u_t\ \mathbf{x}_t, \theta\ \mathrm{d}t$$

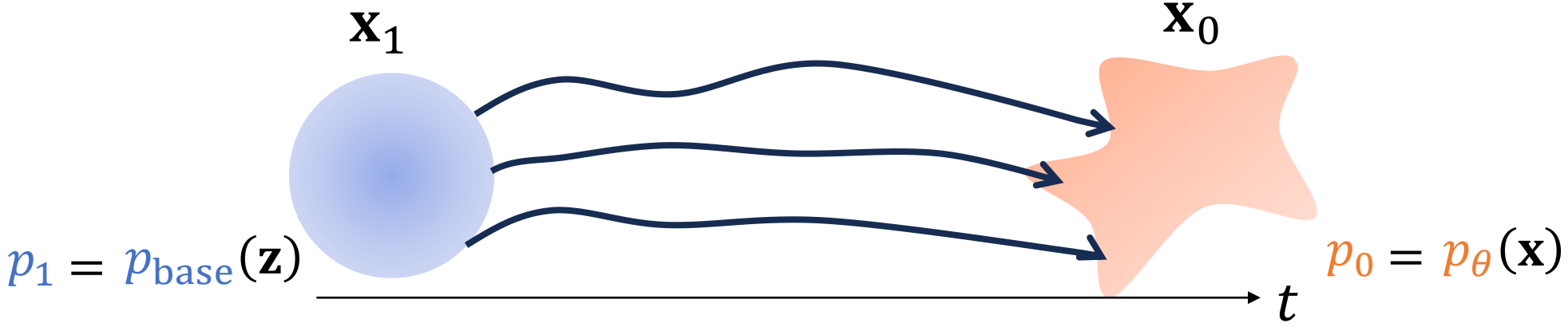
似然计算的过程中： 同时需要反复计算积分，直到从干净图像积分到噪声。才能完整计算似然

整体训练效率很低，需要持续积分与散度计算

[1] 引用网址及参考文献等

4.3.4 Flow Matching

- 连续归一化流



We don't know

$p_t(\mathbf{x}_t)$：**概率路径**

$u_t(\mathbf{x}_t)$：**速度场**

In practice

$$\log p_\theta(\mathbf{x}) = \log p_{\text{base}}(\mathbf{z}) + \int_{t=0}^{1} -\text{div}(u_t(\mathbf{x}_t, \theta))\, dt$$

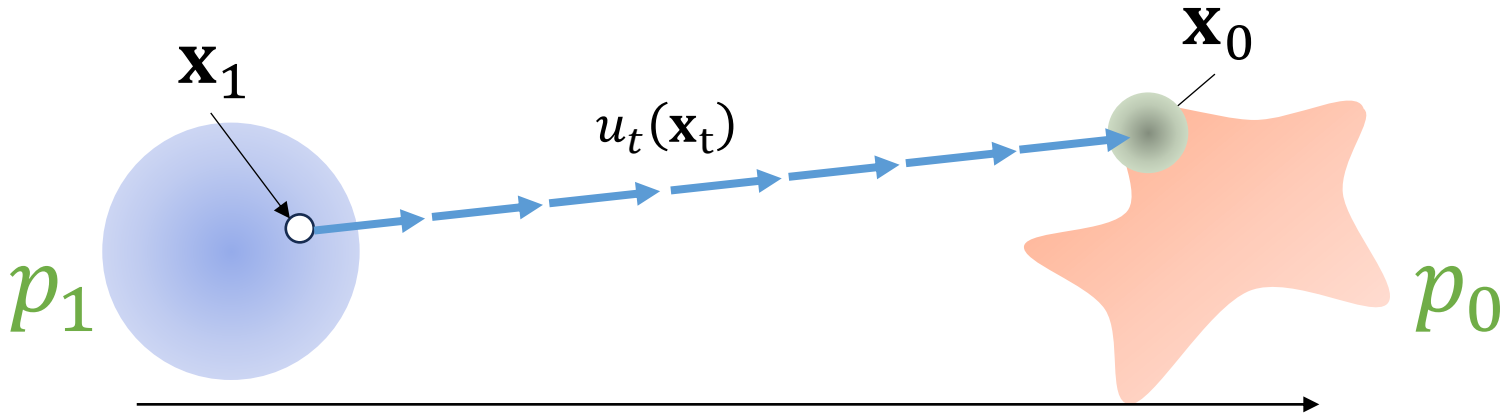
训练过程中需要不断的计算散度项，并且积分

能否不计算似然，而是直接学习速度场？

[1] 引用网址及参考文献等

4.3.4 Flow Matching

- Flow matching : 直接回归生成概率路径的速度场



Match the "flow" $u_t(\mathbf{x}_t)$!

$$\mathrm{L_{FM}} = \mathrm{E}_{t,\, p_t(\mathbf{x}_t)}[||v_t(\mathbf{x}_t, \theta) - u_t(\mathbf{x}_t)||_2^2]$$

Probability path　Neural network　Vector field

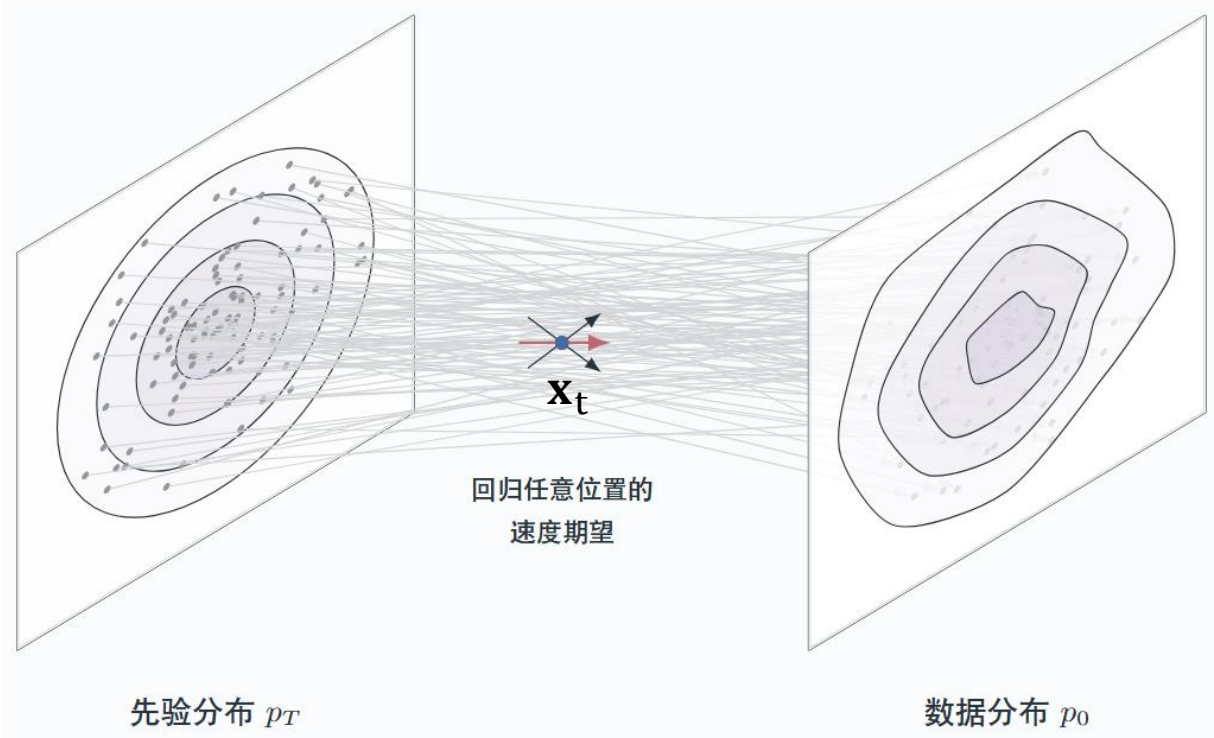
但是，目标概率路径 $p_t(\mathbf{x}_t)$ 和目标速度场 $u_t(\mathbf{x}_t)$ 从哪里来?

4.3.4 Flow Matching

- 难点：边缘速度场无法直接计算

$$\mathrm{L_{FM}} = \mathrm{E}_{t,\, p_t(\mathbf{x}_t)}[||v_t(\mathbf{x}_\mathrm{t}, \theta) - u_t(\mathbf{x}_\mathrm{t})||_2^2]$$

$u_t(\mathbf{x}_\mathrm{t})$: 需要综合整个数据分布中所有可能经过 $\mathbf{x}_\mathrm{t}$的轨迹，无法直接得到



[1] 引用网址及参考文献等

4.3.4 Flow Matching

- Conditional Flow Matching

$$\mathrm{L_{FM}} = \mathrm{E}_{t,\, p_t(\mathbf{x}_t)}[||v_t(\mathbf{x}_\mathrm{t}, \theta) - u_t(\mathbf{x}_\mathrm{t})||_2^2]$$

Probability path $p_t(\mathbf{x}_t) = \int p_t(\mathbf{x}_t|\mathbf{z})\ q(\mathbf{z})\, d\mathbf{z}$

Conditional probability path

Conditional Flow matching (CFM) objective

$$\mathrm{L_{CFM}} = \mathrm{E}_{t,\, q(\mathbf{z}),\, p_t(\mathbf{x}_t|\mathbf{z})}[||v_t(\mathbf{x}_\mathrm{t}, \theta) - u_t(\mathbf{x}_\mathrm{t}|\mathbf{z})||_2^2]$$

$$\nabla_\theta \mathrm{L_{FM}}(\theta) = \nabla_\theta \mathrm{L_{CFM}}(\theta)$$

[1] 引用网址及参考文献等

4.3.4 Flow Matching

- Conditional Flow Matching

$$\mathrm{L_{CFM}} = \mathrm{E}_{t,\, q(\mathbf{z}),\, p_t(\mathbf{x}_t|\mathbf{z})}\left[\left|\left|v_t(\mathbf{x_t}, \theta) - u_t(\mathbf{x_t}|\mathbf{z})\right|\right|_2^2\right]$$

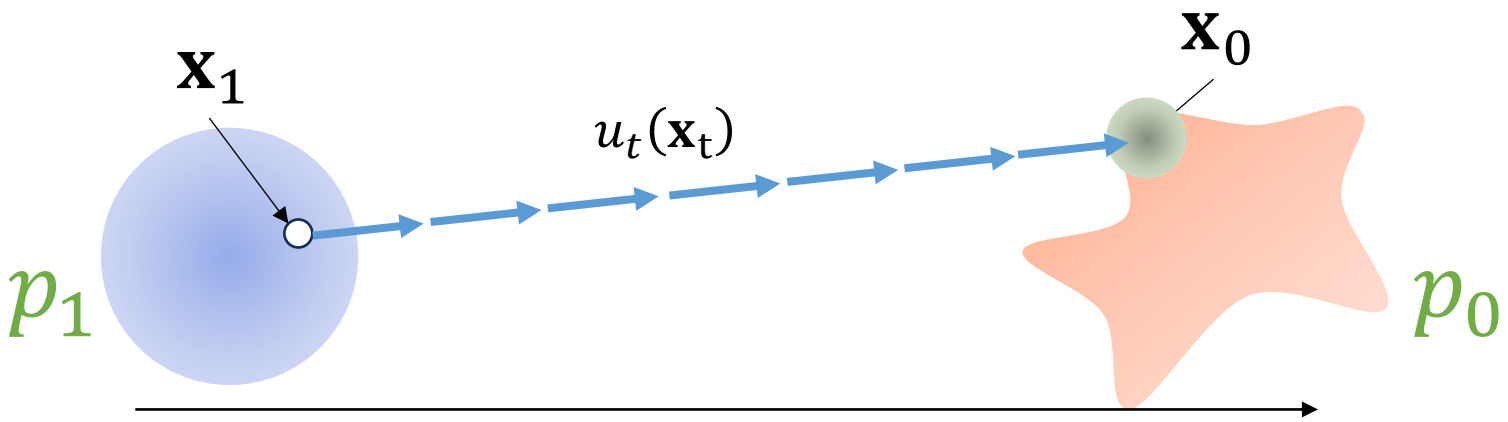
Example : Let Conditional distribution $q(\mathbf{z}) = q(\mathbf{x_0})q(\mathbf{x_1})$

Conditional probability path $p_t(\mathbf{x}|\mathbf{z}) = N(\mathbf{x}|t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \sigma^2)$

Conditional vector field $u_t(\mathbf{x}|\mathbf{z}) = \mathbf{x_1} - \mathbf{x_0}$

$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0 + \sigma\epsilon$$

$$u_t(\mathbf{x}_t|\mathbf{z}) = \frac{d\mathbf{x}_t}{dt} = \mathbf{x_1} - \mathbf{x_0}$$



最常用的路径构造方式：直线路径，进行线性插值

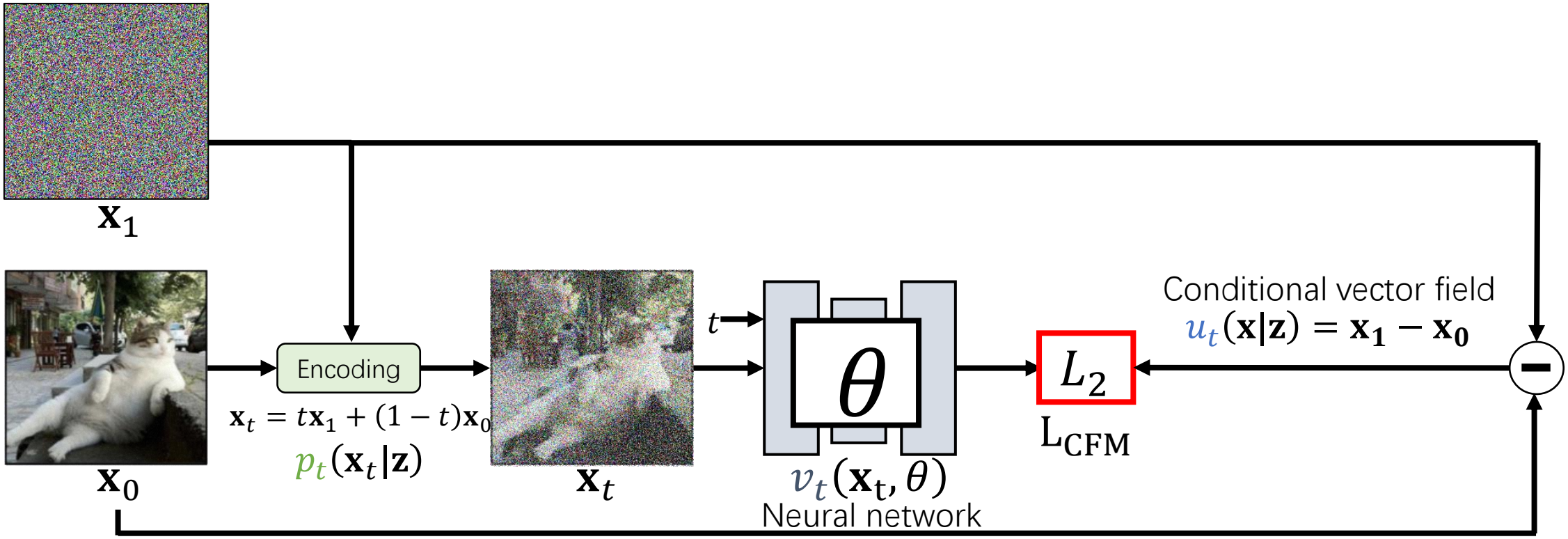
[1] 引用网址及参考文献等

4.3.4 Flow Matching

- 训练方式

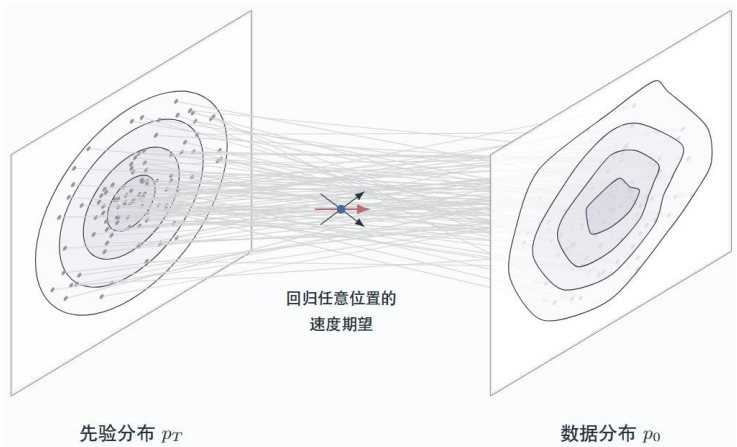
$$\mathrm{L_{CFM}} = \mathrm{E}_{t,\, q(\mathbf{z}),\, p_t(\mathbf{x}_t|\mathbf{z})}[||v_t(\mathbf{x_t}, \theta) - u_t(\mathbf{x_t}|\mathbf{z})||_2^2]$$

Conditional distribution $q(\mathbf{z}) = q(\mathbf{x_0})q(\mathbf{x_1})$



4.3.4 Flow Matching

- 采样方式：离散ODE积分



Flow matching 学习到的将是期望速度场
所以实际的采样路径并非是笔直的



4.3.5 概率路径设计

- 寻找更直的轨迹

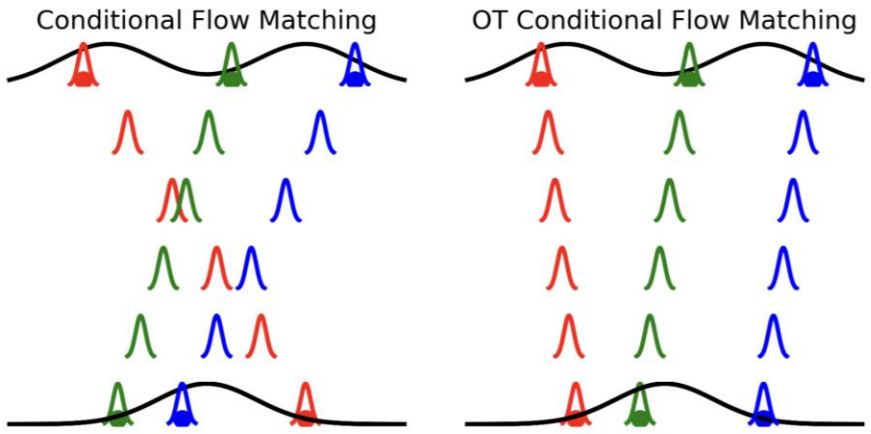
Example : Optimal transport CFM

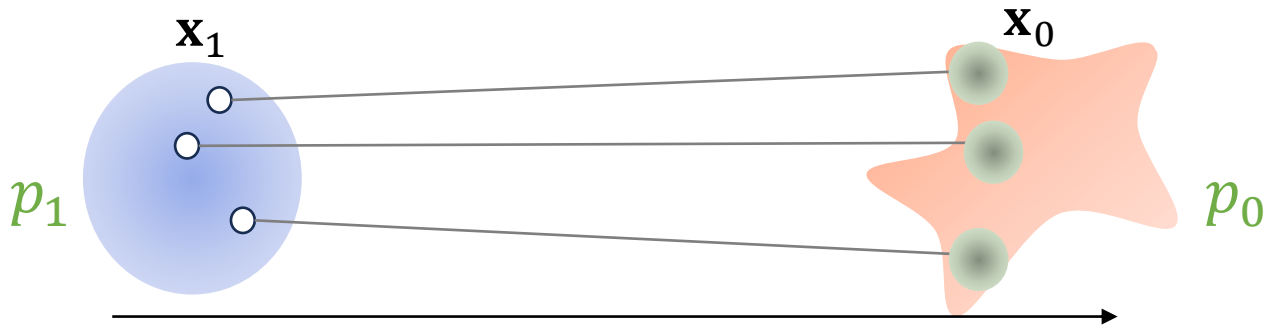
采样N对样本，先最优匹配，产生最优路径集合

Conditional distribution $q(\mathbf{z}) = \pi(\mathbf{x}_0, \mathbf{x}_1)$

Conditional probability path $p_t(\mathbf{x}|\mathbf{z}) = N(\mathbf{x}|t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \sigma^2)$

Conditional vector field $u_t(\mathbf{x}|\mathbf{z}) = \mathbf{x_1} - \mathbf{x_0}$

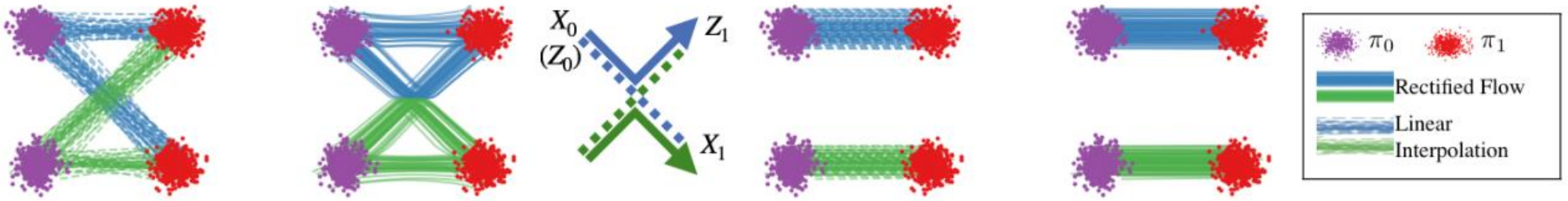




4.3.5 概率路径设计

- Rectified Flow

利用已训练模型重新建立噪声与数据之间的端点配对，再基于新配对训练更直的流。



(a) Linear interpolation $X_t = tX_1 + (1-t)X_0$

(b) Rectified flow Z_t induced by (X_0, X_1)

(c) Linear interpolation $Z_t = tZ_1 + (1-t)Z_0$

(d) Rectified flow Z'_t induced by (Z_0, Z_1)

保持端点边缘分布不变，同时缩短运输路径，支持更少步数的 ODE 采样。

$$\mathbb{E}[\|Z_0 - Z_1\|] = \text{Length}\left(\quad\right) \overset{(*)}{\leq} \text{Length}\left(\quad\right) \overset{(**)}{=} \text{Length}\left(\quad\right) = \mathbb{E}[\|X_0 - X_1\|]\,,$$

4.3.5 概率路径设计

- Other works
  - PeRFlow 将生成轨迹划分为几个离散的时间窗口，单独拉直每个区间内的片段，这种局部化降低了模拟训练轨迹的计算成本，并最小化了数值误差
  - Rectified-CFG++ 采用了一种自适应预测器-校正器方案，每个生成步骤首先执行条件更新，将样本锚定在学到的传输路径附近，然后基于条件速度与无条件速度之间的差异应用流形感知校正

深度学习中的连续动力学机制

4.1 连续动力学的数学基础

4.2 表示空间中的连续动力学

4.3 确定性概率动力学

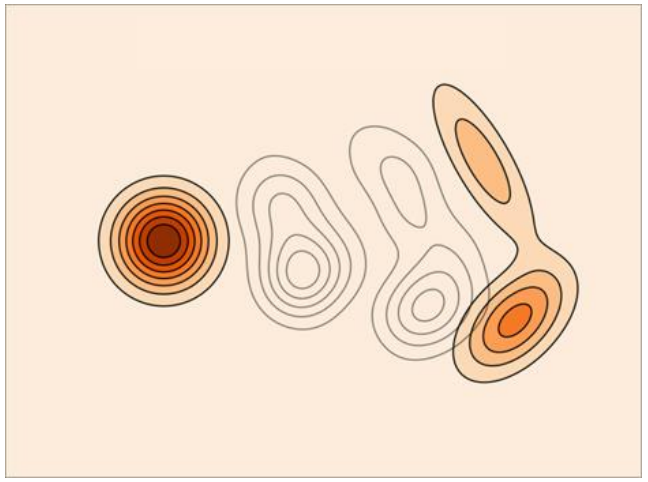
4.4 随机概率动力学

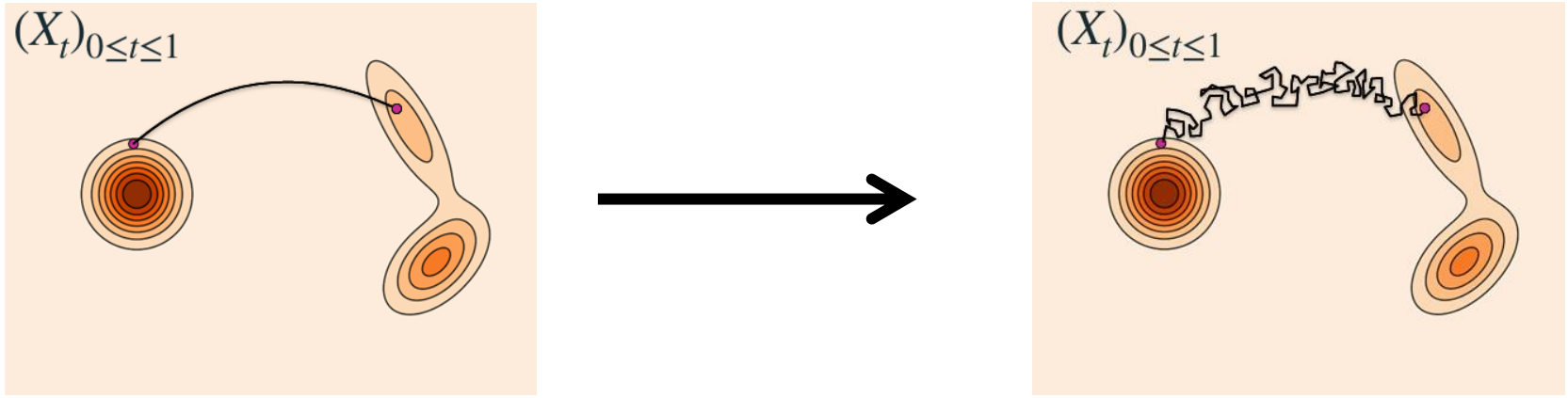
4.5 离散状态上的连续时间动力学

4.4.1 随机微分方程

Flow matching

- 运动轨迹的描述：ODE
- 确定性轨迹 $\frac{dX_t}{dt} = v_t(X_t)$
- 用点的位置变化来刻画分布的变化





但是，真实转移的过程实际不是确定的，而是一个概率分布（转移核），所以真实的路径往往是一种随机运动，而不是确定性的ODE

4.4.1 随机微分方程

- **ODE到SDE**
  - **ODE描述变量x，在具备初值条件下确定性演化**

$$\frac{\mathrm{d}\mathbf{x}(t)}{\mathrm{d}t} = \mathbf{f}(\mathbf{x}(t), t), \quad \mathbf{x}(0) = \mathbf{x}_0. \xrightarrow{\text{离散时间下}} \mathbf{x}_{t+\Delta t} = \mathbf{x}_t + \mathbf{f}(\mathbf{x}_t, t)\Delta t.$$

  - **从 ODE to SDE**

加入微小噪声 ↓

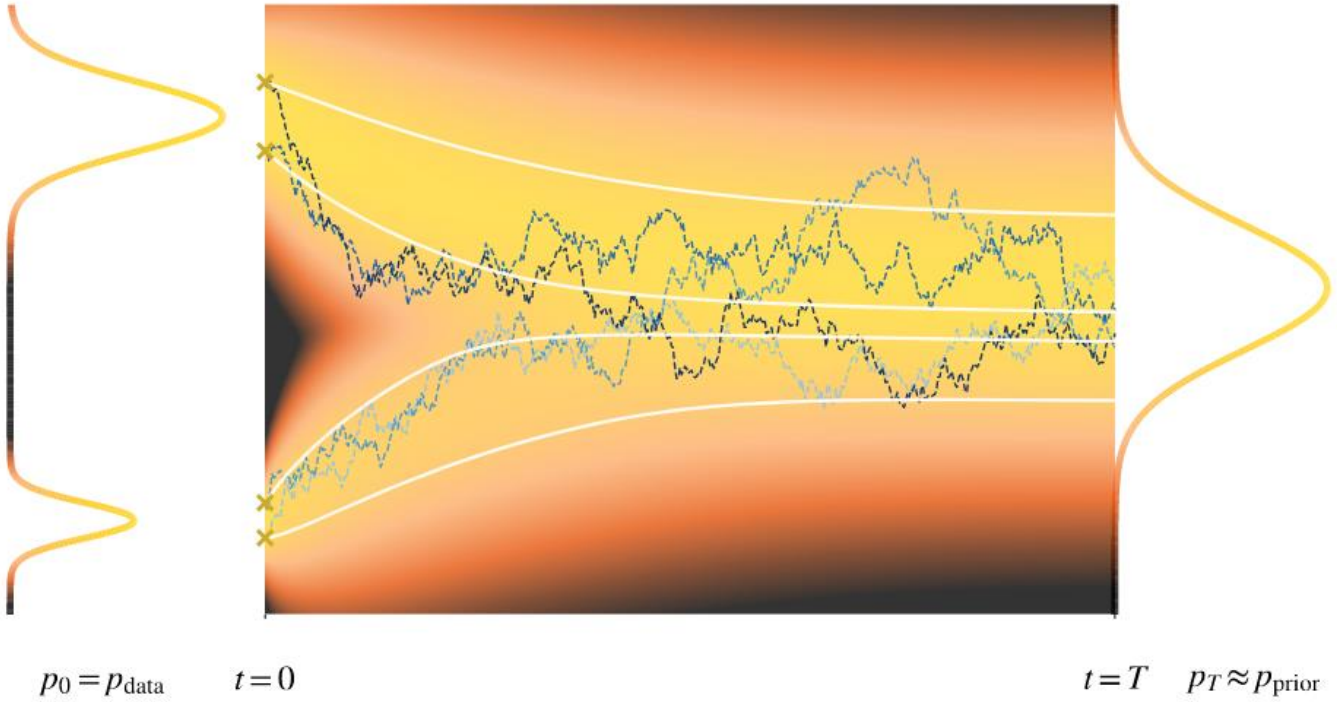
$$\mathrm{d}\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t)\,\mathrm{d}t + g(t)\,\mathrm{d}\mathbf{w}(t). \xleftarrow{\text{连续时间下}} \mathbf{x}_{t+\Delta t} = \mathbf{x}_t + \mathbf{f}(\mathbf{x}_t, t)\Delta t + g(t)\sqrt{\Delta t}\cdot\boldsymbol{\epsilon}_t,$$

  - **所以 SDE与ode不同在于每一步加入微小的噪声扰动**

4.4.2 正向扩散过程

- SDE具体的加噪轨迹
  - 一旦指定了漂移系数f 和扩散系数 g，就完全确定了正向过程

$$\mathrm{d}\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t)\,\mathrm{d}t + g(t)\,\mathrm{d}\mathbf{w}(t).$$



4.4.2 正向扩散过程

- **两种常用的加噪轨迹**

DDPM



$X_{i+1} = \sqrt{1-\beta_i}\, X_i + \sqrt{\beta_i}\varepsilon$　加噪过程是通过，逐步减少原图信号，增加噪声信号

NCSN



$X_{i+1} = X_i + \sqrt{\sigma_{i+1}^2 - \sigma_i^2}\varepsilon$　加噪过程是通过，只逐步增加噪声信号

4.4.2 正向扩散过程

- 从离散时间的角度来看：从 X_t 到 $X_{t+\Delta t}$ ，用以下形式表示

$$\text{NCSN:} \quad \mathbf{x}_{t+\Delta t} = \mathbf{x}_t + \sqrt{\sigma_{t+\Delta t}^2 - \sigma_t^2}\boldsymbol{\epsilon}_t \quad \approx \mathbf{x}_t + \sqrt{\frac{\mathrm{d}\sigma_t^2}{\mathrm{d}t}\Delta t}\boldsymbol{\epsilon}_t$$

$$\text{DDPM:} \quad \mathbf{x}_{t+\Delta t} = \sqrt{1-\beta_t}\mathbf{x}_t + \sqrt{\beta_t}\boldsymbol{\epsilon}_t \quad \approx \mathbf{x}_t - \frac{1}{2}\beta_t\mathbf{x}_t\Delta t + \sqrt{\beta_t \Delta t}\boldsymbol{\epsilon}_t,$$

4.4.2 正向扩散过程

- 从离散时间的角度来看：从 X_t 到 $X_{t+\Delta t}$ ，用以下形式表示

$$\text{NCSN:} \quad \mathbf{x}_{t+\Delta t} = \mathbf{x}_t + \sqrt{\sigma_{t+\Delta t}^2 - \sigma_t^2}\boldsymbol{\epsilon}_t \qquad \approx \mathbf{x}_t + \sqrt{\frac{\mathrm{d}\sigma_t^2}{\mathrm{d}t}\Delta t}\boldsymbol{\epsilon}_t$$

$$\text{DDPM:} \quad \mathbf{x}_{t+\Delta t} = \sqrt{1-\beta_t}\mathbf{x}_t + \sqrt{\beta_t}\boldsymbol{\epsilon}_t \qquad \approx \mathbf{x}_t - \frac{1}{2}\beta_t\mathbf{x}_t\Delta t + \sqrt{\beta_t \Delta t}\boldsymbol{\epsilon}_t,$$

- 其中 $\boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ，这两种噪声注入过程都遵循**共同的结构**

$$\mathbf{x}_{t+\Delta t} \approx \mathbf{x}_t + \mathbf{f}(\mathbf{x}_t, t)\Delta t + g(t)\sqrt{\Delta t}\boldsymbol{\epsilon}_t,$$

$$\begin{cases} \text{NCSN:} \quad \mathbf{f}(\mathbf{x}, t) = 0, \qquad g(t) = \sqrt{\dfrac{\mathrm{d}\sigma^2(t)}{\mathrm{d}t}} \\ \text{DDPM:} \quad \mathbf{f}(\mathbf{x}, t) = -\dfrac{1}{2}\beta(t)\mathbf{x}, \qquad g(t) = \sqrt{\beta(t)}. \end{cases}$$

4.4.2 正向扩散过程

- **NCSN与DDPM连续化后对应两种SDE形式**
  - **NCSN：** Variance Exploding SDE（VE SDE）
  - **DDPM：** Variance Preserving SDE （VP SDE）

	VE SDE	VP SDE
$\mathbf{f}(\mathbf{x}, t)$	$\mathbf{0}$	$-\frac{1}{2}\beta(t)\mathbf{x}$
$g(t)$	$\sqrt{\frac{\mathrm{d}\sigma^2(t)}{\mathrm{d}t}}$	$\sqrt{\beta(t)}$
SDE	$\mathrm{d}\mathbf{x}(t) = g(t)\,\mathrm{d}\mathbf{w}(t)$	$\mathrm{d}\mathbf{x}(t) = -\frac{1}{2}\beta(t)\mathbf{x}(t)\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\mathbf{w}(t)$
$p_t(\mathbf{x}_t \vert \mathbf{x}_0)$	$\mathcal{N}\left(\mathbf{x}_t; \mathbf{x}_0, \left(\sigma^2(t) - \sigma^2(0)\right)\mathbf{I}\right)$	$\mathcal{N}\left(\mathbf{x}_t; \mathbf{x}_0 e^{-\frac{1}{2}\int_0^t \beta(\tau)\,\mathrm{d}\tau}, \mathbf{I} - \mathbf{I}e^{-\int_0^t \beta(\tau)\,\mathrm{d}\tau}\right)$
p_{prior}	$\mathcal{N}(\mathbf{0}, \sigma^2(T)\mathbf{I})$	$\mathcal{N}(\mathbf{0}, \mathbf{I})$

4.4.3 反向SDE与分数函数

- 如何采样
  - 在数学中，一个正向SDE可以写出它的**反向SDE形式**，逆转分布的变化

正向

$$\mathrm{d}\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t)\,\mathrm{d}t + g(t)\,\mathrm{d}\mathbf{w}(t).$$

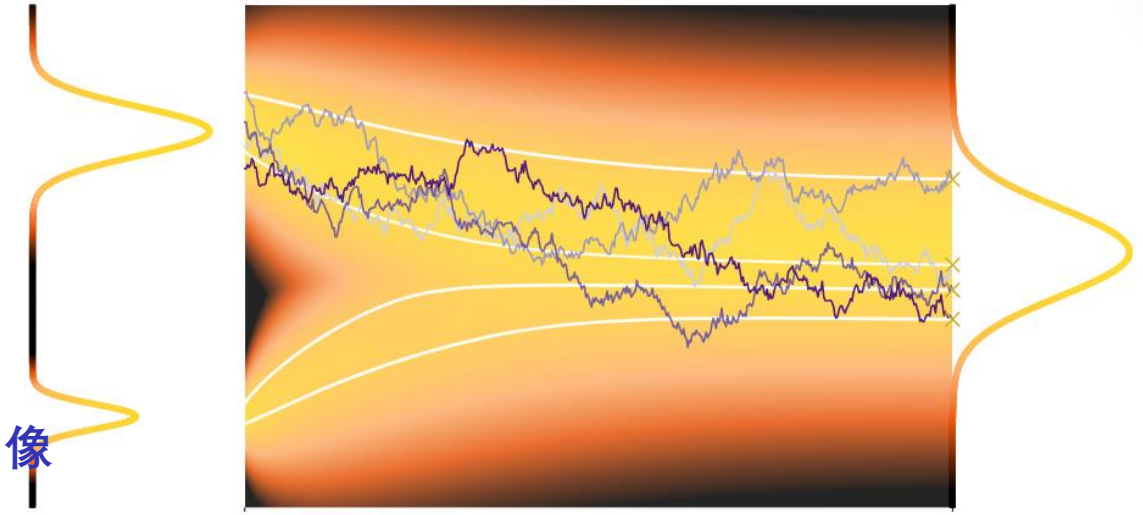
反向

$$\mathrm{d}\bar{\mathbf{x}}(t) = \left[\mathbf{f}(\bar{\mathbf{x}}(t), t) - g^2(t)\nabla_{\mathbf{x}} \log p_t(\bar{\mathbf{x}}(t))\right]\mathrm{d}t + g(t)\,\mathrm{d}\bar{\mathbf{w}}(t),$$

  - 其中未知的量为 $\nabla_{\mathbf{x}} \log p_t(\bar{\mathbf{x}}(t))$

这部分我们称之为分数/梯度

如果有了这个分数，我们就可以通过反向SDE去采样清晰的图像



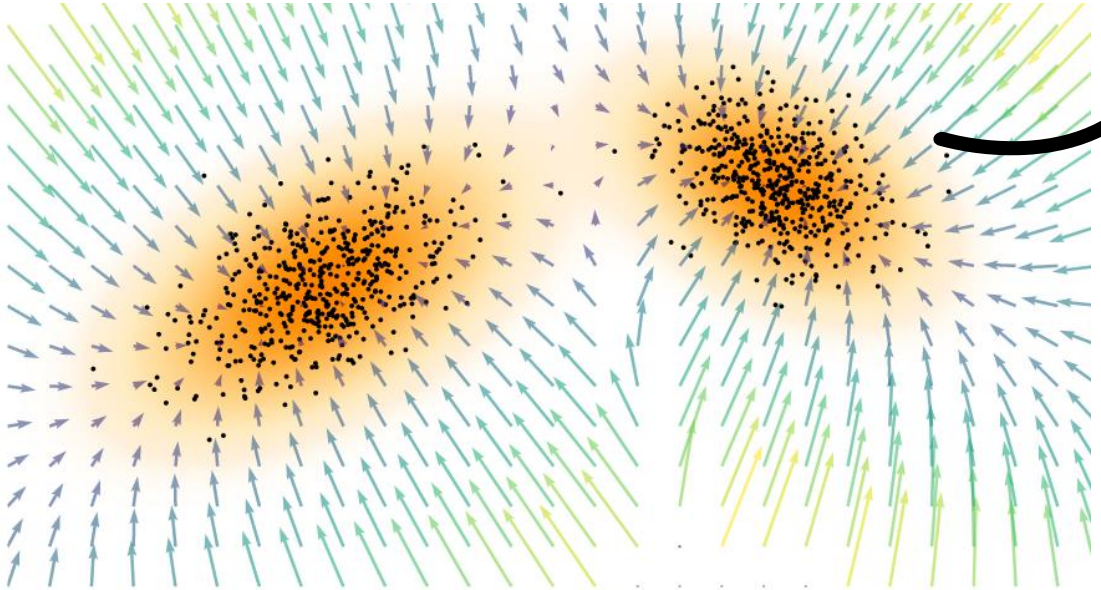
$$\mathbf{x}_{t-\Delta t} \leftarrow \mathbf{x}_t - \left[\mathbf{f}(\mathbf{x}_t, t) - g^2(t)\mathbf{s}_{\phi\times}(\mathbf{x}_t, t)\right]\Delta t + g(t)\sqrt{\Delta t}\cdot\boldsymbol{\epsilon}$$

4.4.3 反向SDE与分数函数

- **Score其实就是梯度**
  - 分数是空间R^D上**对数概率密度p（x）**的梯度:

$$\mathbf{s}(\mathbf{x}) := \nabla_{\mathbf{x}} \log p(\mathbf{x}), \qquad \mathbf{s}: \mathbb{R}^D \to \mathbb{R}^D.$$

  - 直观地讲，Score指向概率较高的区域，为数据最有可能出现的位置提供局部指导，它是数据的对数概率密度增长最快的方向

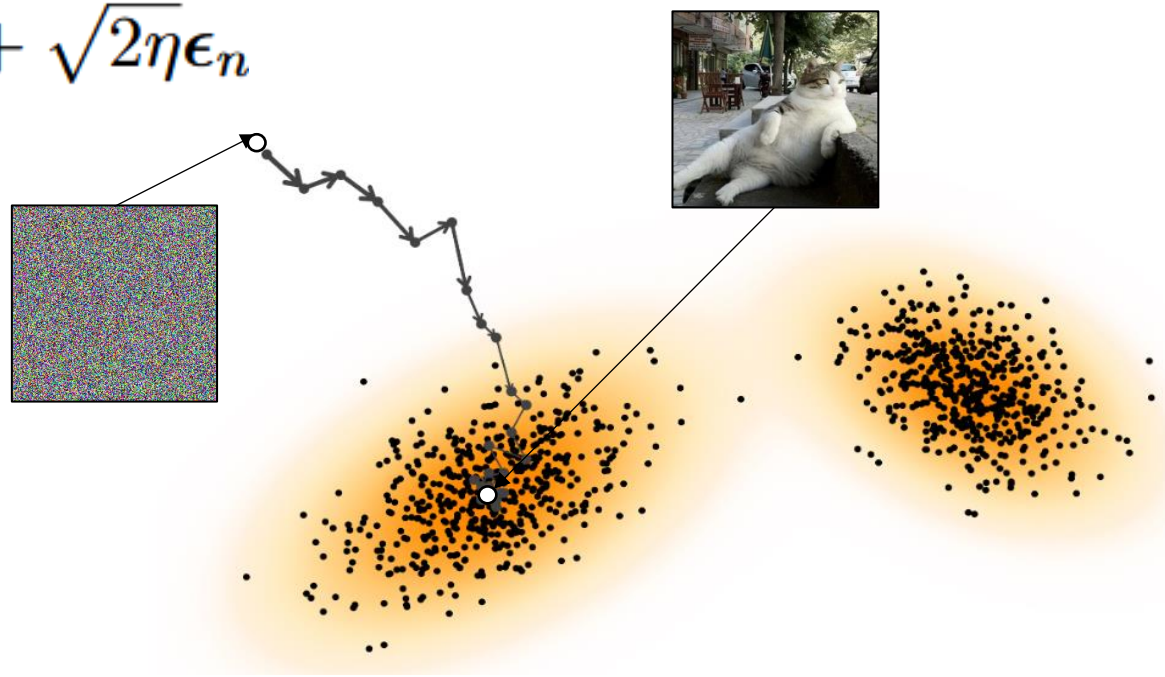


4.4.3 反向SDE与分数函数

- **当我们有了Score，可以怎么走?**
  - 朗之万动力学
    - 让一个粒子在能量场中被梯度吸引到高密度区
    - 但同时加一些噪声，使粒子能在整个分布中随机游走
  - 采样公式

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \eta \nabla_{\mathbf{x}} \log p_\phi(\mathbf{x}_n) + \sqrt{2\eta}\epsilon_n$$

    - η表示步长
    - $\epsilon_n \in \mathcal{N}(\mathbf{0}, \mathbf{I})$



4.4.4 去噪分数匹配

- 如何去得到Score
  - 使用神经网络来近似梯度，但是真实梯度未知

$$\mathcal{L}_{\mathrm{SM}}(\boldsymbol{\phi}) := \frac{1}{2}\mathbb{E}_{\mathbf{x}\sim p_{\mathrm{data}}}\left[\|\mathbf{s}_{\boldsymbol{\phi}}(\mathbf{x}) - \mathbf{s}(\mathbf{x})\|_2^2\right].$$

未知

  - 第一种方法（数学推导消去真实梯度值）

We can express the following equation as:

$$\mathcal{L}_{\mathrm{SM}}(\boldsymbol{\phi}) = \widetilde{\mathcal{L}}_{\mathrm{SM}}(\boldsymbol{\phi}) + C.$$

where

$$\widetilde{\mathcal{L}}_{\mathrm{SM}}(\boldsymbol{\phi}) := \mathbb{E}_{\mathbf{x}\sim p_{\mathrm{data}}(\mathbf{x})}\left[\mathrm{Tr}\left(\nabla_{\mathbf{x}}\mathbf{s}_{\boldsymbol{\phi}}(\mathbf{x})\right) + \frac{1}{2}\|\mathbf{s}_{\boldsymbol{\phi}}(\mathbf{x})\|_2^2\right]$$

计算复杂度 $O(D^2)$，无法扩展至高维数据

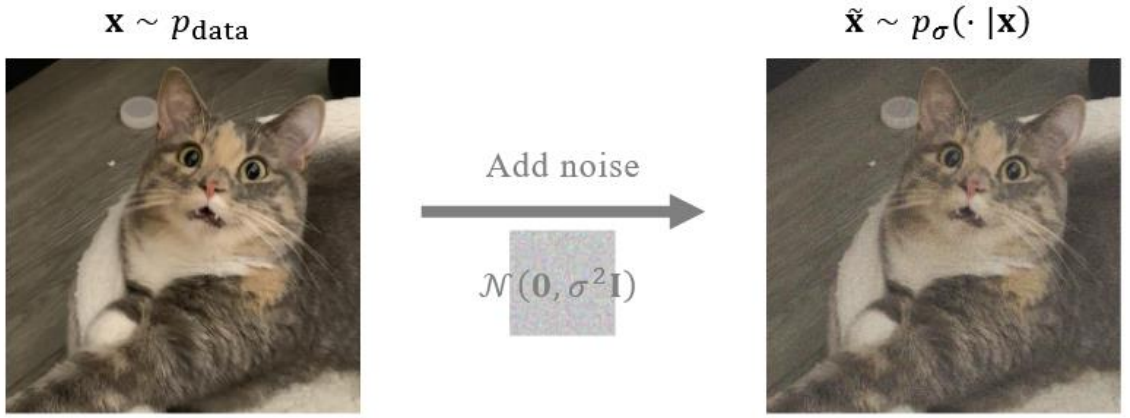
D=256×256×3

4.4.4 去噪分数匹配

- 如何去得到Score
  - 第二种方法（去噪分数匹配）加入一个极小方差的噪声，可以近似得到梯度

$$\mathcal{L}_{\mathrm{SM}}(\phi) = \frac{1}{2}\mathbb{E}_{\mathbf{x}\sim p_{\mathrm{data}}(\mathbf{x})}\left[\|\mathbf{s}_\phi(\mathbf{x}) - \nabla_{\mathbf{x}} \log p_{\mathrm{data}}(\mathbf{x})\|_2^2\right]$$

$$\tilde{\mathbf{x}} = \mathbf{x} + \sigma\epsilon, \longrightarrow$$



$$p_\sigma(\tilde{\mathbf{x}}|\mathbf{x}) = \mathcal{N}(\tilde{\mathbf{x}}; \mathbf{x}, \sigma^2\mathbf{I}).$$

$$\nabla_{\tilde{\mathbf{x}}} \log p_\sigma(\tilde{\mathbf{x}}|\mathbf{x}) = \frac{\mathbf{x} - \tilde{\mathbf{x}}}{\sigma^2}.$$

此时，带有微小噪声的 x 可以计算出分数

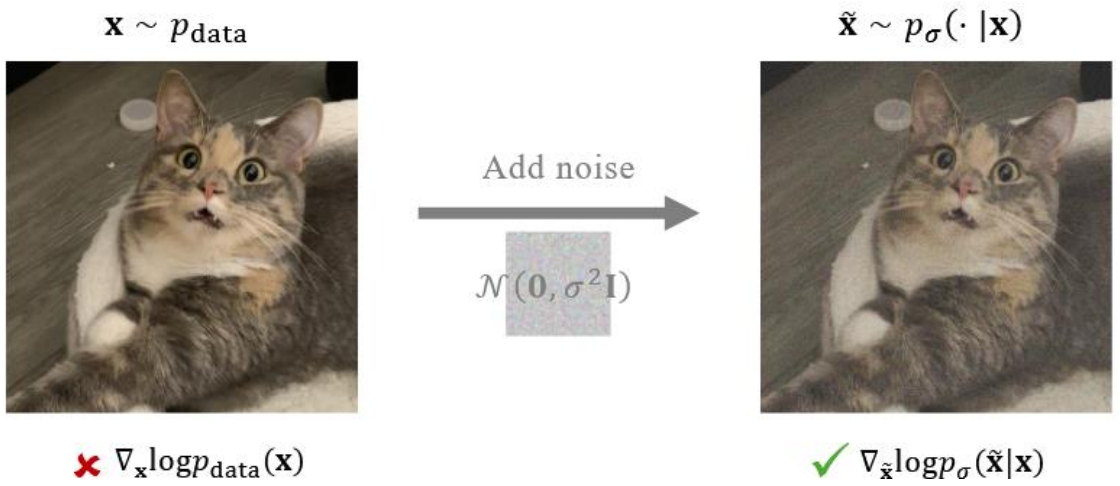
4.4.4 去噪分数匹配

- 如何去得到Score
  - 第二种方法（去噪分数匹配）加入一个极小方差的噪声，可以近似得到梯度

$$\mathcal{L}_{\mathrm{SM}}(\phi) = \frac{1}{2}\mathbb{E}_{\mathbf{x}\sim p_{\mathrm{data}}(\mathbf{x})}\left[\left\|\mathbf{s}_\phi(\mathbf{x}) - \nabla_{\mathbf{x}}\log p_{\mathrm{data}}(\mathbf{x})\right\|_2^2\right]$$

$$\mathcal{L}_{\mathrm{SM}}(\phi;\sigma) := \frac{1}{2}\mathbb{E}_{\tilde{\mathbf{x}}\sim p_\sigma}\left[\left\|\mathbf{s}_\phi(\tilde{\mathbf{x}};\sigma) - \nabla_{\tilde{\mathbf{x}}}\log p_\sigma(\tilde{\mathbf{x}})\right\|_2^2\right]$$

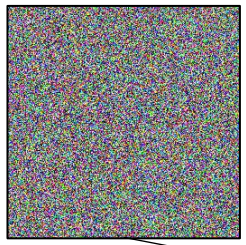
$$\begin{cases} \tilde{\mathbf{x}} = \mathbf{x} + \sigma\boldsymbol{\epsilon}, \\ p_\sigma(\tilde{\mathbf{x}}|\mathbf{x}) = \mathcal{N}(\tilde{\mathbf{x}};\mathbf{x},\sigma^2\mathbf{I}). \\ \nabla_{\tilde{\mathbf{x}}}\log p_\sigma(\tilde{\mathbf{x}}|\mathbf{x}) = \dfrac{\mathbf{x}-\tilde{\mathbf{x}}}{\sigma^2}. \end{cases}$$



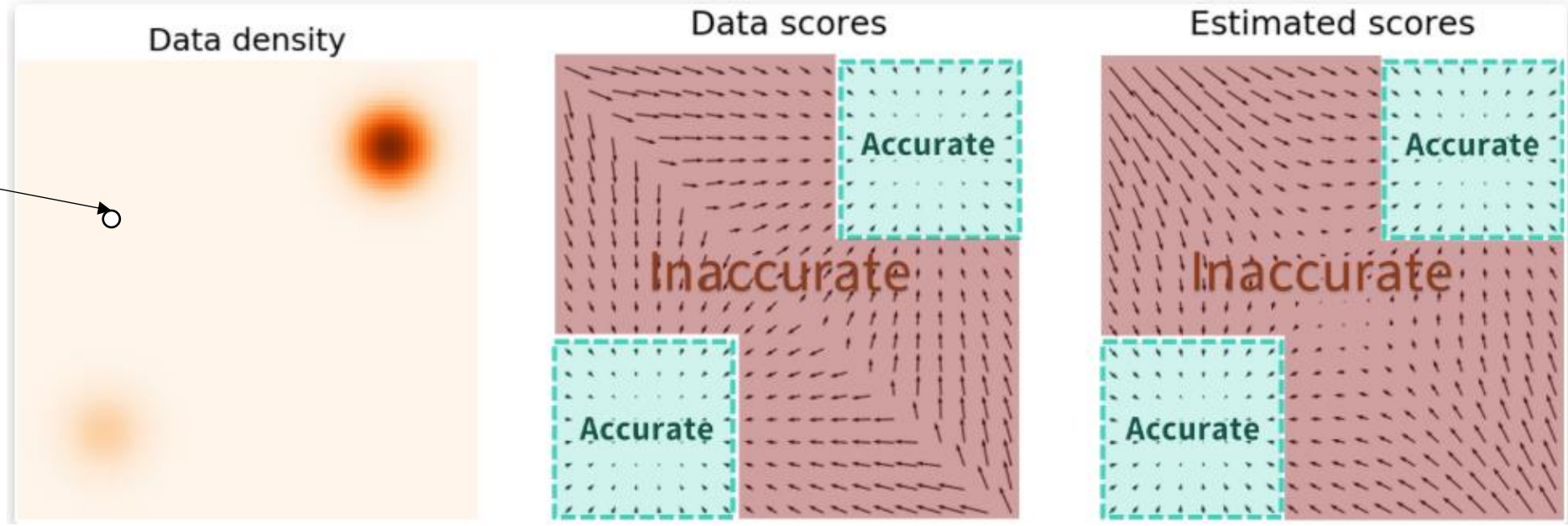
只要σ足够小，学到的 score 就近似原始数据分布的梯度

4.4.4 去噪分数匹配

- **单尺度去噪分数匹配（只加一个小噪声）**
  - **在低密度区域，估计的分数函数不准确，因为很少有数据点可用于计算分数匹配目标**

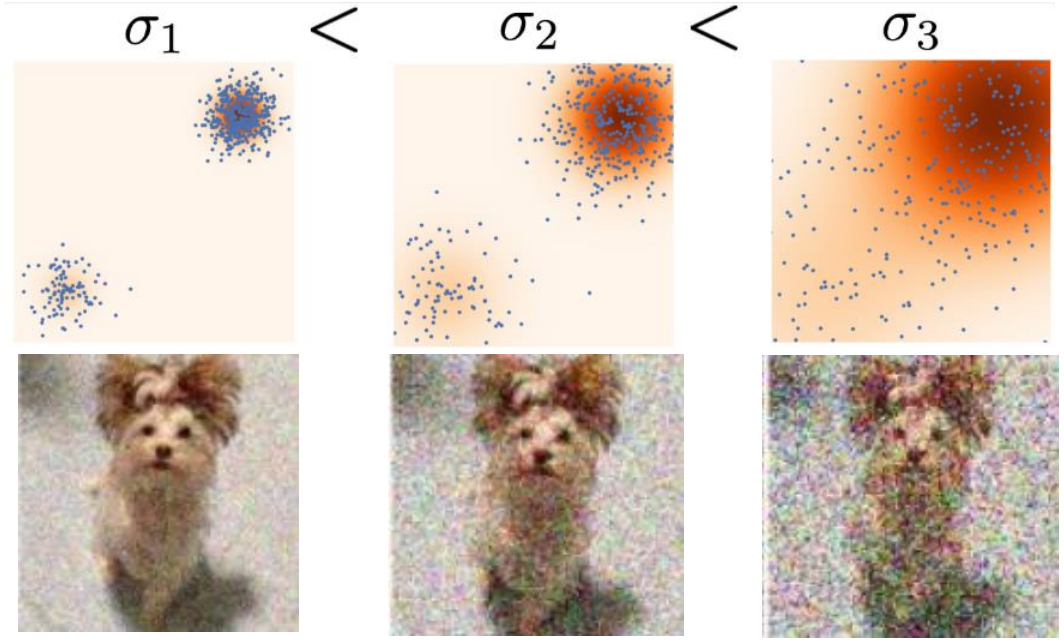


低密度区域
没有训练数据



4.4.4 去噪分数匹配

- **多尺度去噪分数匹配（NCSN: Noise Conditional Score Network）**
  - **使用多个尺度的噪声扰动**
  - **当噪声幅度足够大时，它可以填充低数据密度区域，以提高估计分数的准确性**



$$p_\sigma(\tilde{\mathbf{x}}|\mathbf{x}) = \mathcal{N}(\tilde{\mathbf{x}}; \mathbf{x}, \sigma^2\mathbf{I}).$$

$$\mathcal{L}_{\mathrm{SM}}(\phi;\sigma) := \frac{1}{2}\mathbb{E}_{\tilde{\mathbf{x}}\sim p_\sigma}\left[\left\|\mathbf{s}_\phi(\tilde{\mathbf{x}};\sigma) - \nabla_{\tilde{\mathbf{x}}}\log p_\sigma(\tilde{\mathbf{x}})\right\|_2^2\right]$$

此时模型的输入
为加噪样本和当前的噪声尺度

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \eta\nabla_{\mathbf{x}}\log p_\phi(\mathbf{x}_n) + \sqrt{2\eta}\epsilon_n$$

4.4.4 去噪分数匹配

- **训练**

- **SDE的损失为：** $$\mathcal{L}_{\mathrm{SM}}(\phi;\omega(\cdot)) := \frac{1}{2}\mathbb{E}_{t\sim p_{\mathrm{time}}}\mathbb{E}_{\mathbf{x}_t\sim p_t}\left[\omega(t)\left\|\mathbf{s}_\phi(\mathbf{x}_t,t)-\boxed{\nabla_{\mathbf{x}}\log p_t(\mathbf{x}_t)}\right\|_2^2\right]$$

训练时可以通过该公式前向采样
求导就可以计算真实梯度

	VE SDE	VP SDE
$\mathbf{f}(\mathbf{x},t)$	$\mathbf{0}$	$-\frac{1}{2}\beta(t)\mathbf{x}$
$g(t)$	$\sqrt{\frac{\mathrm{d}\sigma^2(t)}{\mathrm{d}t}}$	$\sqrt{\beta(t)}$
SDE	$\mathrm{d}\mathbf{x}(t)=g(t)\,\mathrm{d}\mathbf{w}(t)$	$\mathrm{d}\mathbf{x}(t)=-\frac{1}{2}\beta(t)\mathbf{x}(t)\,\mathrm{d}t+\sqrt{\beta(t)}\,\mathrm{d}\mathbf{w}(t)$
$p_t(\mathbf{x}_t\|\mathbf{x}_0)$	$\mathcal{N}\left(\mathbf{x}_t;\mathbf{x}_0,\left(\sigma^2(t)-\sigma^2(0)\right)\mathbf{I}\right)$	$\mathcal{N}\left(\mathbf{x}_t;\mathbf{x}_0e^{-\frac{1}{2}\int_0^t\beta(\tau)\,\mathrm{d}\tau},\mathbf{I}-\mathbf{I}e^{-\int_0^t\beta(\tau)\,\mathrm{d}\tau}\right)$
p_{prior}	$\mathcal{N}(\mathbf{0},\sigma^2(T)\mathbf{I})$	$\mathcal{N}(\mathbf{0},\mathbf{I})$

4.4.4 去噪分数匹配

- 采样/推理
  - 蓝色部分，在做反向SDE的迭代

Algorithm 2 PC sampling (VE SDE)

$\mathbf{x}_N \sim \mathcal{N}(\mathbf{0}, \sigma_{\max}^2 \mathbf{I})$
for $i = N-1$ **to** 0 **do**
  $\mathbf{x}'_i \leftarrow \mathbf{x}_{i+1} + (\sigma_{i+1}^2 - \sigma_i^2)\mathbf{s}_{\boldsymbol{\theta}*}(\mathbf{x}_{i+1}, \sigma_{i+1})$
  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  $\mathbf{x}_i \leftarrow \mathbf{x}'_i + \sqrt{\sigma_{i+1}^2 - \sigma_i^2}\mathbf{z}$
  for $j = 1$ **to** M **do**
    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
    $\mathbf{x}_i \leftarrow \mathbf{x}_i + \epsilon_i \mathbf{s}_{\boldsymbol{\theta}*}(\mathbf{x}_i, \sigma_i) + \sqrt{2\epsilon_i}\mathbf{z}$
return $\mathbf{x}_0$

Algorithm 3 PC sampling (VP SDE)

$\mathbf{x}_N \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
for $i = N-1$ **to** 0 **do**
  $\mathbf{x}'_i \leftarrow (2 - \sqrt{1-\beta_{i+1}})\mathbf{x}_{i+1} + \beta_{i+1}\mathbf{s}_{\boldsymbol{\theta}*}(\mathbf{x}_{i+1}, i+1)$
  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  $\mathbf{x}_i \leftarrow \mathbf{x}'_i + \sqrt{\beta_{i+1}}\mathbf{z}$
  for $j = 1$ **to** M **do**
    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
    $\mathbf{x}_i \leftarrow \mathbf{x}_i + \epsilon_i \mathbf{s}_{\boldsymbol{\theta}*}(\mathbf{x}_i, i) + \sqrt{2\epsilon_i}\mathbf{z}$
return $\mathbf{x}_0$

Predictor

Corrector

4.4.4 去噪分数匹配

- 采样/推理
  - 橙色部分，在每次SDE迭代后，使用郎之万方程矫正轨迹

Algorithm 2 PC sampling (VE SDE)

$\mathbf{x}_N \sim \mathcal{N}(\mathbf{0}, \sigma_{\max}^2 \mathbf{I})$
for $i = N-1$ **to** 0 **do**
  $\mathbf{x}'_i \leftarrow \mathbf{x}_{i+1} + (\sigma_{i+1}^2 - \sigma_i^2)\mathbf{s}_{\boldsymbol{\theta}*}(\mathbf{x}_{i+1}, \sigma_{i+1})$
  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  $\mathbf{x}_i \leftarrow \mathbf{x}'_i + \sqrt{\sigma_{i+1}^2 - \sigma_i^2}\mathbf{z}$
  for $j = 1$ **to** M **do**
    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
    $\mathbf{x}_i \leftarrow \mathbf{x}_i + \epsilon_i \mathbf{s}_{\boldsymbol{\theta}*}(\mathbf{x}_i, \sigma_i) + \sqrt{2\epsilon_i}\mathbf{z}$
return $\mathbf{x}_0$

Algorithm 3 PC sampling (VP SDE)

$\mathbf{x}_N \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
for $i = N-1$ **to** 0 **do**
  $\mathbf{x}'_i \leftarrow (2 - \sqrt{1-\beta_{i+1}})\mathbf{x}_{i+1} + \beta_{i+1}\mathbf{s}_{\boldsymbol{\theta}*}(\mathbf{x}_{i+1}, i+1)$
  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  $\mathbf{x}_i \leftarrow \mathbf{x}'_i + \sqrt{\beta_{i+1}}\mathbf{z}$ — Predictor
  for $j = 1$ **to** M **do** — Corrector
    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
    $\mathbf{x}_i \leftarrow \mathbf{x}_i + \epsilon_i \mathbf{s}_{\boldsymbol{\theta}*}(\mathbf{x}_i, i) + \sqrt{2\epsilon_i}\mathbf{z}$
return $\mathbf{x}_0$

4.4.4 概率流ODE

- Probability Flow ODE（PF-ODE）
  - 数学上，对于SDE存在一个确定性 ODE 具有相同的边际分布

SDE

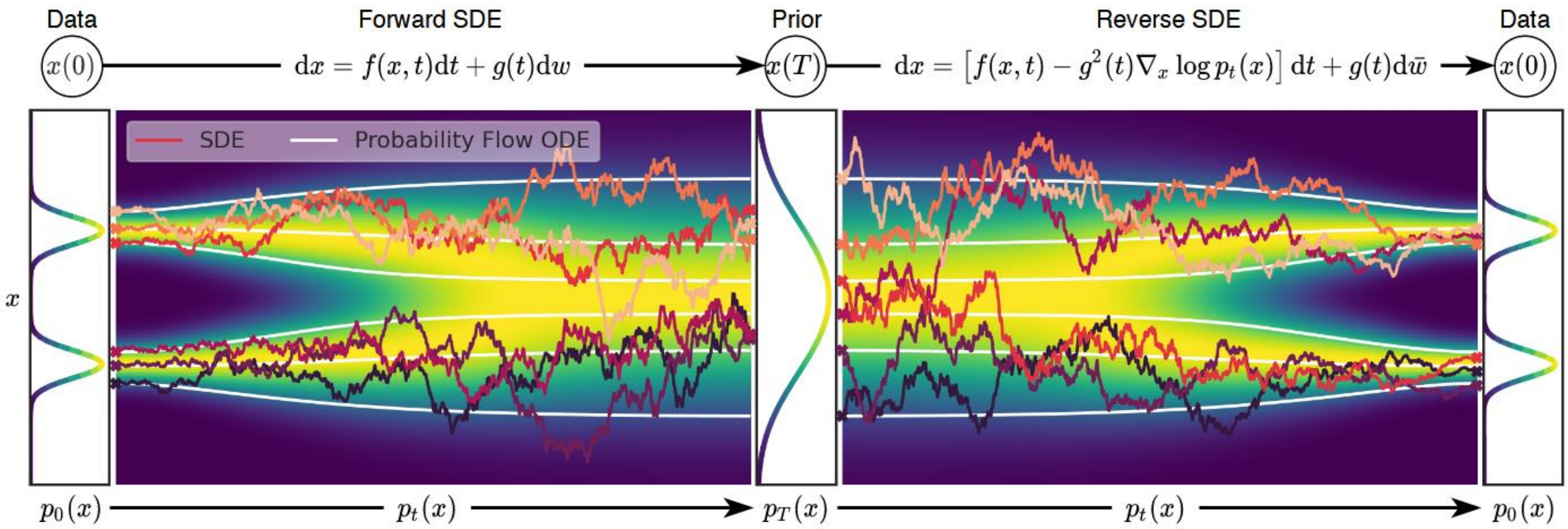
$$\mathrm{d}\bar{\mathbf{x}}(t) = \left[\mathbf{f}(\bar{\mathbf{x}}(t), t) - g^2(t)\nabla_{\mathbf{x}} \log p_t(\bar{\mathbf{x}}(t))\right] \mathrm{d}t + g(t)\,\mathrm{d}\bar{\mathbf{w}}(t),$$
$$\bar{\mathbf{x}}(T) \sim p_{\text{prior}} \approx p_T.$$

PF-ODE

$$\frac{\mathrm{d}}{\mathrm{d}t}\tilde{\mathbf{x}}(t) = \mathbf{f}(\tilde{\mathbf{x}}(t), t) - \frac{1}{2}g^2(t)\nabla_{\mathbf{x}} \log p_t(\tilde{\mathbf{x}}(t)).$$

  - 如此，我们就可以使用目前成熟的ODE求解器，加速采样的过程
  - 例如DPM-solver，DPM-solver++

4.4.4 概率流ODE


Data
Forward SDE
Prior
Reverse SDE
Data
$x(0)$
$\mathrm{d}x = f(x,t)\mathrm{d}t + g(t)\mathrm{d}w$
$x(T)$
$\mathrm{d}x = \left[f(x,t) - g^2(t)\nabla_x \log p_t(x)\right]\mathrm{d}t + g(t)\mathrm{d}\bar{w}$
$x(0)$
SDE
Probability Flow ODE
x
$p_0(x)$
$p_t(x)$
$p_T(x)$
$p_t(x)$
$p_0(x)$

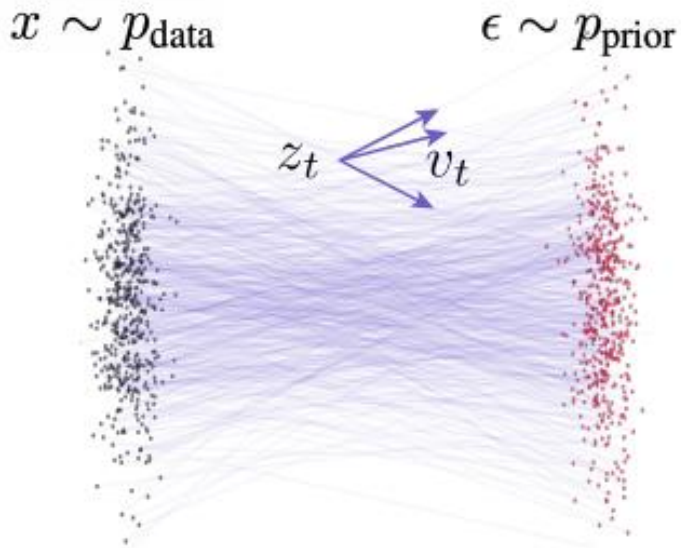
4.4.4 概率流ODE

- **Flow Matching构造中间点的过程，和Diffusion或者NCSN的加噪本质上是一种方式。**

NCSN: $$X_{i+1} = X_i + \sqrt{\sigma_{i+1}^2 - \sigma_i^2}\varepsilon$$

DDPM: $$X_{i+1} = \sqrt{1-\beta_i}\, X_i + \sqrt{\beta_i}\varepsilon$$

Flow Matching: $$\mathrm{Z_t} = (1-t)X + t\varepsilon$$



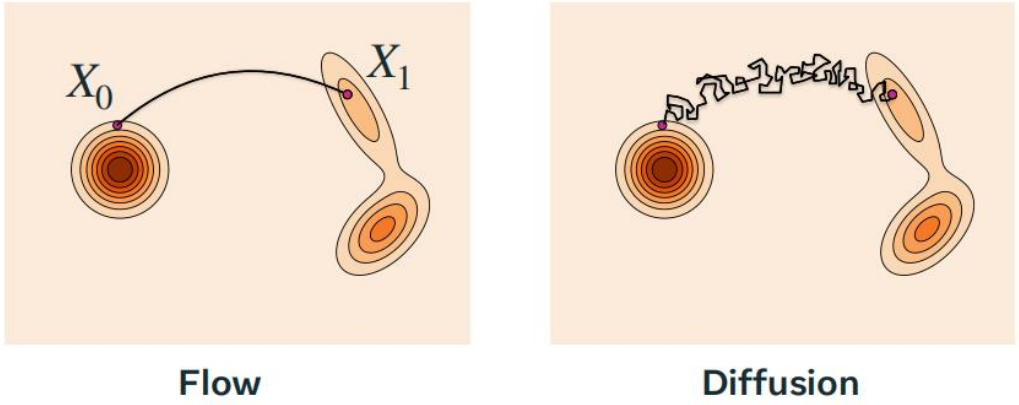
4.4.4 概率流ODE

- 虽然，在Diffusion和Flow matching两类模型，出现了不同的训练目标
- 但是，他们本质上是可以互相推导的
- **分数** 到 **噪声**：

$$\mathbf{s}_\theta(\mathbf{x}_t, t) \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) = \mathbb{E}_{q(\mathbf{x}_0)}[\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{x}_0)] = \mathbb{E}_{q(\mathbf{x}_0)}\left[-\frac{\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)}{\sqrt{1-\bar{\alpha}_t}}\right] = -\frac{\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)}{\sqrt{1-\bar{\alpha}_t}}$$

- **分数** 到 **速度**：

$$v(x,t) = f(x,t) - \tfrac{1}{2} g(t)^2 \nabla_x \log p_t(x)$$



Flow

Diffusion

深度学习中的连续动力学机制

4.1 连续动力学的数学基础

4.2 表示空间中的连续动力学

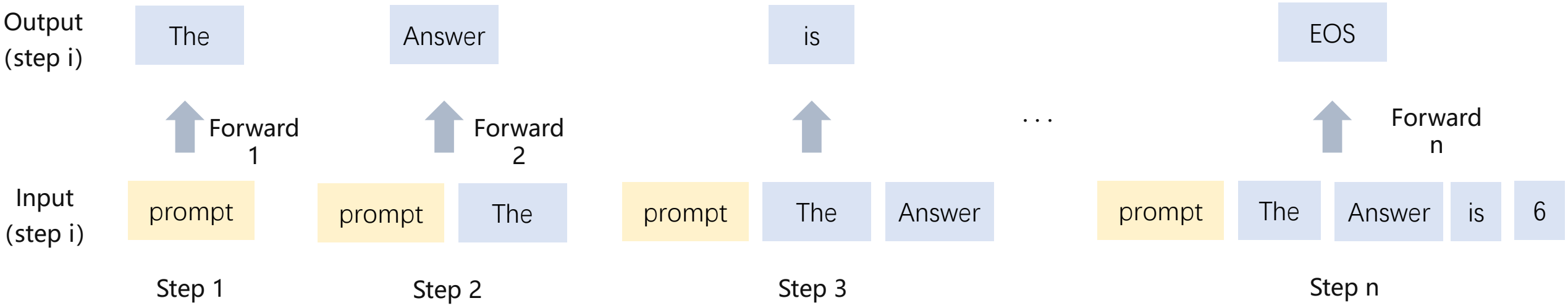
4.3 确定性概率动力学

4.4 随机概率动力学

4.5 离散状态上的连续时间动力学

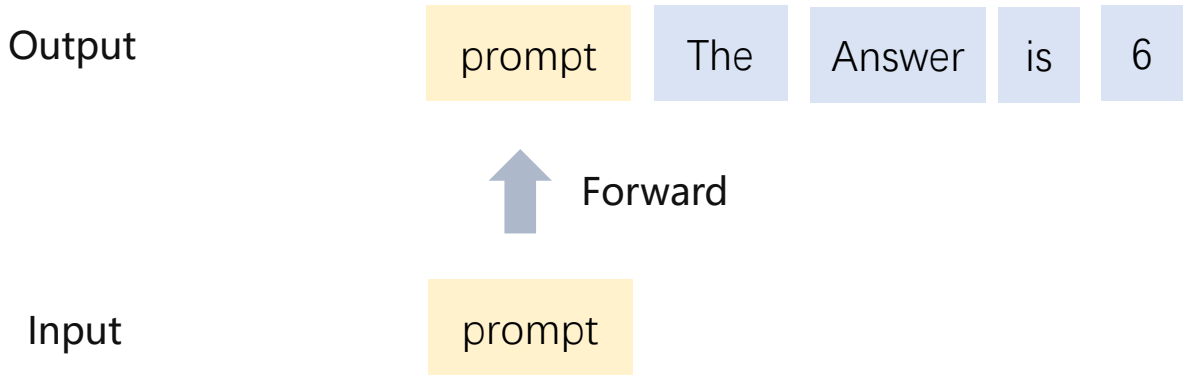
4.5.1 自回归生成与非自回归生成

- 自回归生成
  - 从左到右逐个生成文本token，当前token的生成依赖此前已生成的所有token
  - 具有严格的顺序依赖
  - 建模简单，自然支持变长序列（生成到EOS时结束）；但难以并行生成，长序列推理速度较慢



4.5.1 自回归生成与非自回归生成

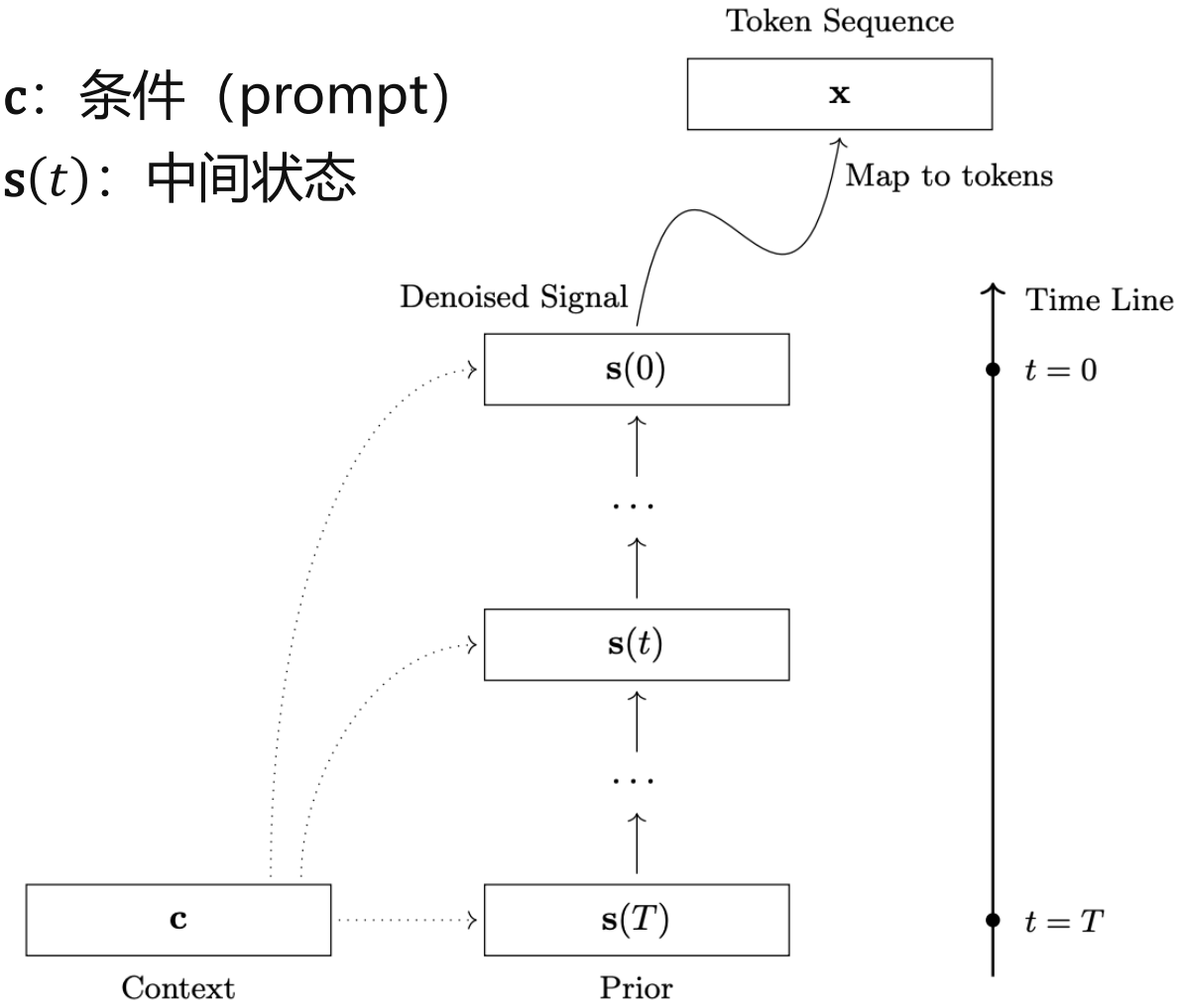
- 非自回归生成
  - 并行预测整个目标序列
  - 各个位置的预测彼此不依赖，可以并行完成
  - 推理速度快，延迟低；但缺少token间因果关系建模，导致生成质量下降



4.5.1 自回归生成与非自回归生成

● 扩散模型作为非自回归生成

- $\mathbf{c}$：条件（prompt）
- $\mathbf{s}(t)$：中间状态

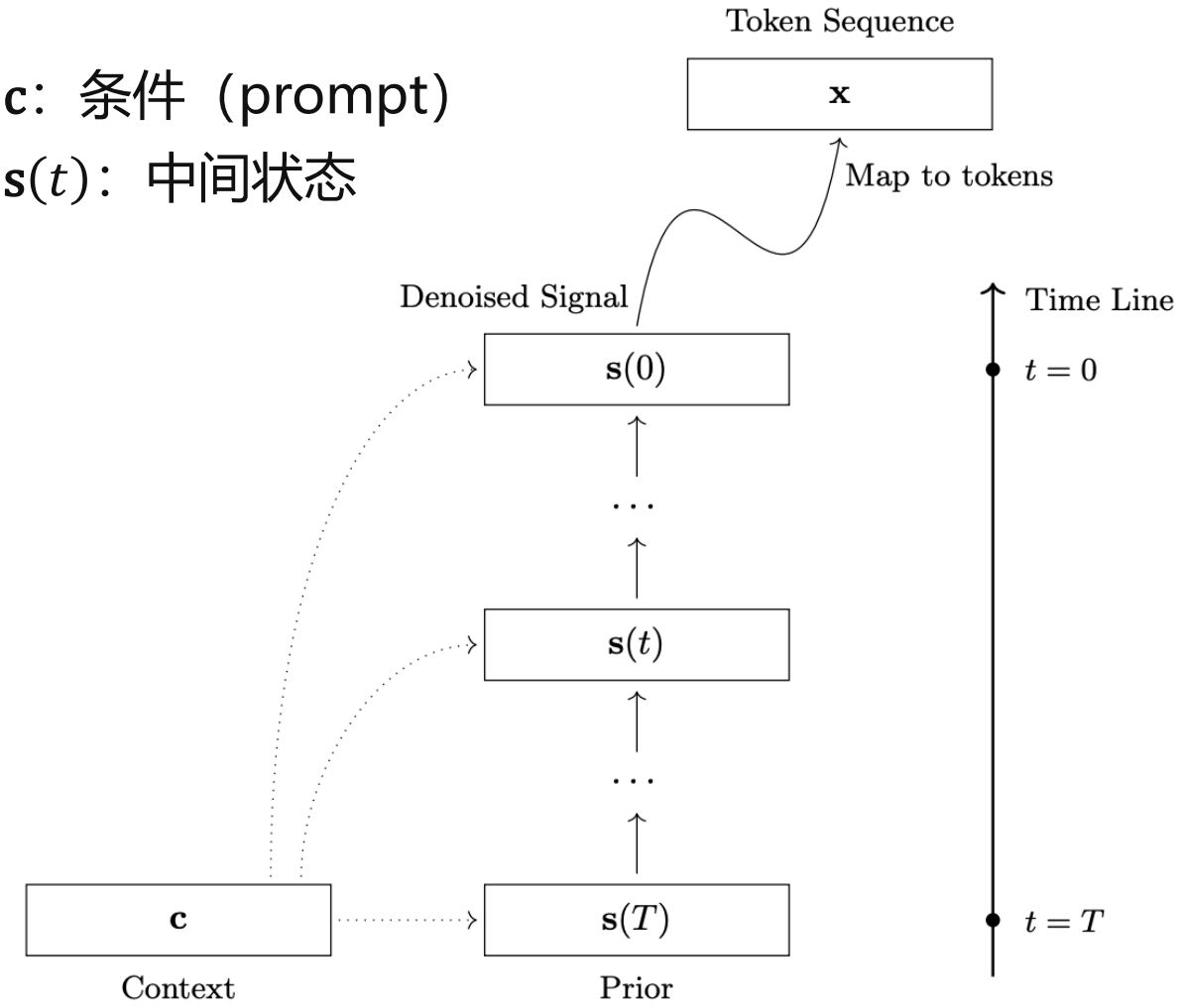


- 主流的文本扩散模型使用非自回归生成范式，在每个去噪步骤中同时更新整个序列
- $\mathbf{s}(T)$：起始状态，一般从先验分布采样（例如高斯向量或随机token序列）
- $\mathbf{c}$：条件prompt

4.5.1 自回归生成与非自回归生成

- 扩散模型作为非自回归生成

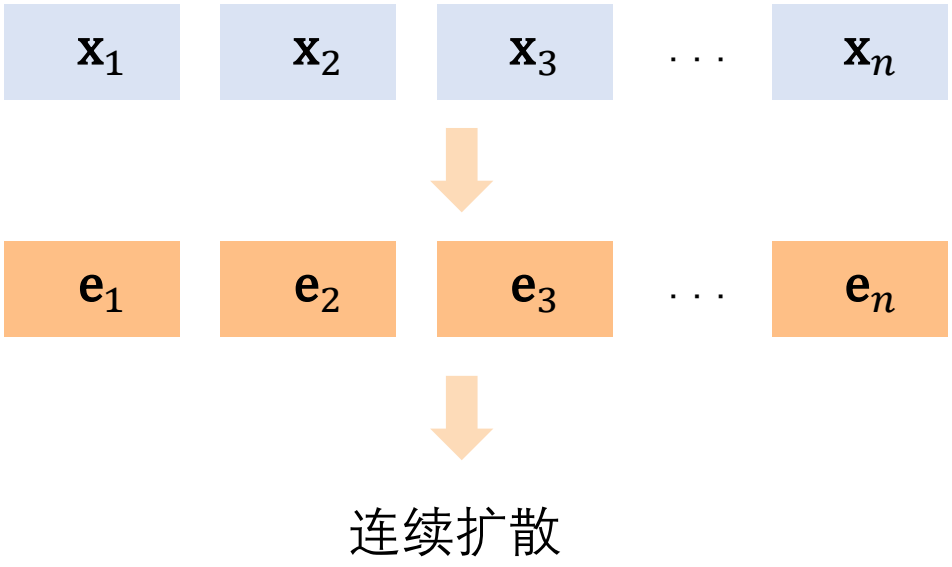
- $\mathbf{c}$：条件（prompt）
- $\mathbf{s}(t)$：中间状态



- $\mathbf{s}(t)$：扩散模型各个时间步的中间状态
- $\mathbf{s}(0)$：去噪终点得到的干净状态，使用取整或argmax等操作映射回token序列

4.5.2 连续空间上的文本扩散

- 为什么要在连续空间上做扩散
  - 文本 token 是离散变量，无法直接应用标准高斯扩散
  - 将 token 映射到连续的 embedding 空间 $(\mathbb{R}^d)$，再进行连续扩散



$$\mathrm{Embed}(\mathbf{x}) = [\mathbf{e}_1, \ldots, \mathbf{e}_n] = \mathbf{e}.$$

$$\mathbf{e}_i \in \mathbb{R}^d, \qquad \mathbf{e} \in \mathbb{R}^{d \times n}.$$

4.5.2 连续空间上的文本扩散

- 前向与反向过程
  - 前向扩散过程（文本的干净表示 -> 噪声）：

$$d\mathbf{s}(t) = f(\mathbf{s}(t), t)\, dt + g(t)\, d\mathbf{w}.$$

    - f：漂移系数
    - g：扩散系数
    - $\mathbf{w}$：标准维纳过程

  - 反向过程：从噪声恢复文本表示

$$d\mathbf{s}(t) = \left[f(\mathbf{s}(t), t) - g(t)^2 \nabla_{\mathbf{s}(t)} \log p_t\left(\mathbf{s}(t)\right)\right] dt + g(t)\, d\bar{\mathbf{w}}.$$

    - $\bar{\mathbf{w}}$：反向时间下的标准维纳过程

4.5.2 连续空间上的文本扩散

- 序列长度预测
  - 文本扩散使用非自回归建模：

$$\mathbf{x}^* = \arg\max_{\mathbf{x}} \Pr(\mathbf{x} \mid \mathbf{c})$$

  - 因此扩散过程开始前，需要确定目标状态的长度n
  - 两种方案：
    - 显式预测长度n
    - 规定最大长度窗口，在生成的EOS处截断

4.5.2 连续空间上的文本扩散

- 输出映射
  - 去噪过程最终得到的是仍然是连续向量，必须映射回离散词表

$$\mathbf{s}(0) = [\hat{\mathbf{e}}_1, \ldots, \hat{\mathbf{e}}_n]$$

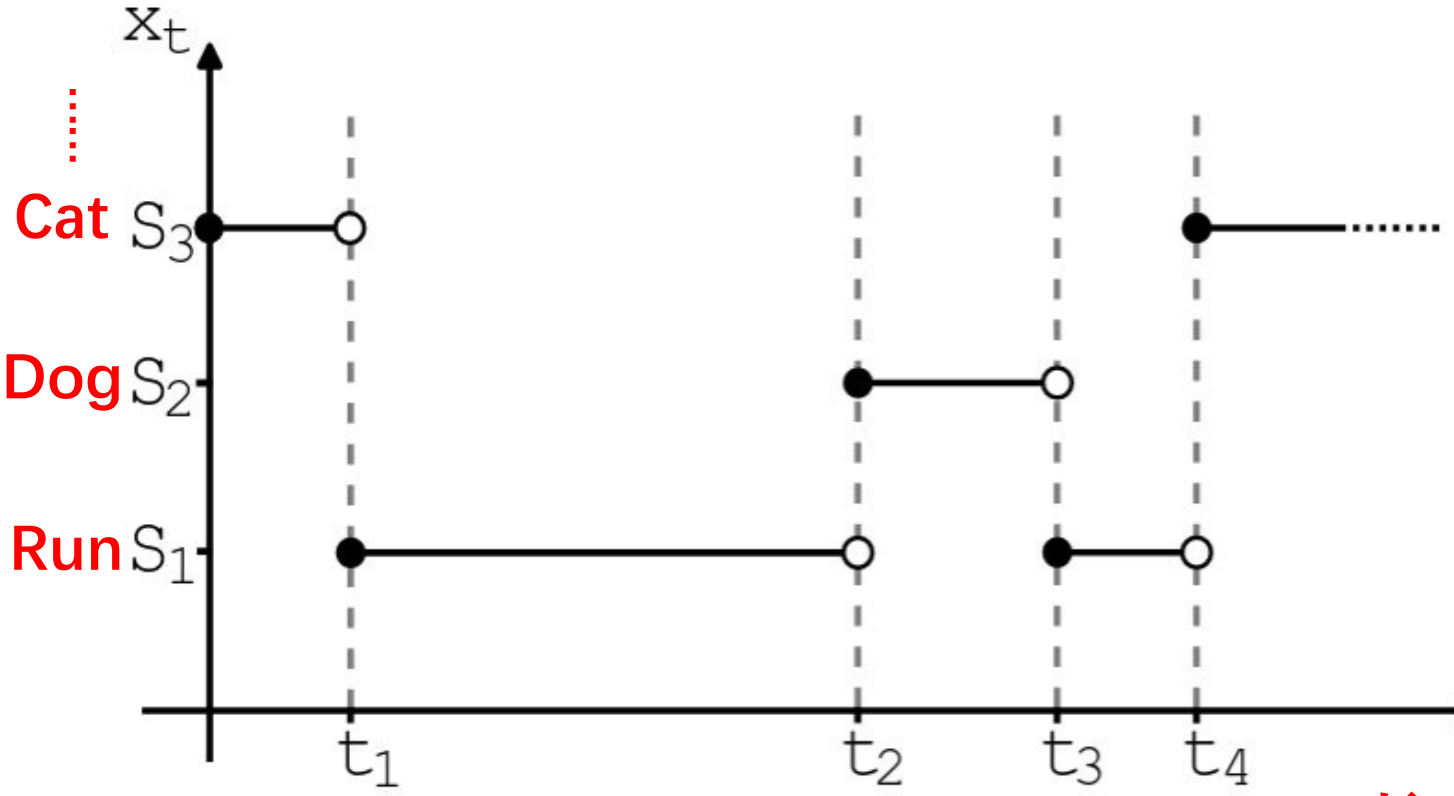
  - 实际上，模型生成的 $\hat{\mathbf{e}}_i$ 未必在词表V中
  - 最简单的方法：最近邻映射

$$x_i = \arg\min_{v \in V} \|\hat{\mathbf{e}}_i - \mathbf{e}_v\|_2^2.$$

  - 问题：对词表本身而言，这种基于最近邻的舍入未必是有意义的，往往需要加入额外的约束或使用额外设计的更复杂的输出映射
  - 所以：下面我们需要探究，如何在离散的Token空间上，引入扩散模型

4.5.3 词元空间上的文本扩散

- 词元空间：不存在**Diffusion/SDE**方程，也不存在**Flow/ODE**方程
  - 但是：学习流匹配与去噪的原理可以推广到离散数据！
  - 数学模型：连续时间马尔可夫链（CTMCs）



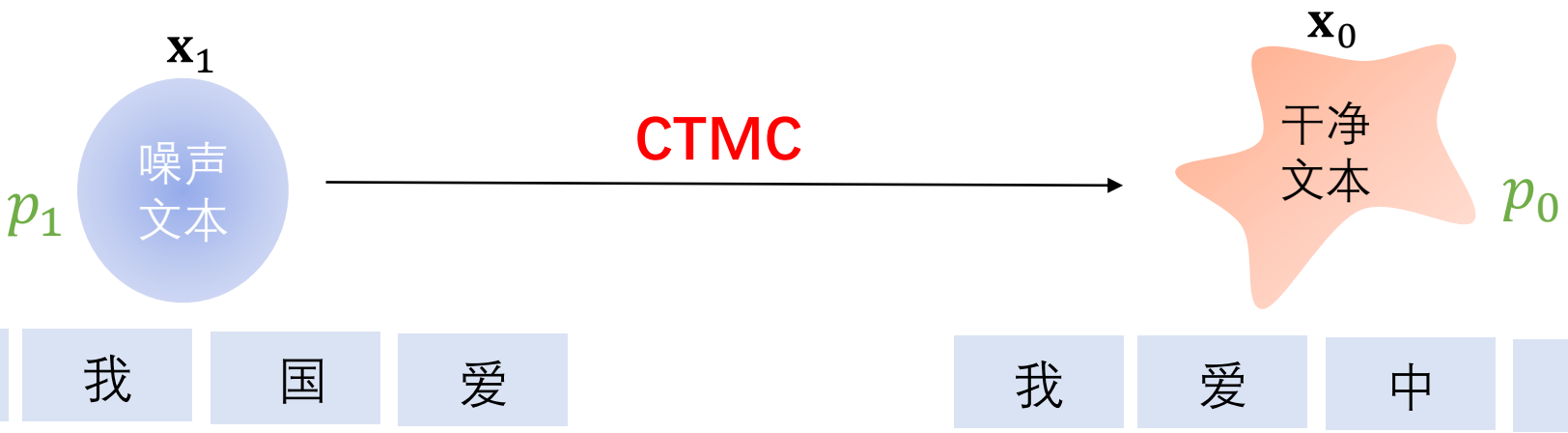
- X_t任意时刻只能取**一个合法 token**
- token 会**保持一段随机时间**，然后瞬时跳转
- 时间 t连续，但样本轨迹是**阶梯状**的。

时间是连续的

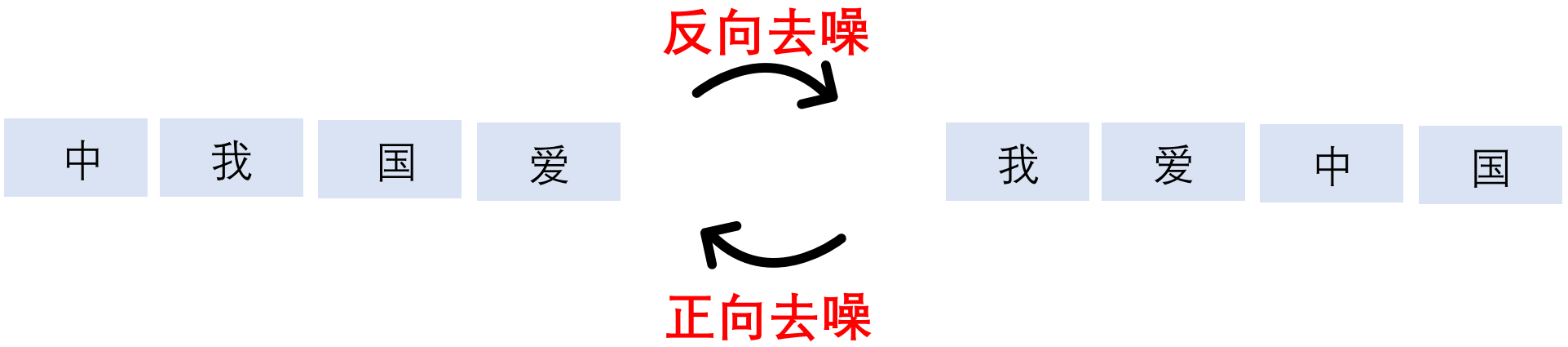
连续的是时间，离散的是状态。专门描述这种跳跃轨迹的数学工具就是 CTMC

4.5.3 词元空间上的文本扩散

- 目标：通过CTMC将“噪声文本”转换成“干净文本”

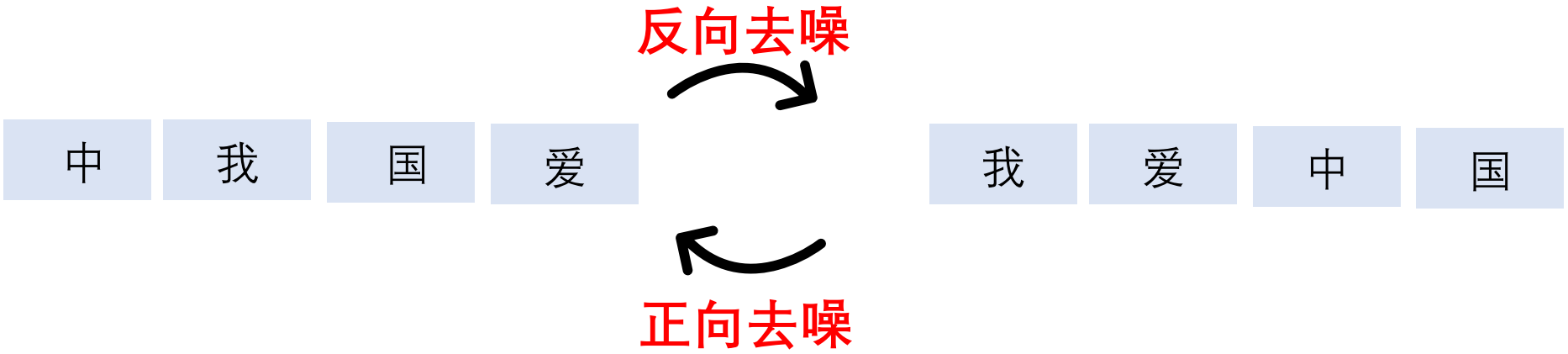


- 目标：与之前的SDE一样，同样有着前向加噪和反向去噪的阶段



4.5.3 词元空间上的文本扩散

- 目标：与之前的SDE一样，同样有着前向加噪和反向去噪的阶段



- 方法：不同于ODE中的速度场，离散空间中的通过**概率转移矩阵**来完成加噪和去噪阶段

$$\frac{dp_t(\mathbf{x})}{dt} = \sum_{\mathbf{y}\in V^n} p_t(\mathbf{y})[\mathbf{Q}^{\text{seq}}(t)]_{\mathbf{yx}},$$

概率转移矩阵

x和 y 表示离散空间V^n中的特定状态（**即 token 序列**）

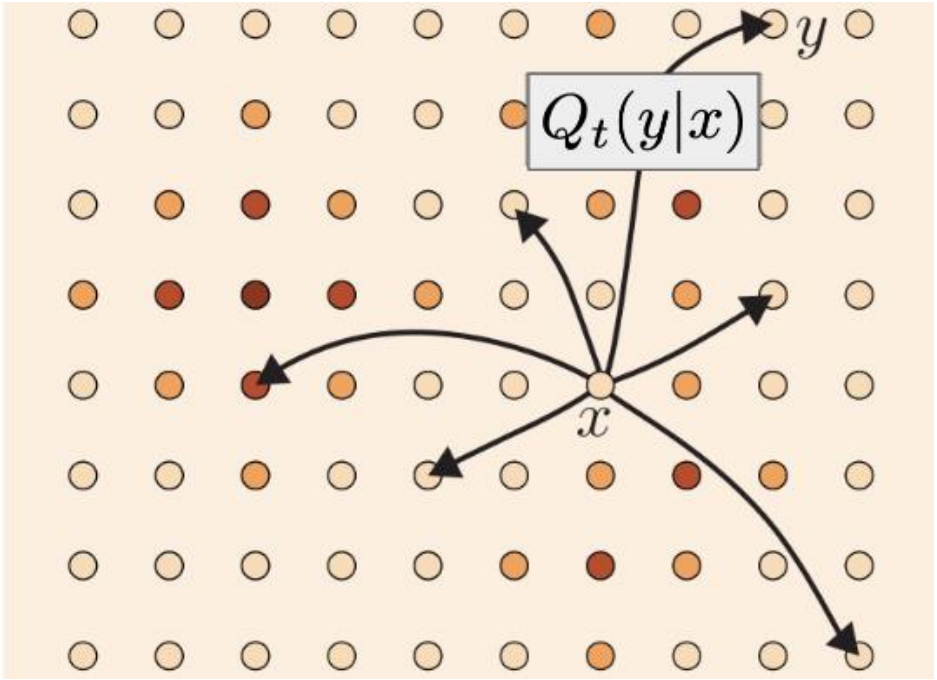
4.5.3 词元空间上的文本扩散

- 方法：离散空间中的通过**概率转移矩阵**来完成加噪和去噪阶段

$$\frac{dp_t(\mathbf{x})}{dt} = \sum_{\mathbf{y}\in V^n} p_t(\mathbf{y})[\mathbf{Q}^{\text{seq}}(t)]_{\mathbf{yx}},$$

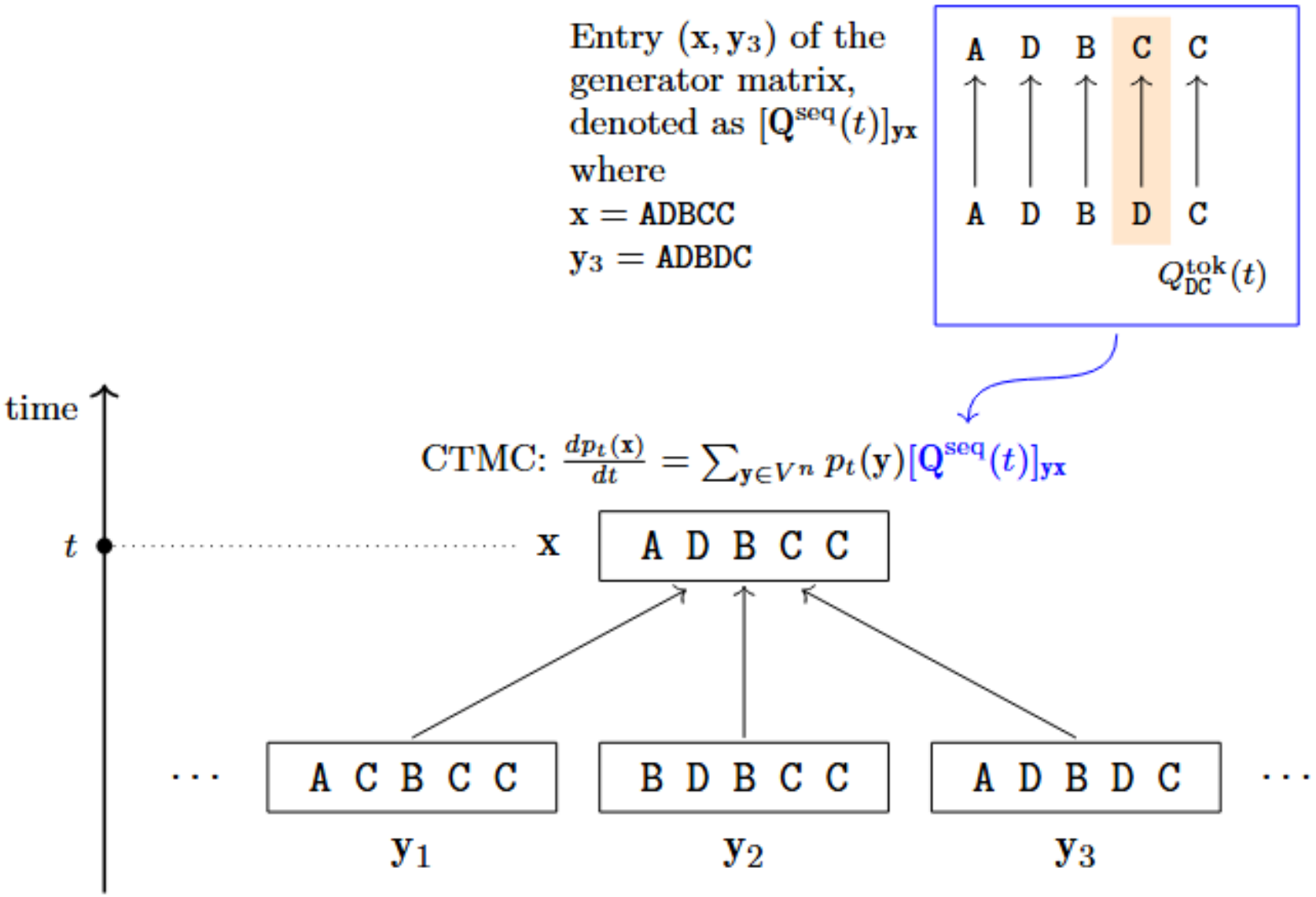
概率转移矩阵

x和 y 表示离散空间V^n中的特定状态（即 token 序列）



4.5.3 词元空间上的文本扩散

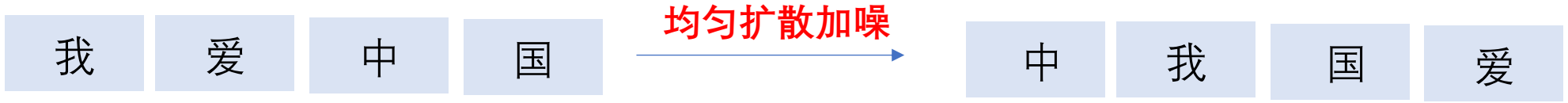
- 简单实例



4.5.3 词元空间上的文本扩散

- 前向过程: 并非添加高斯噪声，而是通过随机token替换来破坏文本，由 $\mathbf{Q}^{\text{tok}}(t)$ 决定

  - 均匀扩散：Token以均匀速率跳转到词表中的任意其他Token，最终将分布推向一个词表V上的均匀分布



  - 掩码扩散：Token以特定速率转移至特殊的 [MASK] 标记且不可逆



4.5.3 词元空间上的文本扩散

- 反向过程: 与之前的SDE的性质类似，如果前向过程是一个具有生成器 $\mathbf{Q}^{\text{tok}}(t)$ 的CTMC，那么从T到0反向过程 也是一个CTMC，反向的生成器的概率矩阵如下

$$\tilde{Q}_{vu}^{\text{tok}}(t) = Q_{uv}^{\text{tok}}(t)\frac{p_t(u)}{p_t(v)}$$

类似于SDE的得分函数 引导生成过程朝向高概率的 token

$$\nabla \log p_t(\mathbf{x})$$

- 其中 u, v ∈ V 代表词汇表中的特定 token 值。这里，$\tilde{Q}_{vu}^{\text{tok}}(t)$ 表示反向过程中从值 v 转移到值 u 的速率，$p_t(v)$ 是 token 在时间 t 取值为 v 的边缘概率。

注意：该方程主要用于提供直觉，说明为什么概率比或后验重加权会引导反向过程朝向高概率的 token。在实践中，我们使用神经网络（通常是 Transformer）对反向动力学进行参数化，该网络预测一个去噪分布，例如 pθ(x(0)|x(t), c)

4.5.3 词元空间上的文本扩散

- D3PM：离散时间上的结构化转移矩阵
  - 该模型训练为直接从噪声输入中预测干净的数据分布 $p_\theta(\mathbf{x}(0)|\mathbf{x}(t), \mathbf{c})$
  - D3PM 使用结构化转移矩阵，可以支持不同类型的离散噪声

$$P_t = \begin{array}{c|ccc} & A & B & [M] \\ \hline A & 0.8 & 0.1 & 0.1 \\ B & 0.1 & 0.8 & 0.1 \\ [M] & 0 & 0 & 1 \end{array}$$

4.5.3 词元空间上的文本扩散

- D3PM：训练时可以计算真实反向后验，这个真实后验作为模型转移概率的监督信号

一个前向加噪步骤　　从干净数据到中间状态的概率

$$q(\mathbf{x}(t-\Delta t)|\mathbf{x}(t),\mathbf{x}(0),\mathbf{c}) = \frac{q(\mathbf{x}(t)|\mathbf{x}(t-\Delta t),\mathbf{c})\,q(\mathbf{x}(t-\Delta t)|\mathbf{x}(0),\mathbf{c})}{q(\mathbf{x}(t)|\mathbf{x}(0),\mathbf{c})}$$

去噪概率　　归一化项

$$q(\mathbf{x}(t)|\mathbf{x}(0),\mathbf{c}) = \prod_{i=1}^{n} q(x_i(t)|x_i(0),\mathbf{c}),$$

4.5.3 词元空间上的文本扩散

- D3PM 中的结构化前向过程
  - 均匀扩散

$$q(x(t)|x(0),\mathbf{c}) \quad = \quad \alpha_t \cdot \mathbb{I}(x(t)=x(0)) + (1-\alpha_t)\cdot\frac{1}{|V|},$$

  - α_t是随时间变化的信号保留率
  - 以 α_t的权重保留原 token
  - 其余概率质量分配到整个词表

4.5.3 词元空间上的文本扩散

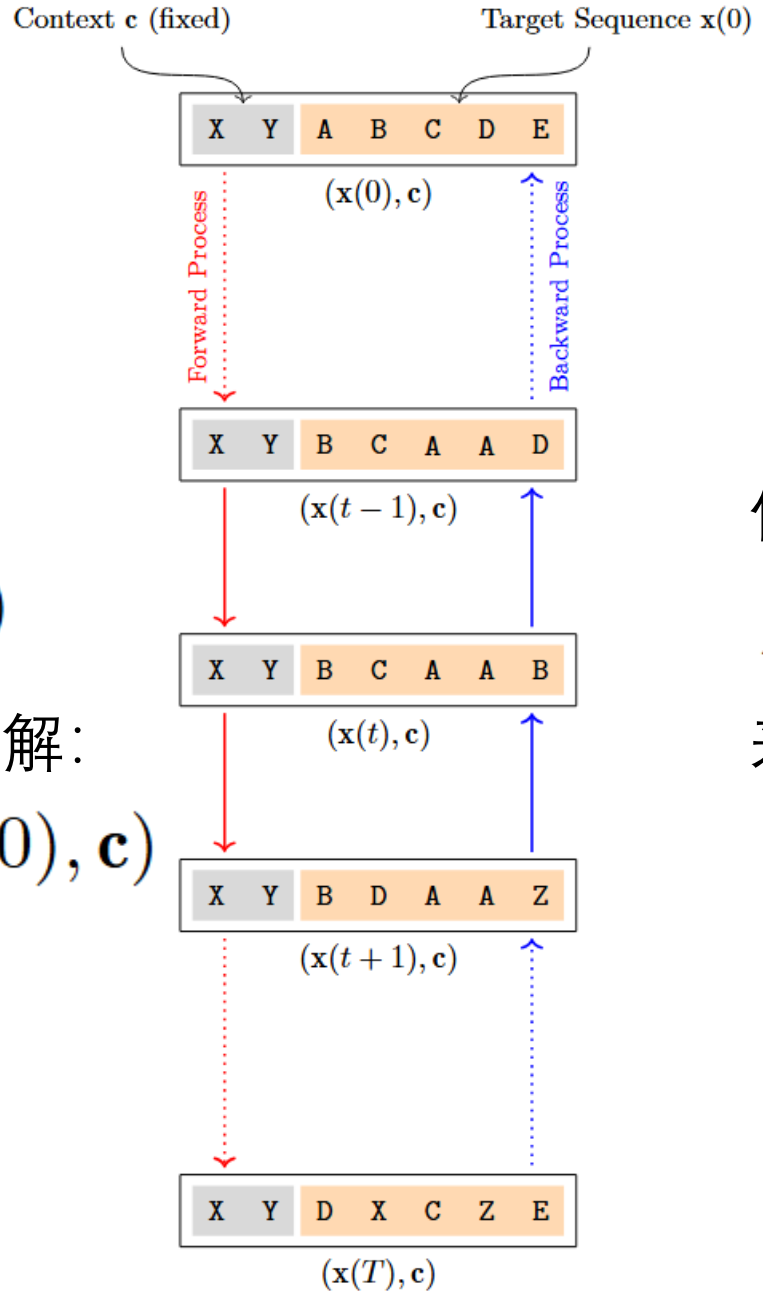
- D3PM 中的结构化前向过程
  - 吸收态掩码扩散

$$q(x(t)|x(0),\mathbf{c}) = \begin{cases} \alpha_t, & \text{if } x(t) = x(0), \\ 1-\alpha_t, & \text{if } x(t) = \texttt{[MASK]}, \\ 0, & \text{otherwise.} \end{cases}$$

- $token$ 要么保留原值
- 要么变成特殊的 {MASK}
- {MASK}是吸收状态

4.5.3 词元空间上的文本扩散

- D3PM 的训练过



计算前向转移概率：

$$q(\mathbf{x}(t)|\mathbf{x}(t-1),\mathbf{c})$$

反向后验概率的闭式解：

$$q(\mathbf{x}(t-1)|\mathbf{x}(t),\mathbf{x}(0),\mathbf{c})$$

使用网络

$$p_\theta(\mathbf{x}(t-1)|\mathbf{x}(t),\mathbf{c})$$

来预测 $\mathbf{x}(t-1)$

为了训练θ，我们最小化

$$q(\mathbf{x}(t-1)|\mathbf{x}(t),\mathbf{x}(0),\mathbf{c})$$

$$p_\theta(\mathbf{x}(t-1)|\mathbf{x}(t),\mathbf{c})$$

这两者之间的KL散度

4.5.3 词元空间上的文本扩散

- 似然需要对所有轨迹边缘化

$$\log p_\theta(\mathbf{x}(0)|\mathbf{c}) = \log \sum_{\mathbf{x}(1:T)} q(\mathbf{x}(1:T)|\mathbf{x}(0),\mathbf{c}) \frac{p_\theta(\mathbf{x}(0:T)|\mathbf{c})}{q(\mathbf{x}(1:T)|\mathbf{x}(0),\mathbf{c})}$$

$$= \log \mathbb{E}_{q(\mathbf{x}(1:T)|\mathbf{x}(0),\mathbf{c})} \left[\frac{p_\theta(\mathbf{x}(0:T)|\mathbf{c})}{q(\mathbf{x}(1:T)|\mathbf{x}(0),\mathbf{c})} \right].$$

- 对岁所有中间轨迹求和
- 在分子和分母前引入前向轨迹分布q，把求和写成期望。

4.5.3 词元空间上的文本扩散

- 使用 Jensen 不等式构造证据下界

$$\log p_\theta(\mathbf{x}(0)|\mathbf{c}) \quad \geq \quad \mathbb{E}_{q(\mathbf{x}(1:T)|\mathbf{x}(0),\mathbf{c})}\left[\log \frac{p_\theta(\mathbf{x}(0:T)|\mathbf{c})}{q(\mathbf{x}(1:T)|\mathbf{x}(0),\mathbf{c})}\right].$$

  - 原始对数似然中，对数位于期望外部。应用 Jensen 不等式后，将对数移到期望内部，得到一个可以通过前向过程采样估计的似然下界，也就是 ELBO。

4.5.3 词元空间上的文本扩散

- D3PM 的负 ELBO

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q(\mathbf{x}(1:T)|\mathbf{x}(0),\mathbf{c})}\Bigg[-\log p(\mathbf{x}(T)) + \sum_{t=1}^{T} \text{KL}\Big(q(\mathbf{x}(t-1)|\mathbf{x}(t),\mathbf{x}(0),\mathbf{c}) \,\|\, p_\theta(\mathbf{x}(t-1)|\mathbf{x}(t),\mathbf{c})\Big)\Bigg],$$

- D3PM 首先定义一个可计算的离散前向过程；训练时利用 X(0)计算真实反向后验；再通过逐时间步的 KL 散度训练反向模型。

4.5.3 词元空间上的文本扩散

- **掩码扩散模型：以{MASK} 为吸收态**
  - 向过程逐步将原始序列中的标记替换为 [MASK]。它从一个完全观测到的序列开始，逐渐破坏信息，直到整个序列变为仅包含掩码的序列。

$$\mathbf{x}(0) \longrightarrow \mathbf{x}(1) \longrightarrow \cdots \longrightarrow \mathbf{x}(T) = [\mathrm{MASK}]^n$$

  - 生成器将概率质量逐渐移向 [MASK]
  - 一旦 token 进入 [MASK]，前向过程中就保持在该状态
  - 条件 c始终不被掩码

4.5.3 词元空间上的文本扩散

- **掩码扩散模型：前向过程：每个位置独立进入吸收态**
- 掩码扩散可以写为每个标记的吸收转移。

$$q(\mathbf{x}(t)|\mathbf{x}(0),\mathbf{c}) = \prod_{i=1}^{n} q(x_i(t)|x_i(0),\mathbf{c}),$$

- **其中单标记前向核为**

$$q(x_i(t)|x_i(0),\mathbf{c}) = \alpha_t \cdot \mathbb{I}(x_i(t)=x_i(0)) + (1-\alpha_t)\cdot \mathbb{I}(x_i(t)=\texttt{[MASK]}).$$

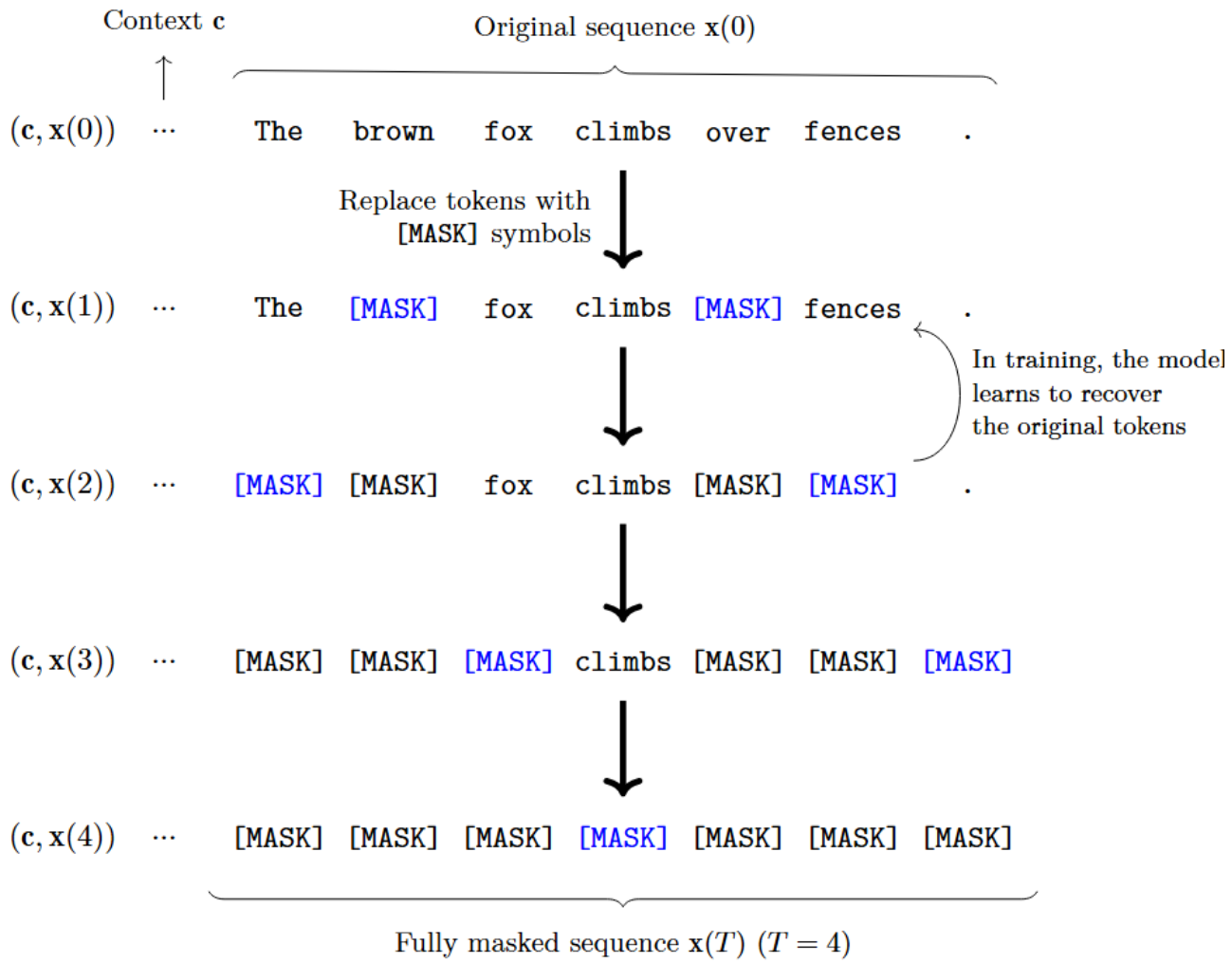
- $\alpha_t \in [0,1]$ 是单调递减的调度函数
- 以概率 α保留原 token
- 以概率 1-α将其变为[MASK]

4.5.3 词元空间上的文本扩散

- 掩码扩散模型：前向加噪等价于采样一个掩码向量

$$m_{t,i} \sim \text{Bernoulli}(1-\alpha_t),$$

$$x_i(t) = \begin{cases} \texttt{[MASK]}, & m_{t,i} = 1, \\ x_i(0), & m_{t,i} = 0. \end{cases}$$



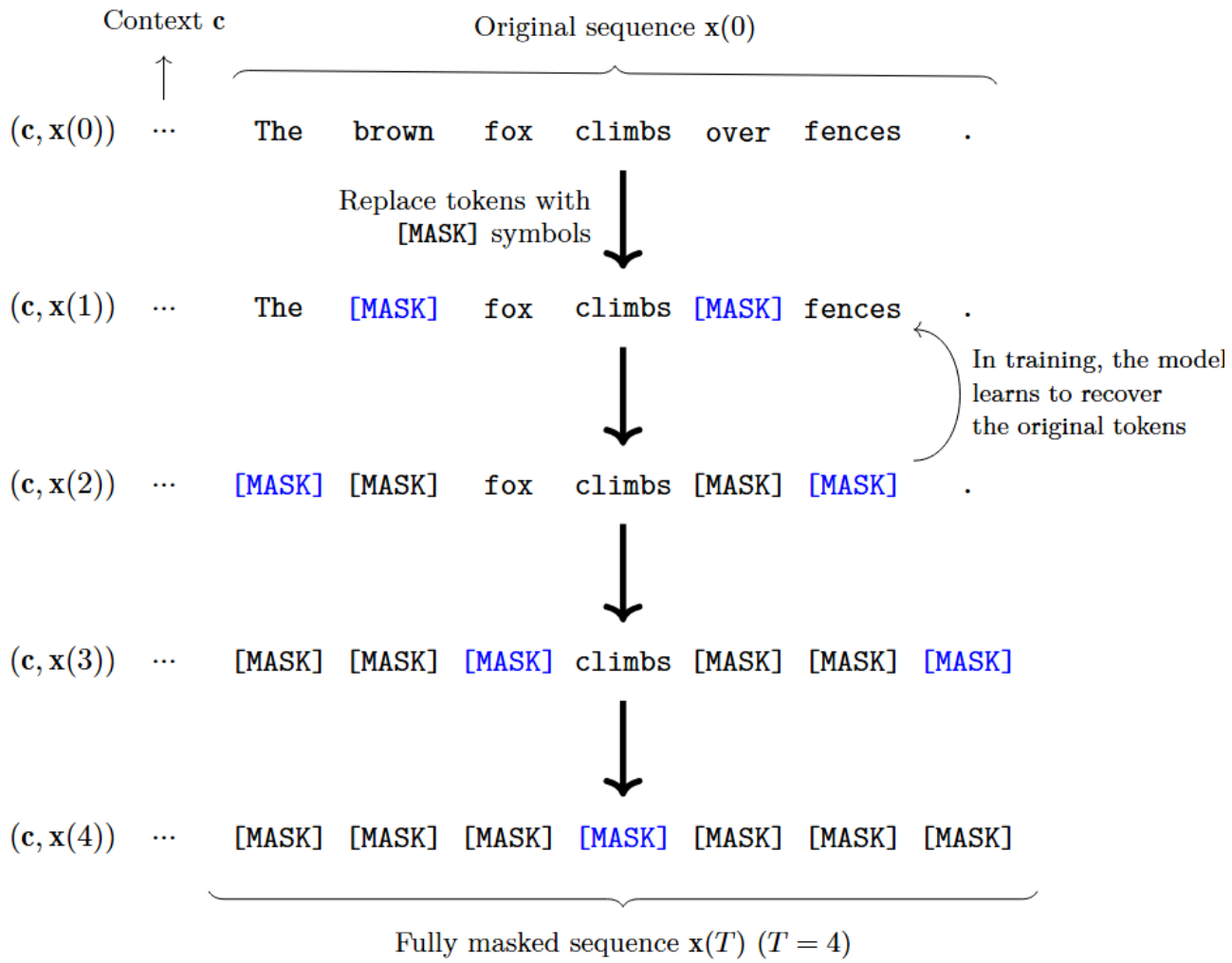
- 扩散轨迹可以解释为重复增加掩码位置的集合，直到最终所有位置都被掩码

4.5.3 词元空间上的文本扩散

- 掩码扩散模型：前向加噪等价于采样一个掩码向量

$$m_{t,i} \sim \text{Bernoulli}(1-\alpha_t),$$

$$x_i(t) = \begin{cases} \texttt{[MASK]}, & m_{t,i} = 1, \\ x_i(0), & m_{t,i} = 0. \end{cases}$$



- 扩散轨迹可以解释为重复增加掩码位置的集合，直到最终所有位置都被掩码

4.5.3 词元空间上的文本扩散

- 掩码扩散模型：反向过程：从全 Mask 状态恢复文本

$$p_\theta(\mathbf{x}(t-1)|\mathbf{x}(t),\mathbf{c}), \quad t=T,T-1,\ldots,1,$$

- 去噪网络预测原始序列的分布

$$p_\theta(\mathbf{x}(0)|\mathbf{x}(t),\mathbf{c}) = \prod_{i=1}^{n} p_\theta(x_i(0)|\mathbf{x}(t),\mathbf{c}).$$

- 反向步骤只更新被掩码的位置

$$p_\theta(x_i(t-1)|\mathbf{x}(t),\mathbf{c}) = \begin{cases} \mathbb{I}(x_i(t-1)=x_i(t)), & \text{if } x_i(t)\neq \texttt{[MASK]}, \\ \pi_{\theta,t,i}(\cdot|\mathbf{x}(t),\mathbf{c}), & \text{if } x_i(t)=\texttt{[MASK]}, \end{cases}$$

$$\pi_{\theta,t,i}(v|\mathbf{x}(t),\mathbf{c}) = p_\theta(x_i(0)=v|\mathbf{x}(t),\mathbf{c}), \quad v\in V,$$

4.5.3 词元空间上的文本扩散

- 掩码扩散模型：训练目标，恢复被掩码位置的原始token
  - 先定义掩码索引集

$$\mathcal{M}(t) \quad = \quad \{i \in \{1, \ldots, n\} : x_i(t) = \texttt{[MASK]}\}.$$

  - 损失函数则定义为掩码位置上的条件负对数似然

$$\begin{aligned} & \mathcal{L}_{\text{denoise}}(\theta) \\ = \quad & \mathbb{E}_{(\mathbf{x}(0),\mathbf{c})\sim p_{\text{data}}} \, \mathbb{E}_{t\sim U[T]} \, \mathbb{E}_{\mathbf{x}(t)\sim q(\mathbf{x}(t)|\mathbf{x}(0),\mathbf{c})} \left[- \sum_{i\in\mathcal{M}(t)} \log p_\theta(x_i(0)|\mathbf{x}(t),\mathbf{c}) \right]. \end{aligned}$$

4.5.3 词元空间上的文本扩散

- 掩码扩散模型：对不同时间步重新加权
  - 可选地，可以对时间步进行重新加权以强调某些噪声水平

$$\mathcal{L}_{\text{denoise}}(\theta) = \mathbb{E}\left[\gamma_t \cdot \left(-\sum_{i \in \mathcal{M}(t)} \log p_\theta(x_i(0)|\mathbf{x}(t), \mathbf{c})\right)\right],$$

  - 小的 t 对应轻度损坏（即较容易的去噪），而大的 t 对应重度损坏（即较难的去噪）。权重 γt 可以用来平衡这些状态。

4.5.3 词元空间上的文本扩散

- 掩码扩散模型：MDM 的采样与去掩码调度
  - 首先，从全 Mask 状态开始

$$\mathbf{x}(T) = [\text{MASK}]^n,$$

  - 迭代的采样

$$\mathbf{x}(t-1) \sim p_\theta(\mathbf{x}(t-1)|\mathbf{x}(t), \mathbf{c}), \quad t = T, T-1, \ldots, 1.$$

  - 逐位置更新

$$x_i(t-1) = \begin{cases} x_i(t), & x_i(t) \neq [\text{MASK}], \\ \text{Cat}(\pi_{\theta,t,i}(\cdot|\mathbf{x}(t), \mathbf{c})), & x_i(t) = [\text{MASK}], \end{cases}$$

4.5.3 词元空间上的文本扩散

- 掩码扩散模型：MDM 的采样与去掩码调度
  - 实践中不一定一次填满所有 Mask

$$\mathcal{U}(t) \subseteq \mathcal{M}(t)$$

  - 只更新选中的位置

$$x_i(t-1) = \begin{cases} x_i(t), & i \notin \mathcal{U}(t), \\ \mathrm{Cat}(\pi_{\theta,t,i}(\cdot|\mathbf{x}(t),\mathbf{c})), & i \in \mathcal{U}(t). \end{cases}$$

  - 例子：一个简单的选择是确定性调度，即每步揭示固定的比例（例如，在 t 上呈线性或余弦变化），使得掩码标记的期望数量从 n 减少到 0

结论与未来方向

总结

- 在本文中，我们介绍了常微分方程（ODEs）的基本概念，并探讨了其在离散过程（如神经网络架构）和连续过程（如流模型）建模中的应用。
- 我们进一步将讨论扩展到更广泛的微分方程框架，包括随机微分方程（SDEs），重点阐述了这些数学工具如何定义视觉与语言扩散建模中的生成过程。
- 在此过程中，我们解决了诸如动力系统刚性和标记空间离散扩散复杂性等实际挑战。

未来

- **缩放与数值稳定性问题：**随着模型规模扩展至数十亿参数，确保底层常微分方程的数值稳定性变得日益困难。未来的研究可以探索更鲁棒的自适应求解器和正则化技术，以处理大规模架构中的刚性动力学问题，同时避免过高的计算开销

- **高效的推理方法：**减少采样步数仍然是实际部署的主要优先事项。轨迹校正、一致性蒸馏和专用高阶求解器等技术为实现仅需一步或少数几步的高保真生成提供了途径，前沿包括Meanflow，DMD，Shortcut model等一系列少步方法。

- **基于连续流的语言模型：**将连续建模应用于文本等离散数据并不直接。探索原生的离散流匹配和更复杂的连续松弛方法，可能催生出更有效的非自回归语言模型，从而在质量上媲美自回归模型

未来

- **模型可解释性：**分析固定点、吸引子以及学习轨迹的几何结构，可以更深入地理解神经网络在现实世界问题上的工作原理

- **动力系统观点扩展到更广泛的学习任务：**不仅限于本文讨论的问题。例如，将优化过程本身视为一个动力系统，可能有助于将训练稳定性、泛化能力与经典稳定性及扰动理论结果联系起来，并可能提供一个统一框架来理解算法选择（例如，步长调度、动量或隐式更新）如何与模型架构及数据相互作用。